SWC3 – Vollständiges Lehrskript: Programmieren in C

C23 · Einführung, Grundlagen, Übersetzung, Speicherverwaltung, Bibliotheken und Interoperabilität

Lehrskript auf Basis der Foliensätze von Erik Pitzer, FH OÖ – Campus Hagenberg, WS 2026/27

7. November 2026 · überarbeitete und ergänzte Fassung

Inhaltsverzeichnis

1 Orientierung: Warum C, welches C und wie dieses Skript zu lesen ist....4
1.1 Historische Einordnung und Standardisierung....4
1.2 Entwurfsziele und typische Einsatzfelder....4
2 C-Einführung: Vom Quelltext zum ersten Programm....7
2.1 Eigenschaften von C und Unterschied zu Java....7
2.2 Das erste vollständige C-Programm....10
2.3 Schlüsselwörter, Arten von Datentypen und Zeichenketten....13
2.4 Ganzzahlen, Größen und Wertebereiche....15
2.5 Gleitkommazahlen, Dezimalzahlen und komplexe Zahlen....18
2.6 Literale, Typumwandlungen und Bezeichner....19
2.7 Deklarationen, Definitionen, Initialisierung und Konstanten....21
2.8 volatile, restrict, Scope, Lebensdauer und Speicherklassen....25
3 C-Grundlagen I: Zusammengesetzte Datentypen, Zeiger und Arrays....31
3.1 Aufzählungen (enum)....31
3.2 Strukturen (struct), Initialisierung und Speicherlayout....31
3.3 Unions, gemeinsame Speicherbereiche und Bitfelder....33
3.4 Zeiger und const-Qualifikation....34
3.5 Arrays: Länge, Initialisierung und Indexzugriffe....36
3.6 Variable-length arrays und Array-to-pointer decay....39
3.7 typedef und Binärformat-Beispiel Ogg....42
4 C-Grundlagen II: Ausdrücke, Typkonvertierungen und Kontrollfluss....45
4.1 Ausdrücke und die wichtigsten Operatoren....45
4.2 Operatorpräzedenz, Assoziativität und Auswertungsreihenfolge....46
4.3 Integer Promotions und übliche arithmetische Umwandlungen....48
4.4 Anweisungen, Blöcke und Verzweigungen....51
4.5 Schleifen, break, continue, goto und longjmp....52
5 C-Grundlagen III: Funktionen, Parameter und Fehlerverhalten....56
5.1 Deklarationen, Definitionen und Funktionsprototypen....56
5.2 Call-by-value und Änderungen über Zeiger....57
5.3 Variadische Funktionen und Rückgabewerte....59
5.4 Die Hauptfunktion main und Programmargumente....61
5.5 printf: Formatzeichen, Breite und Genauigkeit....62
5.6 Funktionszeiger und Callback-Funktionen....65
5.7 Undefiniertes, nicht spezifiziertes und implementationsdefiniertes Verhalten....66
6 Übersetzungsprozess: Präprozessor, Compiler und Linker....68
6.1 Header und Implementierungseinheiten....68
6.2 Acht logische Übersetzungsphasen....68
6.3 Präprozessor: Textersetzung, Makros und Konditionen....70
6.4 Bedingte Übersetzung, eingebaute Makros und Include-Guards....73
6.5 Objektdateien, Symboltabellen und Linken....74
7 Speicherverwaltung: Speicherbereiche, Lebensdauer und sichere Zugriffe....78
7.1 Überblick über Code, Data, BSS, Heap und Stack....78
7.2 Ausführbarer Code und gemeinsam genutzte Bibliotheken....78
7.3 Globale Objekte und lokale statische Variablen....80
7.4 Stackframes, Funktionsaufrufe und ABI....80
7.5 Lebensdauer lokaler Variablen, Dangling Pointer und Rekursion....82
7.6 Dynamischer Speicher mit malloc, calloc und free....84

7.7 Sanitizer und Memory-Profiling 87
8 Bibliotheken, Linkverfahren und Interoperabilität 89
8.1 Aufbau einer Bibliothek 89
8.2 GMP-Praxisbeispiel: Initialisieren, Multiplizieren, Ausgabe und Aufräumen 89
8.3 Header-Suchpfad, Bibliothekssuchpfad und Linkerfehler 90
8.4 Statische und dynamische Bindung 92
8.5 C als Sprachbrücke: Java Native Interface (JNI) 94
9 Technischer Anhang und Referenz zu den Foliendiagnosemakros 97
9.1 Standardheader und Hilfsbibliothek 97
9.2 _Generic, Typanzeigen und Ausgabeformatwahl 97
9.3 Weitere Ausgabemakros, Arrays, Zahlenbereiche und Compilerparameter 99
10 Querverbindungen und praktische Lernbeispiele 102
10.1 Ein Zeiger- und Speicherbeispiel vollständig begründet 102
10.2 struct und externe Binärdaten: Warum der Unterschied wichtig ist 102
10.3 Wann eine Warnung wirklich einen Fehler verrät 103
11 Quellenhinweise und dokumentierte Korrekturen 104
12 Folienabgleich und Bearbeitungsumfang 105
12.1 Gesamtübersicht der Originalfolien 105
12.2 01-Überblick.pdf – 7 PDF-Seiten 105
12.3 02-C-Einführung(1).pdf – 39 PDF-Seiten 105
12.4 03-C-Grundlagen.pdf – 50 PDF-Seiten 107
12.5 04-C-Übersetzung.pdf – 16 PDF-Seiten 109
12.6 05-C-Speicherverwaltung.pdf – 15 PDF-Seiten 110
12.7 06-C-Bibliotheken.pdf – 12 PDF-Seiten 111
12.8 07-C-Anhang.pdf – 9 PDF-Seiten 111

1 Orientierung: Warum C, welches C und wie dieses Skript zu lesen ist

Dieses Lehrskript behandelt ausschließlich die Programmiersprache **C**. Der übergeordnete Kurs SWC3 enthält außerdem einen eigenen C++-Teil, der hier bewusst ausgeklammert ist. Als Grundlage dienen die C-Foliensätze 02 bis 07; der Überblick 01 wird für Geschichte, Motivation und Aufbau herangezogen. Die Originalfolien sind im jeweiligen Zusammenhang abgebildet. So bleiben auch Tabellen, maschinenabhängige Versuchsausgaben und schematische Darstellungen direkt verfügbar. Im fortlaufenden Text werden die dahinterstehenden Regeln und Ursachen erklärt. Erläuterungen mit der Kennzeichnung **Ergänzende Einordnung** ergänzen den Folieninhalt und sind nicht als wörtliche Aussage der Lehrveranstaltung zu verstehen.

1.1 Historische Einordnung und Standardisierung

C entstand Anfang der 1970er-Jahre bei Bell Labs durch Dennis Ritchie, ausgehend von B und dessen Vorgänger BCPL. B war weitgehend typenlos; C verband eine systemnahe Programmiersprache mit expliziten Typen, Zeigern, Funktionen und Datenstrukturen. Die Veröffentlichung *The C Programming Language* von Brian Kernighan und Dennis Ritchie (1978) prägte die ursprüngliche Praxis, häufig als **K&R C** bezeichnet. Später folgten die standardisierten Fassungen ANSI C89 beziehungsweise ISO C90, C95, C99, C11, C17 und C23. **Standardisierung** bedeutet, dass Sprachregeln und Anforderungen dokumentiert werden, damit verschiedene Compiler dieselbe Bedeutung eines Programms erhalten, soweit der Standard dies festlegt.

C99 brachte beispielsweise `//`-Kommentare, variable-length arrays, komplexe Zahlen und Ganzzahltypen definierter Breite. C11 ergänzte unter anderem Atomics und `_Generic`. C17 war eine vergleichsweise kleine Revision. C23 enthält unter anderem `bool`, `true`, `false`, `nullptr`, `constexpr`, `typeof` und Bitoperationen im Header `<stdbit.h>`. Diese Neuerungen sind ausdrücklich Bestandteil der vorliegenden Folien. **Wichtig:** Ob ein bestimmter Compiler eine C23-Funktion vollständig unterstützt, hängt von seiner Version und seinem Implementierungsstand ab; die Option `-std=c23` allein garantiert nicht die Implementierung jedes optionalen Merkmals.

Die ISO-Standardisierung von C erfolgt in der Arbeitsgruppe **ISO/IEC JTC 1/SC 22/WG 14**. ISO bezeichnet die International Organization for Standardization, IEC die International Electrotechnical Commission und WG die Working Group. Die historische Zeitleiste zeigt auch FORTRAN, LISP, COBOL, Simula, Smalltalk, Objective-C und C++, aber für dieses Skript sind diese Sprachen nur als Kontext relevant.

1.2 Entwurfsziele und typische Einsatzfelder

C ist **imperativ**: Anweisungen legen Zustandsänderungen und Kontrollfluss fest. Es ist **prozedural**: Ein Programm wird überwiegend in Funktionen zerlegt. Die Sprache ist **statisch typisiert**, weil der Compiler die Typen zur Übersetzungszeit kennt. Gleichzeitig lässt C viele explizite oder implizite Umwandlungen und Zugriffe auf Speicher zu; man nennt es daher häufig relativ schwach typisiert. Aus Quelltext erzeugt ein Compiler gewöhnlich nativen Maschinencode, der direkt auf dem Prozessor ausgeführt wird; eine sprachspezifische virtuelle Maschine ist nicht erforderlich.

C bietet eine direkte Schnittstelle zu Hardware, Betriebssystemen und Binärschnittstellen. Vorteile sind kleine Laufzeitabhängigkeiten, Kontrollierbarkeit von Speicher und Ausführungszeit sowie große Portabilität. Dem stehen Gefahren gegenüber: fehlende automatische Bereichsprüfung, nicht initialisierter Speicher, unzulässige Zeiger und manuelle Freigabe dynamisch angelegter Objekte. Diese Risiken sind keine Randdetails, sondern bestimmen den richtigen Umgang mit Arrays, Funktionen und Bibliotheken. Einsatzgebiete umfassen Betriebssystemkomponenten, eingebettete Geräte, Echtzeitsysteme, Datenbanken und die Anbindung anderer Programmiersprachen.

Die Folien beschreiben einen Zielkonflikt zwischen Compilerentwicklung (Optimierung und Übersetzungszeit), Hardware (einfache Implementierung und Kompatibilität) und Softwareentwicklung (Lesbarkeit, Wiederverwendung, Rückwärtskompatibilität und Fehlervermeidung). Der Standard muss gerade deshalb nicht jede Eigenschaft einer konkreten Maschine festschreiben. Die Beispiele zu Typgrößen und Funktionsadressen zeigen im Verlauf, was festgelegt ist und was plattformabhängig bleibt.

Vertiefung · Schritt für Schritt

Warum C im Kurs trotz moderner Hochsprachen behandelt wird: C macht Speicherdarstellung, Übersetzungsprozess und Binärschnittstellen relativ unmittelbar sichtbar. Bei einem Java-Aufruf kann sich ein Objekt im verwalteten Heap befinden und eine Laufzeitumgebung die Speicherfreigabe übernehmen. In C sind die Operationen zur Reservierung und Freigabe dagegen explizit und die Schnittstelle zwischen Header, Objektdatei und Bibliothek tritt offen hervor. Die Folien stellen dafür drei konkurrierende Perspektiven gegenüber: Compilerschreiber möchten Leistung und schnelle Übersetzung, Hardwareentwickler geringe Implementierungskomplexität und Softwareentwickler Portabilität, Lesbarkeit und Fehlervermeidung. Standardisierung versucht, diese Anforderungen miteinander vereinbar zu machen.



Originalfolie 01, PDF-Seite 3: Klassische Programmiersprachen — Timeline

Interessenskonflikte & Spannungsfelder

- Compiler
  - maximale Performance
  - schnelle Übersetzungszeit
  - wenig Interesse an Austauschbarkeit
- Hardware
  - geringer Aufwand
  - hohe Kompatibilität mit verschiedenen Compilern
- Software
  - Rückwärtskompatibilität
  - Lesbarkeit
  - Wiederverwendbarkeit
  - Interoperabilität
  - Fehlervermeidung

Originalfolie 01, PDF-Seite 4: Interessenskonflikte & Spannungsfelder

Motivation für C und C++

- Grundlage vieler modernen Programmiersprachen
  - Syntax Basis für z.B. C#, D, Java, Javascript
- hardwarenahe mit guten Abstraktionskonzepten
- sehr performant
  - z.B. für embedded, Echtzeitsysteme, Game-Engines
- viele existierende Systeme in C
  - Betriebssystemkerne Linux, Windows, MacOS
  - Datenbanken Oracle, PostgreSQL, SQLite, Redis
- Compiler für (fast)
  - jedes System und
  - jede Architektur
- Grundlegendes Verständnis für Mechanismen hinter modernen Abstraktionen
  - z.B. Verwendung von Bibliotheken
- Interoperabilität (Brücke) zwischen Sprachen
  - Foreign Function Interface (FFI)
  - z.B. Java Foreign Function and Memory API
    - früher Java Native Interface Specification: Contents

Originalfolie 01, PDF-Seite 5: Motivation für C und C++

2 C-Einführung: Vom Quelltext zum ersten Programm

2.1 Eigenschaften von C und Unterschied zu Java

Java-Programme sind typischerweise in Klassen und Objekten organisiert, während C selbst weder Klassen noch Methoden kennt. C besitzt allerdings strukturierte Typen wie `struct` und benutzerdefinierte Typnamen. Ein Java-Objekt wird über eine Referenz angesprochen; in C gibt es explizite **Zeiger**, deren Wert eine Adresse beziehungsweise eine geeignete Repräsentation einer Speicherposition ist. Mit dem Adressoperator `&` erhält man die Adresse eines Objekts, und `*` dereferenziert einen passenden Zeiger. Zeigerarithmetik ist möglich, verlangt aber, dass sich die Adressen innerhalb eines gültigen Speicherbereichs bewegen.

Alle Funktionsparameter werden in C per Wert übergeben. Übergibt man etwa einen Zeiger, wird dessen Zeigerwert kopiert; über die Kopie kann die aufrufende Funktion trotzdem das referenzierte Objekt verändern. Java übergibt ebenfalls Werte, bei Objekten sind diese Werte Referenzen. C verwaltet dynamisch reservierten Speicher normalerweise mittels `malloc` und `free`; Java setzt in der üblichen Ausführungsumgebung Garbage Collection ein. Die C-Standardbibliothek ist wesentlich kleiner als das typische Java-Ökosystem. C-Implementierungscode wird üblicherweise in `.c`-Dateien geschrieben; eine `.h`-Datei stellt dafür Deklarationen nach außen bereit. Die Dateiennamen selbst schaffen dabei keine speziellen Sprach-Namespaces.

Die in den Folien genannten Popularitätsranglisten (TIOBE, PYPL, RedMonk, IEEE Spectrum und GitHut) sind Momentaufnahmen unterschiedlicher Messverfahren. Sie belegen nicht denselben statistischen Gegenstand und sind insbesondere keine Messung der technischen Eignung. Als vertiefende Literatur nennen die Folien vor allem Jens Gustedts *Modern C*, O. Gazis *Modern C Programming*, die C-Referenz von cppreference und die Norm ISO/IEC 9899:2024. Historisch relevant ist Kernighan/Ritchie.

Geschichte von C

- ab 1971 “*new B*” bei Bell Labs von Dennis Ritchie (Turing Award 1983)
  - inspiriert durch
    - *B* (typenlose Sprache) von Ken Thompson (Turing Award 1983)
    - *BCPL* von Martin Richards (IEEE Computer Pioneer Award 2003)
- 1978 “The C Programming Language” von Brian Kernighan und D. Ritchie
  - de-facto Standard *“K&R C”* (C78)
- 1989 ANSI C Standard (C89)
- 1990 ISO C Standard (C90)
- 1995 C95 <wchar.h> <wctype.h>

The development of the C language

- 1999 C99
  - Zeilenendkommentare (`// ...`)
  - Felder mit variabler Länge (VLA)
  - neue Datentypen, z.B. `_Complex`
  - Fixed width integers z.B. `uint32_t`
- 2011 C11
  - alignment
  - generics
  - unicode
- 2018 C17
  - kleinere Änderungen
  - keine neuen Features
- 2023 C23 moderne Features
  - `constexpr`
  - `typeof`
  - `nullptr`
  - Bitfunktionen <stdbit.h>

C Einführung WS 2026/27 SWC3 Erik Pitzer 2

Originalfolie 02, PDF-Seite 3: Geschichte von C

Eigenschaften von C (i)

- imperativ vs. deklarativ
  - direkte Kontrolle über Programmablauf und Änderungen
- prozedural vs. objektorientiert oder funktional
  - Organisation in Prozeduren (und Funktionen)
- statisch typisiert vs. dynamisch typisiert
  - Typ einer Variablen ist dem Compiler bekannt
- relativ schwach typisiert vs. stark typisiert
  - Typ einer Variablen kann (relativ leicht) verändert bzw. neu interpretiert werden
- übersetzt vs. interpretiert
  - C wird von einem Compiler in Maschinensprache übersetzt
  - keine Laufzeitumgebung (*runtime*) oder virtuelle Maschine

Originalfolie 02, PDF-Seite 4: Eigenschaften von C (i)

Eigenschaften von C (ii)

- (relativ) low-level
  - direktere Interaktion mit Hardware und Betriebssystem
  - primitivere Abstraktionskonzepte
- manuelle Speicherverwaltung vs. Garbage Collection
  - planbare Performance
  - weniger Komfort
- portabel und standardisiert vs. plattformspezifisch
  - leichte Verfügbarkeit auf verschiedenen Systemen
- kompakt vs. umfangreich
  - kleine Standardbibliothek
  - hardware-nahe Programmierung (OS, embedded)
- Präprozessor
  - Quelltext wird vor der Übersetzung vorverarbeitet
  - erlaubt Textersetzungen (Makros)

Originalfolie 02, PDF-Seite 5: Eigenschaften von C (ii)

Hauptunterschiede zu Java

- C ist rein prozedural (keine Klassen, keine Objekte)
  - nur strukturierte Datentypen ohne Methoden
- C erlaubt Zeiger und Zeiger-Arithmetik
- C hat eine sehr kleine Standardbibliothek im Vergleich zu Java
- in C werden Funktionsparameter immer per Wert übergeben (call-by-value)
  - Übergabe per Referenz (*call-by-reference*) muss mit Zeiger emuliert werden
  - in Java wird auch per Wert übergeben, Variablen für Objekte sind aber Referenzen und verhalten sich daher eher wie call-by-reference
- in C muss dynamischer Speicher manuell verwaltet werden
  - in Java gibt es einen Garbage Collector
- C Quelltext wird in Header (`*.h`) und Implementierung (`*.c`) getrennt
  - Name der Datei und Pfad sind irrelevant
- C wird zu Maschinensprache übersetzt und direkt vom Prozessor ausgeführt
- C-Zeichenketten sind sehr primitiv

Originalfolie 02, PDF-Seite 6: Hauptunterschiede zu Java

Popularität von C

Quelle	Platz	Stand
TIOBE	2	Sep 2026
PyPL	3	Sep 2026
RedMonk	10	Jun 2025
IEEE Spectrum	8[1]	Sep 2025
GitHut 2.0	8[1]	Mar 2024

[1] Durchschnitt aus mehreren Reihungen

Originalfolie 02, PDF-Seite 7: Popularität von C

Literatur

Hauptquellen

- J. Gustedt. *“Modern C”*, Manning, 2025
- O. Gazi. *“Modern C Programming”*, Springer, 2024
- C reference - cppreference.com (DevDocs — C documentation)

- M. Kalin. “Modern C Up and Running”, Apress, 2022
- ISO/IEC 9899:2024 *“ISO Standard: C23”* (n3220)
- M. Olsson. *“Modern C Quick Syntax Reference”*, Apress, 2019
- S. Dmitrović. *“Modern C for Absolute Beginners”*, Apress, 2021

- B. Kernighan & D. Ritchie. *“The C programming language”*, Prentice-Hall, 1988 (de)
- Z. Shaw. *“Learn C the hard way”*, Addison-Wesley, 2015

C Einführung WS 2026/27 SWC3 Erik Pitzer 7

Originalfolie 02, PDF-Seite 8: Literatur

2.2 Das erste vollständige C-Programm

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Eine **Direktive** wie `#include <stdio.h>` wird vom Präprozessor verarbeitet: Der Header stellt unter anderem die Deklaration von `printf` bereit. `int main(void)` bezeichnet die vom Programmstart verwendete Hauptfunktion; `int` ist ihr Rückgabetyp und `void` in der Parameterliste sagt aus, dass diese Form keine Argumente annimmt. Zwischen `{` und `}` stehen Anweisungen. `printf` schreibt formatierten Text auf die Standardausgabe. In der Zeichenkette bezeichnet `\n` einen Zeilenwechsel. `return 0;` signalisiert eine erfolgreiche Beendigung; bei `main` ist der implizite Rückgabewert bei Erreichen des Funktionsendes ebenfalls null. Der Aufruf `gcc -o hello hello.c` übersetzt und bindet das Programm, `./hello` startet es auf einem typischen Unix-System. Das Kommando `gcc` ist ein konkreter Compileraufruf und kein Bestandteil der C-Syntax.

Das erweiterte Folienprogramm führt mehrere Sprachkonstrukte gleichzeitig vor: `#include <stdlib.h>` ermöglicht `EXIT_SUCCESS`; `main(int argc, char *argv[argc+1])` erhält die Zahl `argc` und eine Folge von Argumentzeigern `argv`, deren letztes Element `argv[argc]` null ist. Das Attribut `[[maybe_unused]]` kennzeichnet seit C23 einen absichtlich unbenutzten Parameter. `double A[5]` definiert fünf Fließkommaelemente; mit `[4] = 3.E+25` und `[3] = 0.0007` werden einzelne Positionen gezielt initialisiert. Nicht genannte Arrayelemente werden bei einer solchen Initialisierung mit null initialisiert. `for (size_t i = 0; i < 5; ++i)` durchläuft die Indexwerte **0, 1, 2, 3 und 4**, nicht 5. `size_t` ist ein vorzeichenloser Typ für Objektgrößen und Indizes. `A[i]*A[i]` berechnet das Quadrat des jeweiligen Werts.

Die Anweisung `printf("A[%zu] = %6g, ... %8g\n", i, A[i], A[i]*A[i]);` illustriert **Formatplatzhalter**: `%zu` erwartet einen Wert des Typs `size_t`, `%6g` gibt einen Fließkommawert mit Mindestfeldbreite 6 aus und `%8g` mit Feldbreite 8. Die Breite beeinflusst die Darstellung, nicht den mathematischen Wert. `3.E+25` bedeutet `3 × 10^25`;

für das Quadrat ergibt sich `9 × 10^50`. Im Quelltext stehen außerdem zwei Kommentarformen: `/* ... */` für Blockkommentare und `// ...` für Kommentare bis zum Zeilenende.

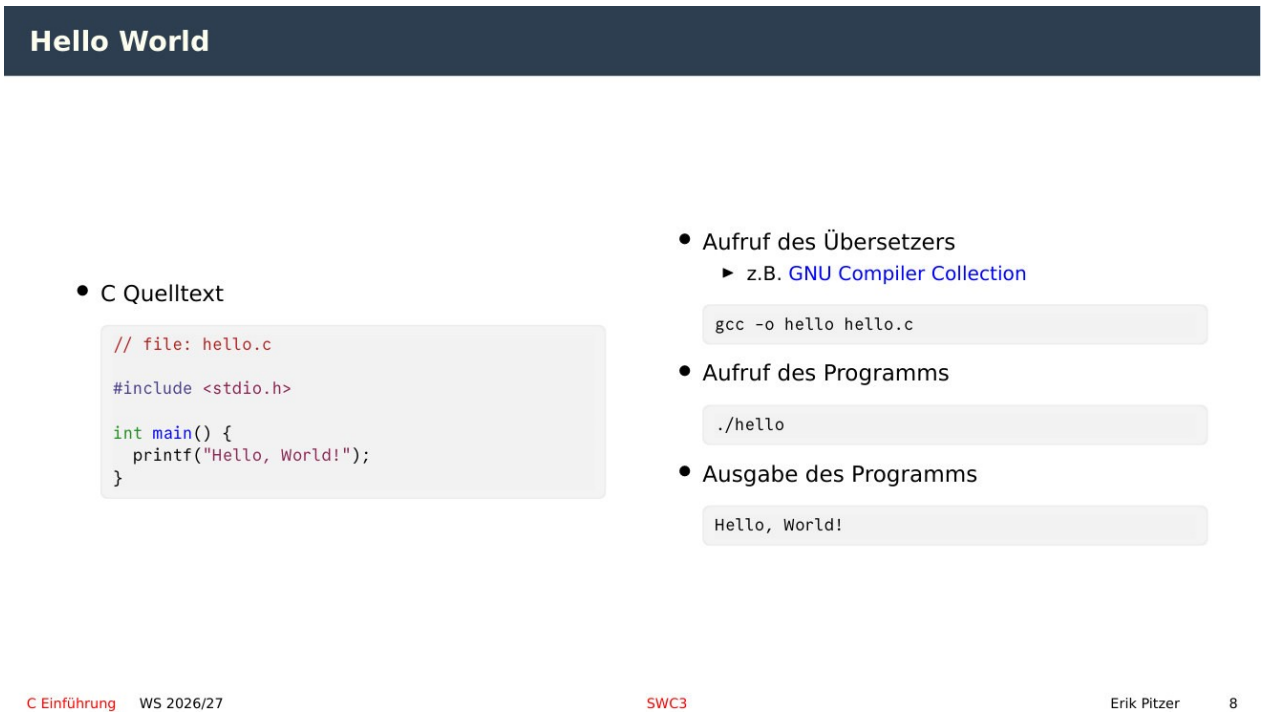
Vertiefung · Schritt für Schritt

Der vollständige Ablauf von „Hello World“ besteht aus vier technisch getrennten Tätigkeiten: Der Präprozessor stellt die Deklaration von `printf` bereit; der Compiler prüft Syntax und Typen und erzeugt Maschinen- beziehungsweise Objektcode; der Linker verbindet den Aufruf mit einer passenden Bibliotheksimplementierung; erst anschließend startet das Betriebssystem die ausführbare Datei. Die Anweisung `printf` liefert selbst einen `int` zurück, nämlich die Zahl der ausgegebenen Zeichen oder einen negativen Wert bei Ausgabefehler. Der Rückgabewert von `main` ist davon unabhängig und signalisiert dem aufrufenden Prozess den Programmstatus.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    if (printf("Hello, World!\n") < 0) {
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

Beim erweiterten Beispiel ist das Array `A[5]` nicht gleich einem Zeiger: In der Deklaration reserviert die Arraydefinition Platz für fünf `double`-Elemente. Die Initialisierer [0], [1], [3], [4] setzen einzelne Positionen; nicht genannte Elemente werden wegen des vorhandenen Initialisierers auf null initialisiert. Die Laufvariable i vom Typ `size_t` nimmt nacheinander die Werte 0, 1, 2, 3, 4 an, nicht 5. In `printf` steht %zu für `size_t`, `%g` für eine möglichst kompakte Fließkommadarstellung, 6 beziehungsweise 8 sind Mindestfeldbreiten. Die Quadrate werden beim Funktionsaufruf durch `A[i]*A[i]` berechnet.



Originalfolie 02, PDF-Seite 9: Hello World

Hello World v2 (i)

```c
// from J. Gustedt "Modern C" (2024)

#include <stdlib.h>
#include <stdio.h>

int main(int argc, [[maybe_unused]] char* argv[argc+1]) {

  double A[5] = {
    [0] = 9.0,
    [1] = 2.9,
    [4] = 3.E+25,
    [3] = 0.0007,
  };

  for (size_t i = 0; i < 5; ++i) {
    printf("A[%zu] = %6g, A[i]² = %8g\n", i, A[i], A[i]*A[i]);
  }

  return EXIT_SUCCESS;
}
```

[2] -- default init mit 0

```
A[0] =      9, A[i]² =       81
A[1] =    2.9, A[i]² =     8.41
A[2] =      0, A[i]² =        0
A[3] = 0.0007, A[i]² =  4.9e-07
A[4] =  3e+25, A[i]² =    9e+50
```


Originalfolie 02, PDF-Seite 10: Hello World v2 (i)

Hello World v2 (ii)

- Kommentare[1] in C entweder `/* inline */` oder als `// Zeilenendkommentar`

```c
// from J. Gustedt "Modern C" (2024)
```

- Inhalt anderer Dateien inkludieren[2]

```c
#include <stdlib.h>
#include <stdio.h>
```

- Hauptfunktion `main()`, mit Argumenten vom Betriebssystem[3]
  - Anzahl `int argc` (*argument count*) und
  - Inhalt `char* argv[argc+1]` (*argument vector*)
  - Attribut `[[maybe_unused]]`: Parameter muss nicht benutzt werden (seit C23)

```c
int main(int argc, [[maybe_unused]] char* argv[argc+1]) { /* ... */ }
```

[1] Comments - cppreference.com
[2] Source file inclusion - cppreference.com
[3] Main function - cppreference.com

Originalfolie 02, PDF-Seite 11: Hello World v2 (ii)

Hello World v2 (iii)

- Deklaration und Definition eines Feldes (Array)[1] von Fließkommazahlen[2] mit dem Namen `A`
  - direkte Initialisierung an verschiedenen Indizes
  - automatische Initialisierung mit `0` für fehlende Indizes

```
8   double A[5] = {
9     [0] = 9.0,
10    [1] = 2.9,
11    [4] = 3.E+25,
12    [3] = 0.0007,
13  };
```

- Schleife[3] über die Zahlen i $\leftarrow [0..5)$

```
15  for (size_t i = 0; i < 5; ++i) { /* ... */ }
```

[1] Array declaration - cppreference.com
[2] Arithmetic types - Real floating types - cppreference.com
[3] for loop - cppreference.com

Originalfolie 02, PDF-Seite 12: Hello World v2 (iii)

Hello World v2 (iv)

- formatierte Ausgabe mit Formatvorlage[1]
  - spezielle Formatangaben (wie `"%zu"` und `"%6g"`) für restlichen Argumente

```
16  printf("A[%zu] = %6g, A[i]² = %8g\n", i, A[i], A[i]*A[i]);
```

- Rückgabewert an das Betriebssystem (`0` → kein Fehler)[2]
  - darf auch weggelassen werden seit C99

```
19  return EXIT_SUCCESS;
```

[1] printf et al. - cppreference.cpp
[2] Main function - cppreference.com

Originalfolie 02, PDF-Seite 13: Hello World v2 (iv)

2.3 Schlüsselwörter, Arten von Datentypen und Zeichenketten

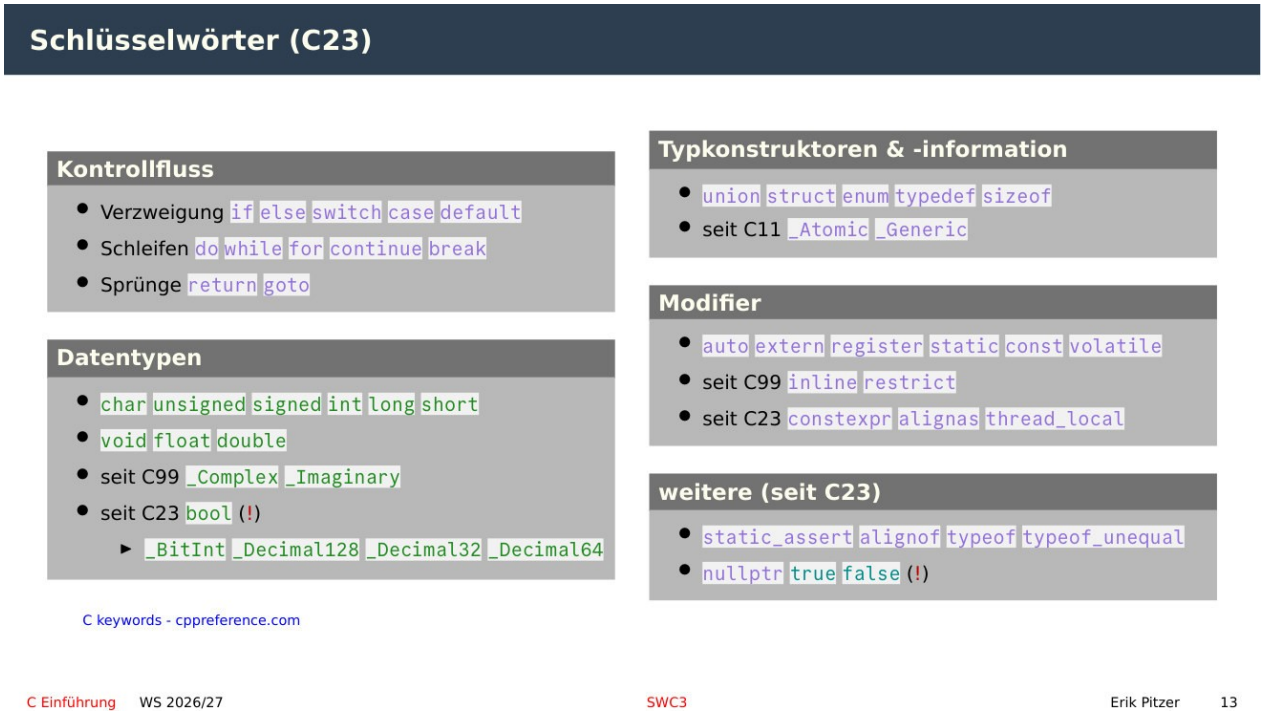
Schlüsselwörter sind vordefinierte Wörter der Sprache und dürfen nicht beliebig als eigene Bezeichner verwendet werden. Die C23-Übersicht gliedert sie nach Kontrollfluss (`if`, `else`, `switch`, `case`, `while`, `for`, `break`, `continue`, `return`, `goto`), Typen (`int`, `char`, `float`, `double`, `void`, `bool` und weitere), Typkonstruktion (`struct`, `union`, `enum`, `typedef`, `sizeof`, `_Generic`), Qualifizierern bzw. Deklarationsangaben (`const`, `volatile`, `static`, `extern`, `auto`, `restrict`, `inline`) sowie C23-Ergänzungen. **Typ** bedeutet die Kategorie von Werten und zulässigen Operationen. Die wichtigsten Familien sind ganzzahlige Typen, Fließkommatypen, Aufzählungen, Zeiger, Arrays, Funktionen und zusammengesetzte Typen.

`void` bezeichnet einen unvollständigen Objekttyp beziehungsweise in der passenden Verwendung das Fehlen eines Ergebniswerts. `bool` repräsentiert die Wahrheitswerte `true` und `false`. `char` speichert eine Zeichencodeeinheit; ein `char` ist nicht automatisch ein vollständiges Unicode-Zeichen. **C-Zeichenketten** sind Folgen von Zeicheneinheiten mit abschließendem Nullzeichen `\0`. Sie enthalten keinen eingebauten Längenwert; die Länge muss etwa durch Suchen des Terminators bestimmt werden. Unicode- und UTF-8-Unterstützung hängt von den verwendeten Zeichen- und Bibliothekstypen ab. Die Folien zeigen auch dezimale, komplexe, imaginäre und atomare Typen, deren tatsächliche Verfügbarkeit bzw. konkrete Ausprägung implementierungsabhängig oder bedingt unterstützt sein kann.

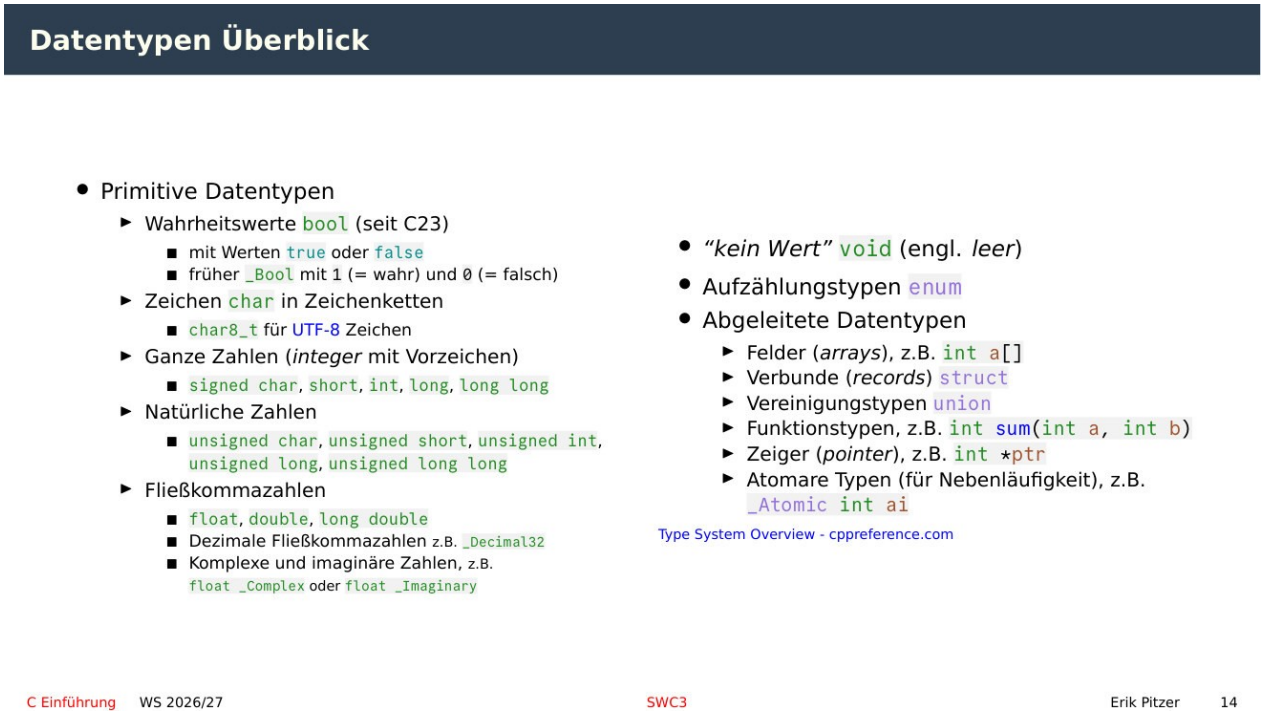
Vertiefung · Schritt für Schritt

Drei Ebenen der Typbetrachtung sind voneinander zu trennen: Ein Typ legt fest, welche Werte und Operationen zulässig sind; ein Objekt ist eine konkrete Instanz mit Speicher und Lebensdauer; ein Ausdruck bezeichnet je nach Art ein Objekt, eine Funktion oder liefert einen Wert. `char` ist ein eigener Integer-Typ, nicht automatisch dasselbe wie `signed char` oder `unsigned char`. Ob ein einfaches `char` negative Werte darstellen kann, ist implementationsabhängig. `void` kann beispielsweise eine Funktion ohne Ergebnis kennzeichnen, darf aber nicht als vollständiger Objekttyp einer normalen Variable verwendet werden. Arrays, Zeiger, Funktions- und Strukturtypen sind unterschiedliche Kategorien mit jeweils eigenen Regeln.

C23 ergänzt Namen wie `bool`, `true`, `false` und `nullptr` sowie bestimmte neue Deklarationsformen. Deshalb lassen sich ältere Beispiele nicht immer unverändert unter dem Sprachmodus C17 kompilieren. Umgekehrt sind nicht alle theoretisch standardisierten optionalen Zahlenformate in jeder konkreten Compilerinstallation implementiert. Für eigene Übungsprogramme sollten Sprachmodus und tatsächliche Compilerunterstützung ausdrücklich kontrolliert werden.



Originalfolie 02, PDF-Seite 14: Schlüsselwörter (C23)



Originalfolie 02, PDF-Seite 15: Datentypen Überblick

2.4 Ganzzahlen, Größen und Wertebereiche

Ein ganzzahliger Datentyp wird mit `signed` oder `unsigned` und einer Größenvariante `short`, `int`, `long`, `long long` (C23 zudem `_BitInt`) bezeichnet. Ohne ausdrückliches Vorzeichen ist `int` vorzeichenbehaftet. Der Standard garantiert bestimmte **Ordnungsbeziehungen der Speichergröße**, etwa `sizeof(char) == 1` und `sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`, aber nicht allgemein `int == 32 Bit` oder `long == 64 Bit`. `sizeof` liefert die Größe in **Bytes**, wobei ein C-Byte aus `CHAR_BIT` Bits besteht. Gerade bei Austausch binärer Daten, Netzwerknachrichten und Dateiformaten darf die konkrete Größe daher nicht stillschweigend angenommen werden.

Bei vorzeichenlosen ganzzahligen Typen ist Arithmetik modulo `2^N` definiert, sofern der Typ `N` Wertbits besitzt: etwa `UINT_MAX + 1u` ergibt wieder 0. Ein Überlauf eines vorzeichenbehafteten Integer-Ausdrucks ist dagegen **undefiniertes Verhalten**; man darf nicht einfach von einer mathematischen Fortsetzung oder Zweierkomplement-Wraparound ausgehen. `<limits.h>` liefert implementierungsabhängige Grenzwerte wie `INT_MAX`, `CHAR_MIN` und `UINT_MAX`. Die Tabelle der Folien gibt ein konkretes Beispiel für GCC auf x86-64 unter Linux an: `char` 8 Bit, `short` 16 Bit, `int` 32 Bit, `long` und `long long` 64 Bit. Eine andere Zielplattform kann andere Größen verwenden.

Für binär definierte Größen bietet `<stdint.h>` Namen wie `uint8_t`, `int32_t` und `uint64_t`, sofern Typen mit genau dieser Breite vorhanden sind. `int_least32_t` bedeutet **mindestens** 32 Bit, `int_fast32_t` einen implementationstypisch effizienten Typ mit mindestens 32 Bit; insbesondere sind `least`- und `fast`-Typen nicht generell genau 32 Bit. `intmax_t` und `uintmax_t` sind auf maximale standardisierte Integerbreite der Implementierung ausgerichtet. **Korrektur zur Tabelle:** Ein 32-Bit-`int32_t` hat als kleinsten Wert **−2.147.483.648**; die betreffende Minusangabe fehlt an einer Stelle der Originaltabelle.

Vertiefung · Schritt für Schritt

Die Darstellung von Ganzzahlen hängt von der Zahl ihrer Wertbits ab. Hat ein `unsigned`-Typ N Wertbits, dann umfasst sein Wertebereich 0 bis 2^N−1 und Rechnen auf diesem Typ erfolgt modulo 2^N. Bei 8 Bit gilt beispielsweise 255 + 1 = 0, wenn die gesamte Rechnung im entsprechenden `unsigned`-Typ erfolgt. Achtung: Der Ausdruck mit `unsigned char` kann aufgrund der Integer Promotions zunächst als `int` gerechnet werden; ein Überlauf des Zieltyps kann dann

erst bei der abschließenden Konversion entstehen. Bei einem `signed`-Typ ist ein arithmetischer Überlauf keine definierte Modulo-Rechnung, sondern undefiniertes Verhalten.

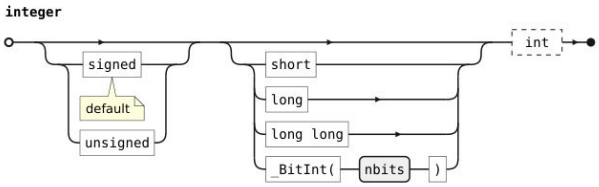
```c
#include <limits.h>
#include <stdint.h>

int main(void) {
    unsigned int u = UINT_MAX;
    u = u + 1u;  // definiert: u wird 0
    uint8_t byte = UINT8_MAX;
    byte = (uint8_t)(byte + 1); // bei 8 Bit: wieder 0
    return (int)(u + byte);
}
```

Die Tabellen mit Größen für GCC auf x86-64 unter Linux sind Beispiele, keine Normgarantien. Insbesondere dürfen `long` und Zeiger nicht pauschal als 64 Bit angenommen werden: Auf verbreiteten Windows-64-Bit-ABIs hat `long` weiterhin 32 Bit. `<stdint.h>` bietet `int32_t` und `uint32_t` genau dann an, wenn der entsprechende exakte Typ vorhanden ist. int_least32_t verlangt mindestens 32 Bit; `int_fast32_t` bezeichnet dagegen einen für eine Implementierung zweckmäßigen Typ mit mindestens dieser Wertbreite und kann breiter sein.

Ganze Zahlen

- Ganzzahlige Datentypen (engl. *integer*) werden angegeben durch
  - Vorzeichen (engl. *signedness*): `signed` (default) oder `unsigned`
  - Länge: `short`, keine Angabe (default), `long`, `long long`, oder `_BitInt()`
  - `int` (optional falls Modifier verwendet)



- Der Wertebereich ist compilerabhängig
  - Standard schreibt nur relative Größe vor
  - `1 == sizeof(char) ≤ sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)`
- Bei vorzeichenlosen Zahlen (`unsigned`) *wohldefinierter* overflow z.B. `UINT_MAX+1` → `0`
- Bei ganzen Zahlen (`signed`) undefined behavior

Arithmetic Types - cppreference.com

C Einführung WS 2026/27 SWC3 Erik Pitzer 15

Originalfolie 02, PDF-Seite 16: Ganze Zahlen

Keine fixen Wertebereiche für Zahlen

- Wertebereiche für Zahlen-Datentypen sind nur relativ zueinander standardisiert
- können von Compiler und Zielplattform oder -architektur abhängen
- Grenzen für ganze Zahlen sind im Header <limits.h> definiert
- Bereiche für Fließkommazahlen finden sich im Header <float.h>

```c
#include <limits.h>
#include <float.h>

int main() {
  SHOW2(CHAR_MIN, CHAR_MAX);
  SHOW(UINT_MAX);
  SHOW2(FLT_MIN, FLT_MAX);
}
```

```
CHAR_MIN = -128;  CHAR_MAX = 127
UINT_MAX = 4294967295
FLT_MIN = 1.17549e-38;  FLT_MAX = 3.40282e+38
```


Originalfolie 02, PDF-Seite 18: Keine fixen Wertebereiche für Zahlen

Wertebereiche *Beispiel* — für GCC und x86_64 unter Linux

Typ	Bits	Minimum	Maximum
char	8	-128	127
unsigned char	8	0	255
signed char	8	-128	127
short	16	-32.768	32.767
signed short	16	-32.768	32.767
unsigned short	16	0	65.535
int	32	-2.147.483.648	2.147.483.647
signed	32	-2.147.483.648	2.147.483.647
unsigned	32	0	4.294.967.295
long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
signed long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long	64	0	18.446.744.073.709.551.615
long long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
signed long long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long long	64	0	18.446.744.073.709.551.615
float	32	1,17549e-38	3,40282e+38
double	64	2,22507e-308	1,79769e+308
long double	128	3,3621e-4932	1,18973e+4932

least und fast sind abhängig von der Architektur

Originalfolie 02, PDF-Seite 19: Wertebereiche Beispiel — für GCC und x86_64 unter Linux

Datentypen mit fixer Größe

- Datentypen mit fixer Größe werden in <stdint.h> definiert

Typ	Bits	Minimum	Maximum
uint8_t	8	0	255
uint32_t	32	0	4.294.967.295
uint64_t	64	0	18.446.744.073.709.551.615
int8_t	8	-128	127
int32_t	32	2.147.483.648	2.147.483.647
int64_t	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
int_least8_t	8	-128	127
int_least32_t	32	2.147.483.648	2.147.483.647
int_least64_t	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
int_fast8_t	8	-128	127
int_fast32_t	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
int_fast64_t	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
intmax_t	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
uintmax_t	64	0	18.446.744.073.709.551.615

C Einführung WS 2026/27 SWC3 Erik Pitzer 19

Originalfolie 02, PDF-Seite 20: Datentypen mit fixer Größe

2.5 Gleitkommazahlen, Dezimalzahlen und komplexe Zahlen

Die binären Fließkommatypen heißen `float`, `double` und `long double`. Viele gebräuchliche Systeme benutzen IEEE-754-Formate; die Folien zeigen für eine 32-Bit-`float`-Zahl **ein Vorzeichenbit, acht Exponentenbits und 23 gespeicherte Fraction-Bits**. Der wesentliche mathematische Aufbau einer normalen binären Fließkommazahl ist `(-1)^s · (1 + m) · 2^e`, wobei `s` das Vorzeichenbit, `m` die aus Bits gebildete Mantissenfraktion und `e` der um den Bias bereinigte Exponent ist. Für Subnormale, Null, unendlich und NaN gelten Sonderregeln. Genauigkeit und Wertebereich sind getrennte Eigenschaften: Ein großer Exponent erlaubt sehr große Beträge, zusätzliche Mantissenbits erlauben mehr signifikante Ziffern.

Viele Dezimalbrüche sind binär nicht exakt darstellbar. Deshalb ergibt die Speicherung von `0.1` in einem üblichen binären `double` eine Näherung, zum Beispiel `0.1000000000000000555...`. C23 kennt optionale beziehungsweise implementierungsabhängig unterstützte Dezimalfließkommatypen wie `_Decimal32`, `_Decimal64` und `_Decimal128`, für die ein Dezimalwert wie `0.1df` exakt darstellbar sein kann. Die konkreten Größen `float = 32`, `double = 64` und `long double = 128 Bit` in den Folien sind **Plattformbeispiele**, keine allgemeine C-Garantie. Bei `<float.h>` ist `FLT_MIN` der kleinste positive **normale** `float`, nicht der kleinste positive subnormale Wert; `FLT_MAX` ist der größte endliche `float`.

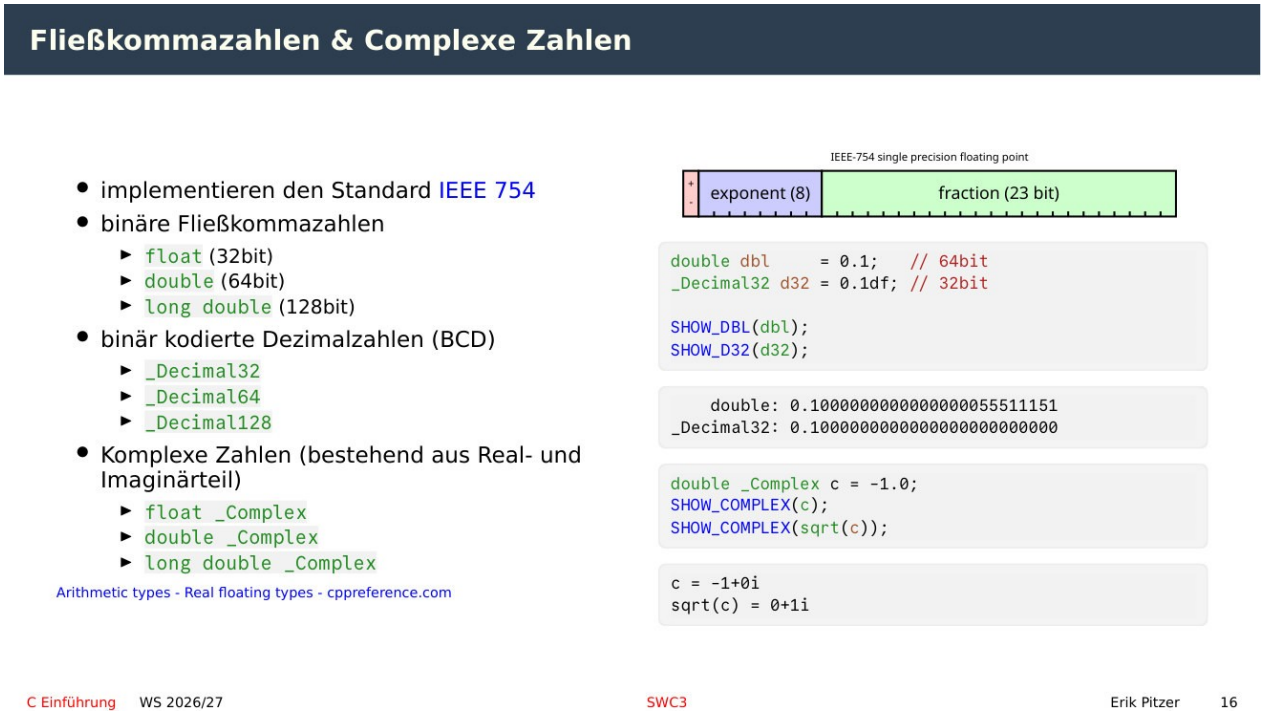
Mit `_Complex` lassen sich komplexe Zahlen darstellen. Eine komplexe Zahl `z = a + bi` besteht aus Realteil `a` und Imaginärteil `b`, mit `i² = −1`. Die Folie illustriert `sqrt(-1 + 0i) = 0 + 1i` auf einem üblichen Hauptzweig der komplexen Wurzel. Funktionen und Makros für komplexe Arithmetik liegen unter anderem in `<complex.h>`. Die gezeigten Bitfeld-Diagramme sind konkrete Darstellungen und sollten nicht mit einer sprachweiten Festlegung der Speicherrepräsentation verwechselt werden.

Vertiefung · Schritt für Schritt

Das im Diagramm gezeigte IEEE-754-Binary32-Feld besitzt ein Vorzeichenbit s, acht gespeicherte Exponentenbits E und 23 Bruchbits F. Für normale, endliche Zahlen gilt Wert = (−1)^s · (1+F/2^23) · 2^(E−127). Die gespeicherte Exponentenzahl enthält also einen Bias von 127, und das führende Eins-Bit der Signifikandendarstellung wird bei normalisierten Zahlen nicht gespeichert. Für E=0 gelten Regeln für Null und Subnormalzahlen, für E=255 Regeln für Unendlichkeit und NaN. Daher ist die verkürzte Normalzahlenformel keine vollständige Beschreibung aller Bitmuster.

Warum die Folie bei `double` den Wert 0.10000000000000000555… ausgibt: 0,1 lässt sich im Binärsystem nur periodisch darstellen. Die Rundung auf eine endliche Zahl von Signifikandbits ergibt den nächstliegenden repräsentierbaren Wert. Daraus folgt, dass exakte Gleichheitsprüfungen bei berechneten Fließkommawerten oft ungeeignet sind; der tolerierte Fehler muss nach absoluter oder relativer Größenordnung der Anwendung gewählt werden. _Decimal32 stellt dezimale Werte dar und kann 0,1 als Dezimalzahl exakt kodieren; ob ein Compiler dieses Format und die gezeigten Bibliotheksfunktionen unterstützt, ist gesondert zu prüfen.

Komplexe Typen wie `double _Complex` speichern Real- und Imaginärkomponente. Für z = −1+0i ergibt die komplexe Quadratwurzel +i im üblichen Hauptzweig. Die komplexe Version wird über die komplexe Mathematikschnittstelle, etwa `csqrt` aus `<complex.h>`, benutzt. Der normale reelle Aufruf `sqrt` mit einem negativen reellen Argument ist kein Ersatz für eine komplexe Quadratwurzel.



Originalfolie 02, PDF-Seite 17: Fließkommazahlen & Complexe Zahlen

2.6 Literale, Typumwandlungen und Bezeichner

Ein **Literal** ist eine direkt im Programmtext angegebene Konstante. `123` ist ein dezimales Integerliteral, `0xff` ein hexadezimales Literal (255), `020` dagegen **oktal** (16), `0b101` ist binär (5), und `0x1p2` ist eine hexadezimale Fließkommaschreibweise für $1 \cdot 2^2 = 4.0$. `1'250'399` trennt seit C23 Ziffern optisch. Suffixe spezifizieren den Typ, etwa `10u` für `unsigned int`, `10L` für `long`, `10LL` für `long long`, `1.0f` für `float` und `1.0L` für `long double`. In C hat ein gewöhnliches Zeichenliteral wie `'a'` den Typ `int`, nicht `char`. Ein Stringliteral wie `"ABC"` legt zudem das abschließende `\0` ab; seine Arraygröße ist deshalb 4. **Ergänzende Einordnung:** Ein gewöhnliches Stringliteral in C hat nicht den Typ `const char[]`; es ist ein Array von `char`, dessen Modifikation trotzdem undefiniertes Verhalten ist. Die Folie vereinfacht die Typangabe hier.

Ein expliziter **Cast** schreibt einen Ausdruck als `(zieltyp)ausdruck`. Dabei können Informationen verlorengehen: `(int)7.9` ergibt 7, weil der Nachkommateil in Richtung null abgeschnitten wird. Bei einer Umwandlung außerhalb des darstellbaren Integerbereichs oder bei problematischen Fließkommaumwandlungen darf man nicht einfach ein Beispielergebnis auf alle Plattformen übertragen. Die auf den Folien gezeigten Ausgaben `(short)INT_MAX == -1` und `(float)DBL_MAX == inf` sind konkrete Implementierungsergebnisse und keine universellen Regeln. Implizite Umwandlungen entstehen außerdem bei Operatoren und Zuweisungen, weshalb Warnungen wie `-Wconversion` nützlich sind.

Bezeichner sind selbst gewählte Namen für Funktionen, Variablen, Typen und weitere Elemente. Üblich sind Buchstaben und Unterstriche am Anfang und danach zusätzlich Ziffern; die Möglichkeiten des Standards bezüglich Unicode und UCN sind umfangreicher als das einfache ASCII-Muster der Folie. Einige Namen sind reserviert, insbesondere solche mit zwei Unterstrichen oder einem führenden Unterstrich gefolgt von einem Großbuchstaben; sie sollten nicht als eigene Bezeichner verwendet werden. In C gibt es mehrere Namensräume, insbesondere für `struct`-/`union`-/`enum`-Tags und für gewöhnliche Bezeichner.

Literale

- Literale sind Elemente der Sprache die *buchstäblich* für sich selbst stehen
- *nackte* Zahlenwerte Werte
  - `true` und `false` → `bool`
  - 5 oder `-3` → `int`
  - `5.5` oder `0.123e-12` → `double`
- Zahlen mit Basis-Präfix
  - `0x` hexadezimal
    - `0xff` → 255
    - `0x1p-1` → 0.5
    - `0x1p2` → 4.0
  - `0` oktal (!): `020` → 16 !!!!
  - `0b` binär: `0b101` → 5
- Zahlen mit Typ-Suffix
  - `0u` → `unsigned int`
  - `0l` → `long`
  - `0ll` → `long long`
  - `0f` → `float`
  - `0ld` → `long double`

Boolean literals - cppreference.com
Integer Literals - cppreference.com
Floating-point literals - cppreference.com

- Kombinationen von Präfix und Suffix
  - `0b1010u` → `(unsigned)10`
- Zifferntrenner für Zahlen
  - `1'250'399` → 1 250 399
- einzelne Zeichen (*character*)
  - `'a'` vom Typ `int` (!)
  - `u8'a'` vom Typ `char8_t`
    - UTF-8 codiertes Zeichen
- Zeichenkettenkonstanten
  - `"Hello World!\n"` → `const char[]`
  - `u8"I♡C ☺"` → `const char8_t[]`)
  - keine Längeninformation
  - Terminatorzeichen (ASCII Code `0`)
- Escape-Codes für Zeichen
  - z.B. `'\n'` für Zeilenumbruch, `'\0'` für Terminatorzeichen

Character literals - cppreference.com

C Einführung WS 2026/27 SWC3 Erik Pitzer 20

Originalfolie 02, PDF-Seite 21: Literale

Typumwandlung (*type cast*) (i)

- Datentypen können mittels Typecast umgewandelt werden
  - dabei können
    - Informationen verloren gehen
    - Umwandlungsfehler entstehen
  - in der Praxis daher sehr sparsam einsetzen

```
SHOW_TYPE('c');
SHOW_TYPE((char)'c');
```

```
'c' = 99 (int)
(char)'c' = 'c' (char)
```

```
SHOW_TYPE(INT_MAX);
SHOW_TYPE((short)INT_MAX);
```

```
INT_MAX = 2147483647 (int)
(short)INT_MAX = -1 (short int)
```

```
SHOW_TYPE(0.1);
SHOW_TYPE((int)7.9);
SHOW_TYPE(DBL_MAX);
SHOW_TYPE((float)DBL_MAX);
```

```
0.1 = 0.1 (double)
(int)7.9 = 7 (int)
DBL_MAX = 1.79769e+308 (double)
(float)DBL_MAX = inf (float)
```

C Einführung WS 2026/27 SWC3 Erik Pitzer 21

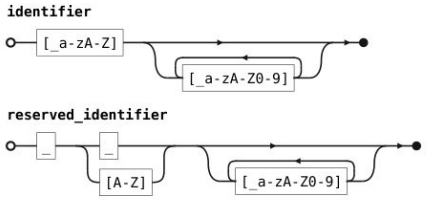
Originalfolie 02, PDF-Seite 22: Typumwandlung (type cast) (i)

Bezeichner (*identifier*)

- Bezeichner (engl. *identifier*) sind alle Namen die nicht von der Sprache vordefiniert sind und werden vergeben für
  - Variablen, z.B. `int counter`
  - Konstanten, z.B. `#define MAX_LENGTH = 12`
  - Datentypen, z.B. `struct time_t`
  - Funktionen, z.B. `printf()`
- jeder Name kann pro Scope 1x vergeben werden, z.B. für Funktion oder Variable

- reservierte Bezeichner
  - beginnend mit zwei Unterstrichen
    - z.B. `__STDC_VERSION__`
  - Beginnend mit einem Unterstrich und einem Großbuchstaben
    - z.B. `_Bool`
- lokale Bezeichner
  - beginnend mit einem Unterstrich gelten nur innerhalb einer Datei
    - z.B. `_local`

- Typischerweise definiert durch folgenden regulären Ausdruck



C Einführung WS 2026/27 SWC3 Erik Pitzer 22

Originalfolie 02, PDF-Seite 23: Bezeichner (identifier)

2.7 Deklarationen, Definitionen, Initialisierung und Konstanten

Die **Deklaration** führt einen Namen und seine Art bzw. seinen Typ ein; die **Definition** stellt in geeigneten Fällen auch das dazugehörige Objekt oder den Funktionskörper bereit. `int sum(int, int);` ist eine Funktionsdeklaration, während `int sum(int a, int b) { return a + b; }` eine Funktionsdefinition ist. Bei Variablen benötigt man die genaue Unterscheidung: `extern int count;` deklariert ein Objekt, ohne in dieser Zeile Speicher dafür zu definieren. Dagegen ist `int count;` auf Datei- und Blockebene grundsätzlich selbst auch eine Definition beziehungsweise eine tentative Definition, wenn keine Initialisierung und keine vorangehenden Besonderheiten vorliegen. Die Folie formuliert `size_t count;` zunächst vereinfachend als bloße Deklaration; diese Unterscheidung ist für mehrfach eingebundene Header entscheidend.

Initialisierung und Zuweisung sind nicht dasselbe. `int x = 3;` legt `x` mit einem Startwert an; `x = 4;` verändert ein bereits vorhandenes Objekt. Bei `int x = {};` ist die leere Initialisierung eine C23-Schreibweise für Nullinitialisierung. `int x;` in einem automatischen lokalen Block erhält im Allgemeinen **keinen definierten Startwert**; globale und `static`-Objekte dagegen werden ohne ausdrücklichen Initialisierer nullinitialisiert. Der Designgedanke *Zero Is Initialized* empfiehlt Datenstrukturen, bei denen der Nullzustand sinnvoll ist. Aus Gründen der Lesbarkeit und zur Vermeidung irreführender Deklaratoren wird die getrennte Deklaration empfohlen: `int *a; int b;` statt `int* a, b;`, denn nur `a` ist im letzten Beispiel ein Zeiger.

`const` verbietet eine Veränderung **über den so qualifizierten lvalue**. Das bedeutet nicht, dass die Variable eine zur Übersetzungszeit auswertbare Konstante sein muss: `const int r = rand();` ist zulässig. Das Entfernen des `const`-Qualifizierers durch einen Cast macht einen schreibgeschützten Gegenstand nicht sicher beschreibbar. `constexpr` seit C23 verlangt dagegen eine konstante Initialisierung unter den entsprechenden Sprachregeln; `constexpr int x = 1;` ist ein geeignetes Beispiel. Vor C23 dienten dafür häufig Präprozessormakros wie `#define PI 3.1415`, die aber lediglich Text ersetzen und selbst keinen eigenständigen typisierten Speichergegenstand definieren.

Vertiefung · Schritt für Schritt

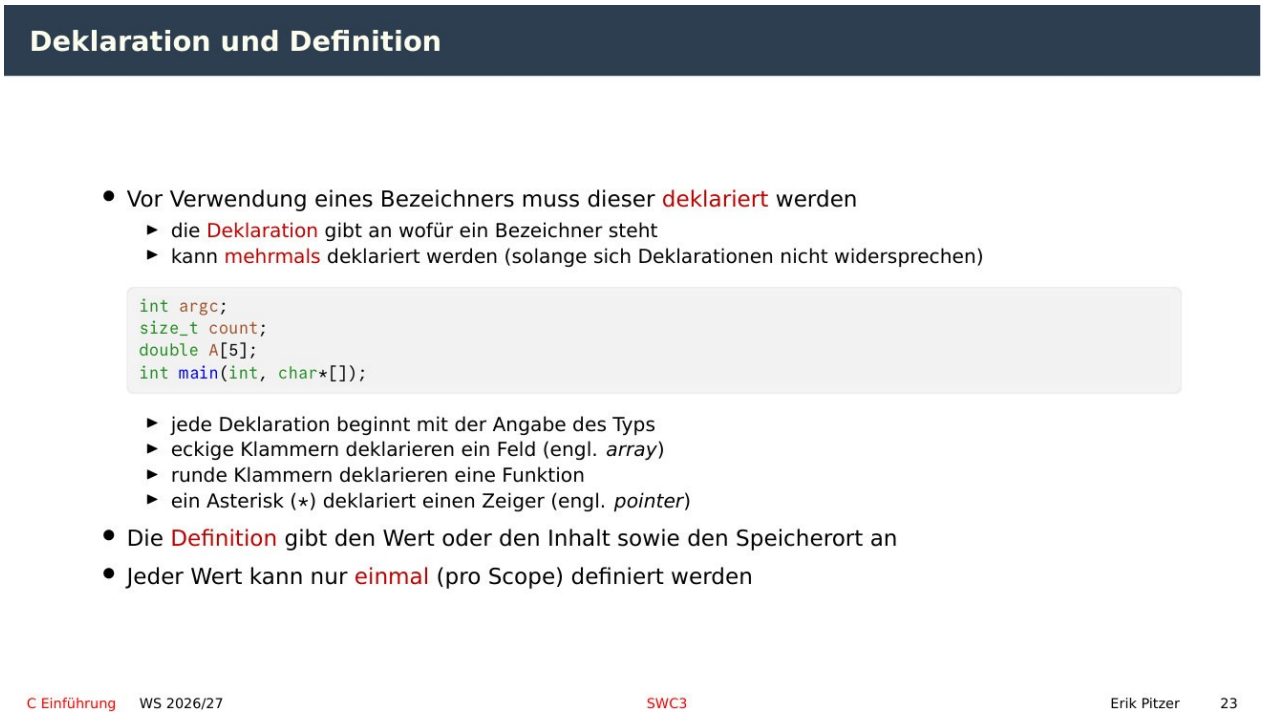
Die Unterscheidung von Deklaration, Definition, Scope und Linkage entscheidet darüber, welche Dateien erfolgreich gemeinsam gelinkt werden können. Ein Header darf `extern int` counter; enthalten, denn das kündigt nur eine

Definition an anderer Stelle an. Steht dagegen `int counter = 0;` in einem Header, der in zwei .c-Dateien inkludiert wird, entstehen zwei äußere Definitionen, die beim Linken typischerweise einen Fehler erzeugen. Eine Funktion darf oft in mehreren Headern deklariert werden, aber ihre externe Definition muss eindeutig sein. Anweisungen und Initialisierer müssen nach ihren Regeln in den richtigen Gültigkeitsbereich gestellt werden.

```
/* counter.h */
#ifndef COUNTER_H
#define COUNTER_H
extern int counter;
void count_up(void);
#endif

/* counter.c */
#include "counter.h"
int counter = 0;
void count_up(void) { ++counter; }
```

Die ZII-Regel („Zero Is Initialized“) ist eine Entwurfsidee, nicht eine automatische Initialisierung aller lokalen Objekte. Die Schreibweise `int a[3] = {};` ist seit C23 eine leere Initialisierung und erzeugt drei Nullen; ohne Initialisierer hätte ein gewöhnliches lokales Array zunächst keine garantierten Werte. `const` bedeutet lediglich, dass über einen `const`-qualifizierten Zugriff nicht verändert werden darf. `constexpr` seit C23 kann bei unterstützten geeigneten Skalarobjekten einen bereits zur Übersetzungszeit feststehenden Wert verlangen; dies ist stärker als bloßes `const`.



Originalfolie 02, PDF-Seite 24: Deklaration und Definition

Deklaration und Definition – Beispiel: Variable

- Deklaration besagt

```
size_t count;
```

  - *irgendwo* gibt es genug Speicher
  - für einen Wert vom Typ `size_t`
  - unter dem Namen `count`
- Hierbei gilt es folgende Teile zu unterscheiden
  - Name des Objekts (`count`) ist ein Bezeichner
  - Spezifikation (der Typ) ist `size_t`
  - Objekt selbst, z.B. Stelle im Speicher für die Variable
  - Wert des Objekts, z.B. `0`

- Definition besagt, dass im aktuellen Scope

```
size_t count = 0;
// alternativ auch
size_t count = {};
```

  - unter dem Namen `count`
  - genug Platz für Wert vom Typ `size_t` reserviert wird
  - und mit `0` initialisiert wird
- korrekte Null-Initialisierung durch `... = {};`
- C Idiom »Zero is initialized« (ZII)
  - Datenstrukturen so konzipieren, dass `0` eine gültige, z.B. leere Initialisierung ist
  - bedeutet aber nicht, dass alle Variablen automatisch mit `0` initialisiert werden

Originalfolie 02, PDF-Seite 25: Deklaration und Definition – Beispiel: Variable

Deklaration und Definition – Beispiel: Funktion

- Deklaration besagt

```
int count(short);
```

  - *irgendwo* gibt es genug Speicher
  - für eine Funktion vom Typ `int(short)`
    - nimmt einen `short`
    - liefert einen `int`
  - unter dem Namen `count`

- Definition besagt, dass im aktuellen Scope

```
int count(short x) { /* ... */ }
```

  - unter dem Namen `count`
  - genug Platz für eine Funktion
  - vom Typ `int(short)` reserviert wird

- Hierbei gibt es wieder
  - Name des Objekts (`count`) ist ein Bezeichner
  - Spezifikation (der Typ) ist `int(short)`
  - Objekt selbst, z.B. Stelle im Speicher die Funktion
  - Wert (Implementierung) des Objekts (in Maschinensprache)

Originalfolie 02, PDF-Seite 26: Deklaration und Definition – Beispiel: Funktion

Deklaration und Definition – Programmierstil

- Deklaration und Definition möglichst nahe oder gleichzeitig
  - Reduktion von nicht initialisierten Variablen

```
int x = 1;
```

- Immer nur eine Definition oder Deklaration pro Zeile
  - C erlaubt eine Definition fortzusetzen
  - verliert dabei aber Erweiterungen wie Zeiger oder Feld

```
int*   a, b; // ==> int *a; int b;  (misleading)
int   *a, b; // ==> int *a; int b;  (a bit better)
```

  - am besten separat definieren (und initialisieren)

```
int *a = {};
int b = {};
```


Originalfolie 02, PDF-Seite 27: Deklaration und Definition – Programmierstil

Konstanten (i)

- `const`-*qualifizierte* Variablen
  - können in schreibgeschütztem Speicher liegen
  - keine (direkten) Änderungen
  - Änderungsversuch → undefiniertes Verhalten

```
const int x = 1; // or `int const x = 1;`
SHOW(x);
// x = 2; <-- does not compile
*(int*)(&x) = 2; // XXX: remove const via cast
SHOW(x);
const int r = rand(); // initialize with function
SHOW(r);
```

```
x = 1
x = 2
r = 1804289383
```

const type qualifier - cppreference.com

- `constexpr` Kennzeichnung (seit C23)
  - echte Konstante: kann nicht verändert werden
  - muss zur Übersetzungszeit bekannt sein
  - kann nicht mit Funktion initialisiert werden

```
constexpr int x = 1;
SHOW(x);
// x = 2; <-- does not compile
*(int*)(&x) = 2; // NOTE: cannot change x
SHOW(x);
// constexpr r = rand(); <-- ERROR: does not compile
```

```
x = 1
x = 1
```

constexpr specifier - cppreference.com

Originalfolie 02, PDF-Seite 28: Konstanten (i)

Konstanten (ii)

- vor C23 wurden echte Konstanten daher oft mit Hilfe des Präprozessors definiert
  - einfache Textersetzung

```
#define PI 3.1415
SHOW_TYPE(PI);

// equivalent to

SHOW_TYPE(3.1415);
```

```
PI = 3.1415 (double)
3.1415 = 3.1415 (double)
```

C Einführung WS 2026/27 SWC3 Erik Pitzer 28

Originalfolie 02, PDF-Seite 29: Konstanten (ii)

2.8 `volatile`, `restrict`, Scope, Lebensdauer und Speicherklassen

`volatile` macht dem Compiler kenntlich, dass Zugriffe auf ein Objekt beobachtbare Bedeutung haben können, selbst wenn er im normalen Programmfluss keine Änderung erkennt, zum Beispiel bei Speicherabbildung von Hardware-Registern. **Ergänzende Einordnung:** `volatile` ersetzt weder atomare Operationen noch Synchronisation zwischen Threads. `restrict` qualifiziert einen Zeiger und ist ein Optimierungsversprechen über die Zugriffsbeziehungen während seiner Verwendung: Der Compiler darf bestimmte Aliasing-Konstellationen ausschließen. Wird das Versprechen verletzt, kann das Programm undefiniertes Verhalten haben.

Scope (Gültigkeitsbereich) bezeichnet den Quelltextbereich, in dem ein Bezeichner sichtbar ist. **Storage duration (Speicherdauer)** bezeichnet, wie lange ein Objekt existiert; beide Eigenschaften sind unabhängig. Ein Dateiobjekt `int global = 0;` hat statische Speicherdauer und wird über mehrere Funktionen hinweg weiterverwendet. `static int hidden = 0;` auf Dateiebene beschränkt zusätzlich die externe Sichtbarkeit durch interne Linkage. Eine lokale automatische Variable `int x = 0;` existiert bei jedem Blockdurchlauf erneut, während `static int calls = 0;` innerhalb einer Funktion denselben gespeicherten Wert über mehrere Aufrufe behält, obwohl der Name nur in diesem Block sichtbar ist.

Im Foliendiagramm wird ein globales `x` in `main` von einem lokalen `x` und im Rumpf einer `for`-Schleife nochmals von einem `x` **überschattet (shadowing)**. Das innere `x` bezeichnet dann ein anderes Objekt. Eine im Kopf von `for (int i=0; ... )` deklarierte Variable ist vom Beginn ihres Gültigkeitsbereichs über die Schleife sichtbar; eine in `{ ... }` des Rumpfes definierte Variable hat dagegen pro Iteration eine neue automatische Lebensdauer. Der Compiler kann dennoch physische Speicherplätze wiederverwenden. `auto` ist die gewöhnliche automatische Speicherdauer für lokale Variablen, `register` war ein Hinweis zur Speicherung, `static` kann statische Speicherdauer bzw. interne Linkage bedeuten, und `extern` verweist auf ein extern gelinktes Objekt oder eine entsprechend deklarierte Funktion. Bei Aufteilung in mehrere Dateien müssen externe Definition und passende Deklaration zusammenpassen.

Vertiefung · Schritt für Schritt

Die acht aufeinander folgenden Scope-Folien sind als schrittweise Demonstration zu lesen. Zunächst wird der globale Name x eingeführt; anschließend entsteht beim Betreten von `main` ein eigener lokaler x, der den äußeren Namen in seinem Block verdeckt. Innerhalb einer verschachtelten Schleife kann wiederum ein neuer Name x angelegt werden.

Der Compiler entscheidet für jede Verwendung eines Namens nach den lexikalischen Gültigkeitsregeln, welche Deklaration gemeint ist. Nach Verlassen des inneren Blocks ist dessen Deklaration nicht mehr sichtbar; das äußere x wird wieder angesprochen. Diese Abfolge betrifft die Namensauflösung und darf nicht mit einer bestimmten physikalischen Stackadresse verwechselt werden.

Dagegen beschreibt Lebensdauer den zeitlichen Bestand eines Objekts. Eine lokale automatic-Variable kann bei jedem Aufruf neu entstehen, während eine lokale `static`-Variable denselben gespeicherten Zustand behält. Linkage wiederum bestimmt, ob derselbe Name über mehrere Übersetzungseinheiten hinweg dasselbe Objekt oder dieselbe Funktion bezeichnet. Die drei Begriffe beantworten somit unterschiedliche Fragen: Wo ist ein Name sichtbar? Wie lange lebt sein Objekt? Können mehrere Dateien dasselbe Symbol meinen?

Weitere Zusätze

- Variablen können als unbeständig »`volatile`« deklariert werden
  - Compiler muss davon ausgehen, dass sich Variable *unbemerkt* ändern kann
  - Optimierung wird eingeschränkt
  - Wert wird immer direkt aus dem Speicher gelesen
  - z.B. Speicherabbildung (*memory mapping*) von Geräten durch Treiber
  - z.B. Variable die von anderem Thread oder Signal-Handler verwendet wird
- Zeiger (*pointer*) können eingeschränkt werden »`restrict`«
  - Compiler darf annehmen, dass sich diese Zeiger (und alles darin) nicht überlappen
  - ermöglicht zusätzliche Optimierungen

Originalfolie 02, PDF-Seite 30: Weitere Zusätze

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (i)

Bezeichner haben verschiedene Gültigkeitsbereiche (engl. *scope*)

- global: überall verfügbar
- innerhalb einer Datei (`static`)
- innerhalb eines *Blocks* { `/* ... */` }
  - Funktion
  - Schleife, Verzweigung, ...
  - anonymer Block

innerhalb eines Scopes sind alle Bezeichner

- des gleichen und
- aller umgebenden Scopes sichtbar
- die vorher deklariert wurden

Variablen haben verschiedene Lebensdauer (engl. *lifetime* oder *extent*)

- global (immer verfügbar)
  - außerhalb von Funktionen
  - innerhalb von Funktionen (`static`)
- solange Block ausgeführt wird
  - Funktion
  - Schleife, Verzweigung, ...
  - anonymer Block

Originalfolie 02, PDF-Seite 31: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (i)

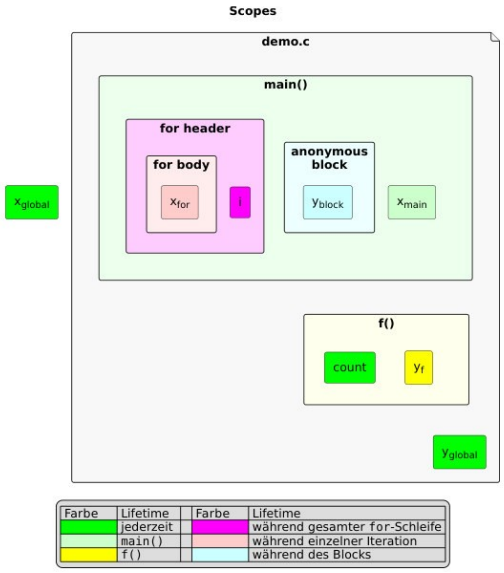
Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (ii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f(void) {
  int y = {}; // function-local y_f

  // function-private global count
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```



Farbe	Lifetime	Farbe	Lifetime
	jederzeit		während gesamter for-Schleife
	main()		während einzelner Iteration
	f()		während des Blocks

Originalfolie 02, PDF-Seite 32: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (ii)

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (iii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f(void) {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Variablen auf höchster Ebene also außerhalb von Funktionen sind verfügbar
  - scope: überall
  - lifetime: jederzeit
  - z.B. `2 int x = {}`
- *globale* `static` Variablen sind verfügbar
  - scope: nur innerhalb der Datei
  - lifetime: jederzeit
  - z.B. `3 static int y = {}`

Originalfolie 02, PDF-Seite 33: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (iii)

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (iv)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

lokale Variablen innerhalb von Blöcken sind verfügbar

- scope:
  - innerhalb des jeweiligen Blocks
  - in allen darin befindlichen Blöcken
- lifetime: solange Block ausgeführt wird
- z.B.
  - 6 `int y = {}`
  - 13 `int x = {}`
  - 15 `int x = i+1`
  - 18 `int y = {}`

bei wiederholtem Durchlauf werden Sie neu angelegt (scope+lifetime)

- z.B. erneuter Funktionsaufruf oder Schleifendurchlauf

Originalfolie 02, PDF-Seite 34: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (iv)

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (v)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};

}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

lokale `static` Variablen werden bei weiteren Funktionsaufrufen wieder verwendet

- scope wie lokale Variablen
  - innerhalb des gleichen Blocks
- lifetime wie globale Variablen (!)
- z.B. 9 `static int count = {}`
- think `static` ≈ `private global`

Originalfolie 02, PDF-Seite 35: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (v)

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (vi)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Schleifen-Kopf-Variablen sind verfügbar
  - scope
    - im Schleifenkopf und
    - im Schleifenrumpf
  - lifetime: existieren während aller Schleifendurchläufe
  - z.B. 14 `int i = 0`
- Schleifen-Rumpf-Variablen sind verfügbar
  - scope: nur im Schleifen-Rumpf
  - lifetime: nur für eine Iteration gültig, werden also bei jedem Durchlauf neu angelegt
  - 15 `int x = i+1`

Originalfolie 02, PDF-Seite 36: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (vi)

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (vii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f() {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Wenn Bezeichner wieder verwendet werden, überschatten sie die weiter außen liegenden Bezeichner
  - 2 `int x = {}` wird in
  - `main()` durch 13 `int x = {}` überschattet
  - welches selbst wiederum von 15 `int x = i+1` innerhalb der `for`-Schleife überschattet wird

Originalfolie 02, PDF-Seite 37: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (vii)

Gültigkeitsbereiche (*scope*) & Lebensdauer (*lifetime*) (viii)

```c
// file: demo.c
int x = {}; // global x_global
static int y = {}; // file-private global y_global

void f(void) {
  int y = {}; // function-local y_f

  // function-private global
  static int count = {};
}

int main() {
  int x = {}; // function-local x_main
  for (int i = 0; i<3; i++) /* for-header i */ {
    int x = i+1; // for-body x_for
  }
  {
    int y = 1; // anonymous block y_block
  }
}
```

- Wenn Bezeichner wieder verwendet werden, überschatten sie die weiter außen liegenden Bezeichner
  - 3 `static int y = {}`
    - wird in `f()` von 6 `int y = {}` überschattet
    - ist in `main()` verfügbar
    - wird im anonymen Block von 18 `int y = 1` überschattet

Originalfolie 02, PDF-Seite 38: Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (viii)

Speicherklassen (*storage classes*)

- `auto` automatische Lifetime, außen nicht sichtbar
  - default für lokale Variablen
  - `register` ermutigt den Compiler CPU-Register zu verwenden
- `static` innerhalb des aktuellen Blocks (scope) dauerhaft (lifetime)
  - auch bei erneutem Durchlauf
- `extern` global sichtbar (scope) und dauerhaft (lifetime)
  - default für globale Variablen
  - explizit normalerweise nur für Deklaration → Variable/Funktion außerhalb

```c
void function();
extern void function(); // equivalent

int x;
extern int x; // equivalent

int main() {
  int x = {};
  auto int x = {} // equivalent
  register int y = {};
}
```

```c
// file: extern-1.c

extern int x; // -->
int print_x() {
  SHOW(x);
}
```

```c
// file: extern-2.c
int x = 1;
extern int print_x(); // -->
int main() {
  print_x();
}
```

```
gcc -o extern extern-1.c extern-2.c
./extern
```

```
x = 1
```

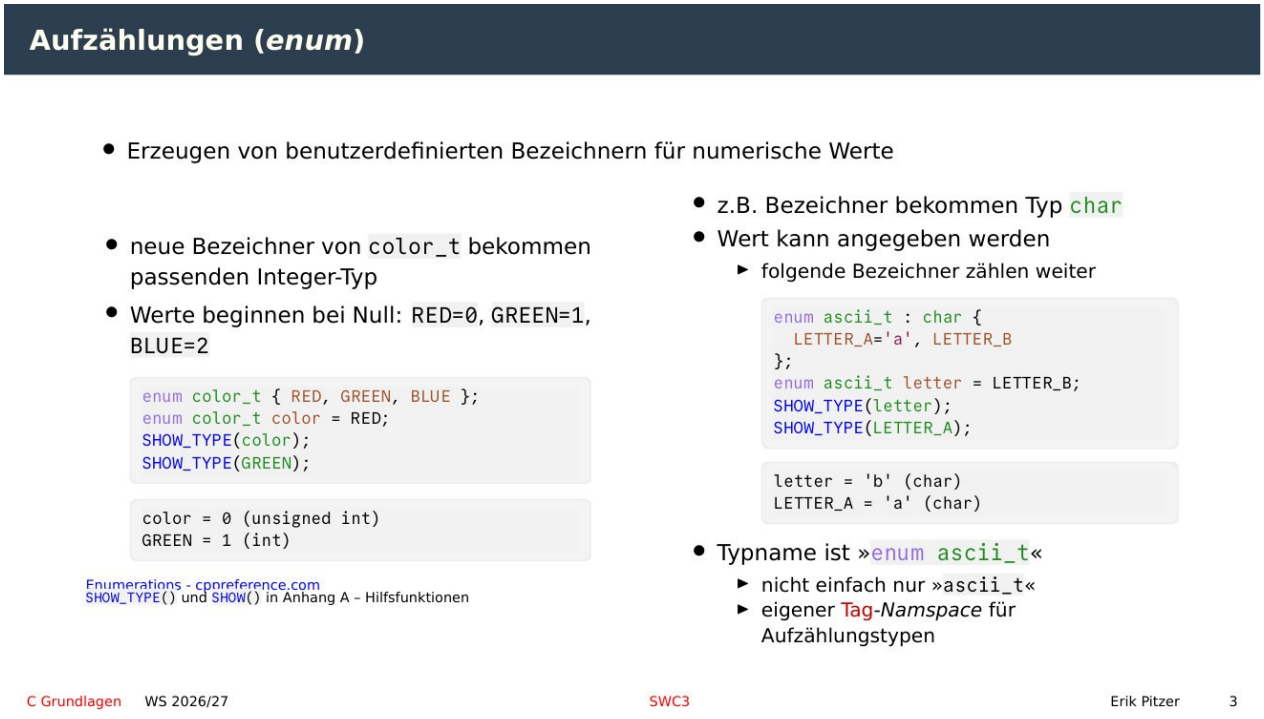

Originalfolie 02, PDF-Seite 39: Speicherklassen (storage classes)

3 C-Grundlagen I: Zusammengesetzte Datentypen, Zeiger und Arrays

3.1 Aufzählungen (`enum`)

Eine **Aufzählung** fasst benannte ganzzahlige Werte unter einem Typ zusammen, zum Beispiel `enum color_t { RED, GREEN, BLUE };`. Ohne individuelle Vorgaben erhalten sie die Werte 0, 1 und 2. Das macht Programme verständlicher, weil `if (color == BLUE)` eine Bedeutung ausdrückt, die `if (color == 2)` ohne zusätzliche Information nicht erkennen lässt. Explizite Startwerte sind ebenfalls möglich: Wird `LETTER_A = 'a'` gesetzt, erhält der nächste Eintrag ohne Angabe den jeweils folgenden ganzzahligen Wert. Die C23-Folien illustrieren außerdem einen **fest vorgegebenen unterliegenden Typ** durch `enum ascii_t : char { ... };` damit lassen sich Darstellungsanforderungen präziser steuern.

Der Tag-Name einer solchen Aufzählung ist `enum color_t`, nicht einfach `color_t`, solange kein passendes `typedef` eingeführt wurde. Tags haben in C einen eigenen Namensraum. Die Ausgabe eines enumerierten Werts über ein Diagnosemakro zeigt den tatsächlich gewählten passenden Integer-Typ einer Implementierung; man darf aus diesem Beispiel nicht schließen, dass jeder Aufzählungstyp automatisch `unsigned int` besitzt. Aufzählungen sind besonders geeignet für Zustände, Kategorien und Bitflags, wobei bei Flags eine Kombination von Einträgen durch bitweises Oder möglich sein kann.



Originalfolie 03, PDF-Seite 4: Aufzählungen (enum)

3.2 Strukturen (`struct`), Initialisierung und Speicherlayout

Eine **Struktur** enthält mehrere benannte Komponenten unter einem gemeinsamen Typ: `struct point_t { int16_t x; int16_t y; };`. Die einzelnen Felder haben jeweils ihren eigenen Typ und sind mit der Punktnotation erreichbar, beispielsweise `p.x`. Die Initialisierung `struct point_t p = {1, 2};` ordnet den Werten nach Feldreihenfolge die Komponenten `x` und `y` zu. `struct point_t p = {.y = 2};` setzt `y` ausdrücklich auf 2 und initialisiert `x` automatisch mit null. `struct point_t p = {};` ist eine C23-Nullinitialisierung. Eine nicht

initialisierte automatische Struktur enthält dagegen unbestimmte Werte; die in Folien beispielhaft gezeigten negativen Zahlen sind kein definierter Ausgangszustand.

Strukturen lassen sich als Ganzes zuweisen, beispielsweise `p0 = p1;`, und als **compound literal** schreiben: `(struct point_t){3, 4}` erzeugt ein Objekt dieses Typs mit passenden Mitgliedswerten. Ein direkter Vergleich per `==` ist für Strukturen in C dagegen nicht erlaubt; man muss relevante Mitglieder vergleichen. `sizeof(struct point_t)` gibt die Gesamtgröße in C-Bytes an.

Die Größe einer Struktur ist nicht immer die Summe der deklarierten Feldgrößen, weil der Compiler **Padding** einfügen kann. Grund dafür sind **Alignment-Anforderungen**: Ein Objekt bestimmter Größe soll häufig an einer Adresse beginnen, die ein Vielfaches einer vorgegebenen Bytezahl ist. Das Foliendiagramm zeigt für `char c; int32_t i32; int16_t i16;` eine Gesamtgröße von 12 statt 7 Bytes. `c` belegt Byte 0, danach folgen 3 Padding-Bytes, `i32` belegt 4–7, `i16` die Bytes 8–9 und am Schluss gibt es 2 Padding-Bytes. Nach Umordnung zu `char c; int16_t i16; int32_t i32;` reichen im gezeigten System 8 Bytes: ein Padding-Byte vor `i16` und dann `i32` an Adresse 4. Die Darstellung erklärt, warum eine Reihenfolge nach absteigender Ausrichtung manchmal Speicher spart. **Ergänzende Einordnung:** Die konkreten Offsets sind ABI- und architekturabhängig, und Padding-Bytes dürfen nicht als bedeutungstragende Nutzdaten betrachtet werden.

Vertiefung · Schritt für Schritt

Das Alignment-Diagramm auf der Originalfolie erklärt sich schrittweise: Unter den gezeigten Annahmen verlangt `int32_t` eine an 4 ausgerichtete Startadresse und `int16_t` eine an 2 ausgerichtete. Beginnt `struct` unaligned_t bei Adresse 0, folgt auf `char` c (Byte 0) ein Abstand von drei Bytes; i32 liegt dann in Byte 4 bis 7, i16 in Byte 8 bis 9 und danach folgen zwei weitere Padding-Bytes, damit die Gesamtgröße ein Vielfaches des Struktur-Alignments von 4 ist. Nach Umsortierung c, i16, i32 reichen unter denselben Annahmen acht Bytes. Der Compiler darf Padding einfügen; ein `struct` darf daher nicht ohne Prüfung als plattformunabhängiges Dateiformat interpretiert werden.

```
#include <stddef.h>
#include <stdint.h>

struct record { char c; int32_t value; int16_t flag; };

/* Feldoffsets und Gesamtgröße sind überprüfbar: */
size_t a = offsetof(struct record, value);
size_t b = sizeof(struct record);
```

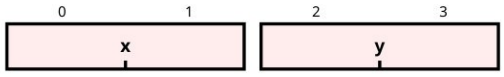
Verbunddatentypen (*struct*) (i)

- Verbunddatentyp mit mehrere Komponenten
  - eigener Tag-*Namespace* (wie `enum`)
  - kein direkter Vergleich von `structs`
    - sogar vom gleichen Typ
  - Elementzugriff `p.x` mit Punktnotation
  - Initialisierung mit geschwungenen Klammern
    - entweder Komponenten der Reihe nach
    - oder mit Punktnotation `{.x = 1}`, fehlende Element auf `0` gesetzt

```
struct point_t {
  int16_t x;
  int16_t y;
};
```

```
SHOW(sizeof(struct point_t));
```

```
sizeof(struct point_t) = 4
```



Struct declaration - cppreference.com

```
struct point_t p; // not initialized
struct point_t p0 = {}; // zero initializer
struct point_t p1 = {1, 2};
struct point_t p2 = { .y = 2 };
SHOW_XY(p); SHOW_XY(p0); SHOW_XY(p1); SHOW_XY(p2);
p.x = 123; // member access
SHOW(p.x);
```

```
p = (-258, -258)
p0 = (0, 0)
p1 = (1, 2)
p2 = (0, 2)
p.x = 123
```

```
struct point_t p0 = {};
struct point_t p1 = {1, 2};
p0 = p1; // direct assignment
SHOW_XY(p0);
p0 = (struct point_t){3, 4}; // compound literal
SHOW_XY(p0);
```

```
p0 = (1, 2)
p0 = (3, 4)
```


Originalfolie 03, PDF-Seite 5: Verbunddatentypen (struct) (i)

Verbunddatentypen (*struct*) (ii)

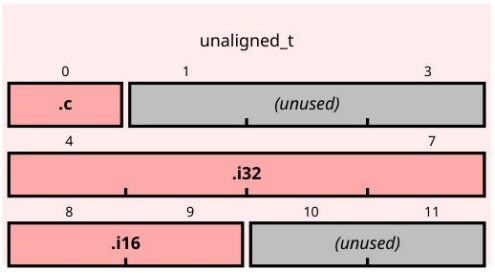
- von Compiler an Architektur ausgerichtet (*aligned*)

```
struct unaligned_t {
  char c;      //   1 byte
  int32_t i32; // + 4 byte
  int16_t i16; // + 2 byte
};             // = 7 byte?
SHOW(sizeof(struct unaligned_t));
```

```
sizeof(struct unaligned_t) = 12
```

```
struct aligned_t {
  char c;      //   1 byte
  int16_t i16; // + 2 byte
  int32_t i32; // + 4 byte
};             // = 7 byte?
SHOW(sizeof(struct aligned_t));
```

```
sizeof(struct aligned_t) = 8
```





Structures in C: From Basics to Memory Alignment

Originalfolie 03, PDF-Seite 6: Verbunddatentypen (struct) (ii)

3.3 Unions, gemeinsame Speicherbereiche und Bitfelder

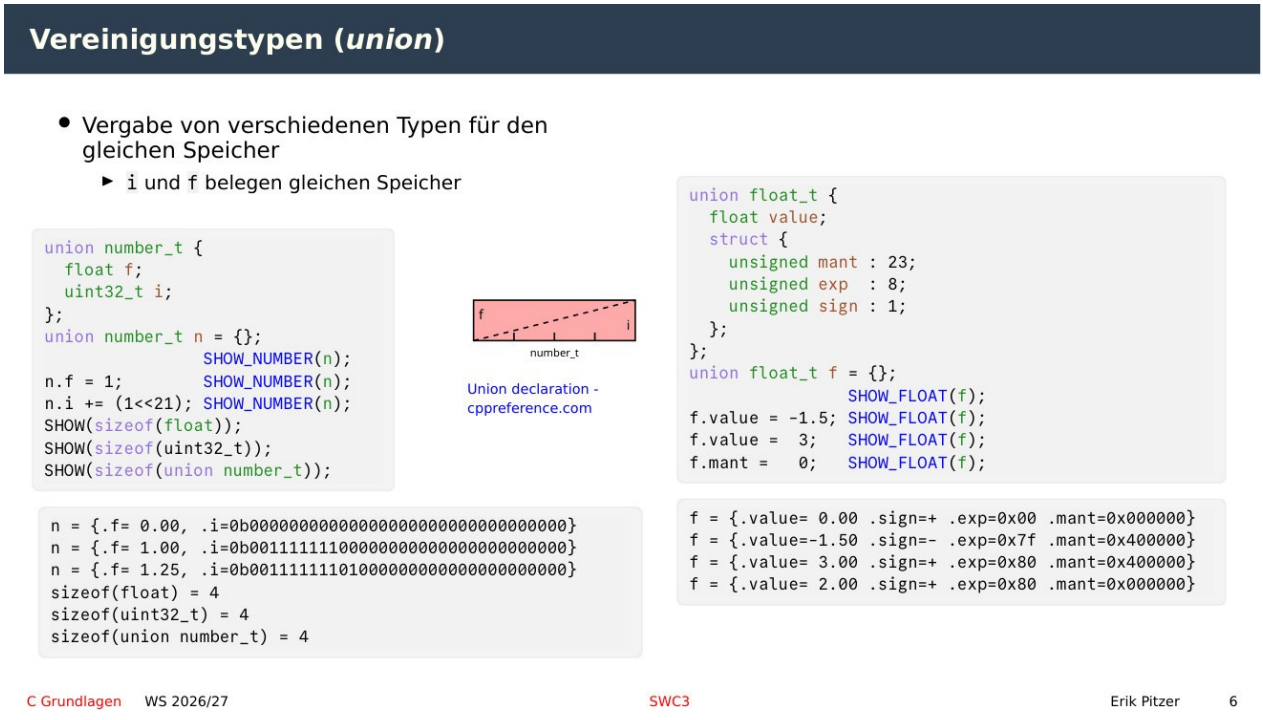
Bei einer **Union** überlagern sich die Speicherbereiche verschiedener Mitglieder. `union number_t { float f; uint32_t i; };` stellt für `f` und `i` denselben Speicher bereit; die Union ist mindestens groß genug für ihr größtes Mitglied und ausreichend ausgerichtet. Wird `n.f = 1.0f` gesetzt und das Bitmuster anschließend als 32-Bit-Ganzzahl betrachtet, kann auf einer üblichen IEEE-754-Maschine `0x3f800000` erscheinen. Wenn das Integer-Bitmuster durch die Addition `1 << 21` verändert wird, kann sich daraus die Fließkommazahl 1,25 ergeben. Dies ist eine Demonstration einer **bestimmten numerischen Speicherrepräsentation**, nicht eine portable Umrechnungsformel zwischen `float` und `uint32_t`.

Die zweite Foliendarstellung zerlegt ein 32-Bit-`float` in `sign`, `exp` und `mant` mittels **Bitfeldern** (23, 8 und 1 Bit) innerhalb einer Struktur und überlagert diese mit einem `float`. Ein Bitfeld speichert nur eine festgelegte Anzahl Bits. Seine Reihenfolge und Anordnung sind allerdings implementationsabhängig; Bitfelder dürfen deshalb ohne Prüfung nicht als universelle Beschreibung eines IEEE-754-Formats oder eines externen Dateiformats verwendet werden. Für portable Binärformate sind explizite Masken, Verschiebungen, `memcpy` und definierte Endianness-Verfahren zu bevorzugen. Die grafischen Ausgaben der Folie führen genau vor, wie Vorzeichen, Exponent und Fraction im Beispiel auf Änderungen reagieren.

Vertiefung · Schritt für Schritt

Bei einer `union` beginnen alle direkten Mitglieder am selben Speicherort, sie belegen aber nicht gleichzeitig mehrere unabhängige Werte wie Felder eines `struct`. Deshalb hat die `union` mindestens so viel Speicher wie ihr größtes Mitglied, eventuell einschließlich Alignment-Padding. Das Folienbeispiel überlagert einen `float` mit einem `uint32_t` und verändert bestimmte Bits, um den ausgegebenen Fließkommawert von 1 auf 1,25 zu beeinflussen. Diese konkrete Bitinterpretation setzt ein 32-Bit-IEEE-754-Format und passende Layoutregeln voraus. Bitfelder werden zusätzlich in einer implementationsabhängigen Reihenfolge angeordnet; ihre Darstellung ist keine portable Netzwerkspezifikation.

Für echte Dateiformate sollte man Bytes explizit zusammensetzen: Little Endian bedeutet beispielsweise, dass die niederwertigsten acht Bits zuerst im Speicher stehen. Den ganzzahligen Wert 0xABCD rekonstruiert man aus den beiden Bytes 0xCD und 0xAB als 0xCD + (0xAB << 8). Eine direkte `union`-Überlagerung mit `int16_t` oder ein ungeprüftes `struct`-Padding wäre dagegen von Typ- und Endian-Eigenschaften der Plattform abhängig.



Originalfolie 03, PDF-Seite 7: Vereinigungstypen (union)

3.4 Zeiger und const-Qualifikation

Ein **Zeiger** ist ein Wert, der auf ein Objekt oder eine Funktion verweisen kann. Im Beispiel `int x = 123; int *p = &x;` erzeugt `&x` die Adresse von `x`; die Variable `p` speichert einen Wert vom Typ `int *`. Durch `*p` liest oder verändert man das referenzierte `int`-Objekt. `p` und `x` sind zwei unterschiedliche Objekte: Das eine speichert einen Zeigerwert, das andere eine Ganzzahl. Das Diagramm der Originalfolie stellt beide an verschiedenen symbolischen Adressen dar. Es ist bewusst schematisch: Der konkrete Adresswert, die Größe eines Zeigers und das Speicherlayout sind von der Plattform abhängig.

Bei Zeigertypen ist zwischen einem konstanten Zeiger und einem Zeiger auf konstante Daten zu unterscheiden. `const int *p` oder `int const *p` heißt: Über `p` darf das referenzierte `int` nicht verändert werden; `p` selbst darf auf ein anderes Objekt zeigen. `int *const p = &x` bedeutet: Der Zeigerwert wird nach der Initialisierung nicht geändert, aber `*p` kann geändert werden. `const int *const p = &x` verbindet beide Einschränkungen. Als Leseregel hilft es, von der Deklaratorvariablen aus zu lesen und die unmittelbaren Bindungen von `*` und `const` zu prüfen.

Zeigersicherheit setzt voraus, dass der Zeiger gültig initialisiert ist, auf einen lebenden Gegenstand des passenden Typs zeigt und ein zulässiger Speicherzugriff stattfindet. `nullptr` (C23) beziehungsweise ein geeigneter Nullzeiger bezeichnet ausdrücklich keinen gültigen Gegenstand. Nullzeiger dürfen verglichen, aber nicht zum Zugriff dereferenziert werden. Auch wenn zwei Zeigerwerte als Hexadezimalzahlen gedruckt werden, folgt aus ihrer Darstellung keine allgemeine Garantie über die Adressarithmetik.

Vertiefung · Schritt für Schritt

Der Adressoperator & liefert bei einem geeigneten Objekt x dessen Adresse. Ein `int *p = &x;` legt zusätzlich eine Zeigervariable p an, die diese Adresse als Wert speichert. Der Ausdruck `*p` liefert dann einen Zugriff auf das durch p bezeichnete `int`-Objekt. Das Sternchen hat also je nach Kontext zwei verschiedene Rollen: in einer Deklaration beschreibt es den Zeigertyp, im Ausdruck dereferenziert es. Ein Zeiger darf nur dereferenziert werden, wenn er gültig ist, korrekt ausgerichtet ist und ein passendes noch lebendes Objekt bezeichnet. `nullptr` seit C23 bezeichnet einen Nullzeigerwert, der gerade kein gültiges `int`-Objekt referenziert.

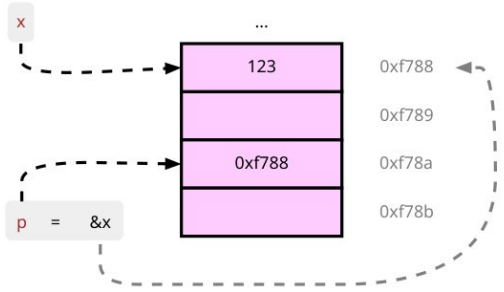
```
int main(void) {
    int x = 10;
    int *p = &x;       // Adresse von x
    *p = 25;          // verändert x
    const int *q = &x; // kein Schreiben durch q
    int *const r = &x; // r kann nicht neu ausgerichtet werden
    (void)q; (void)r;
    return x == 25 ? 0 : 1;
}
```

Beim Lesen von Deklarationen hilft die Bindung der Operatoren: `int *const` p ist ein konstanter Zeiger auf einen veränderbaren `int`; `const int` *p ist ein veränderbarer Zeiger, durch den der `int` nicht verändert werden darf; `const int *const` p kombiniert beide Regeln. Eine `const`-Qualifikation des referenzierten Typs macht das referenzierte Objekt nicht automatisch physikalisch unveränderlich, falls es ursprünglich ohne `const` angelegt wurde.

Zeiger (*pointer*)

- (jeder!) Variablenname steht für Adresse des Inhalts
  - zur Laufzeit verwendet CPU statt Variablennamen nur Adresse
- Deklaration von Zeigertyp mit Hilfe von Sternchen (*asterisk*) »`*`«
- Zeiger enthalten Adresse als Wert, z.B. die Adresse einer anderen Variablen
- Die Adresse einer Variablen erhält man mit dem Adressoperator »`&`«
- Den Wert auf den gezeigt wird erhält man mit Dereferenzierungsoperator »`*`«

```
int x = 123; // [0xf788] = 123;
int* p = &x; // [0xf78a] = 0xf788;
```



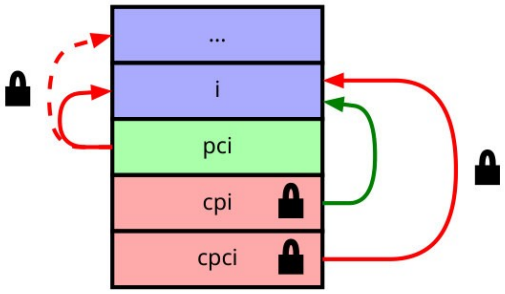
Originalfolie 03, PDF-Seite 8: Zeiger (pointer)

Konstante Zeiger

bei Zeiger und andere komplexere Datentypen kann mehr als ein Teil konstant sein

- Leserichtung von *rechts nach links*, z.B. für `int i`

- `const int * pci = &i`
  - `pci` ist
  - (veränderbarer) Zeiger
  - auf `int`-Konstante
- `int * const cpi = &i`
  - `cpi` ist
  - konstanter Zeiger
  - auf (veränderbaren) `int`
- `const int * const cpci = &i`
  - `cpci` ist
  - konstanter Zeiger
  - auf `int` Konstante



Originalfolie 03, PDF-Seite 9: Konstante Zeiger

3.5 Arrays: Länge, Initialisierung und Indexzugriffe

Ein **Array** enthält mehrere zusammenhängende Elemente desselben Typs. `char a[3] = {1, 2, 3};` ist ein Array mit genau drei `char`-Elementen, `char b[] = {1, 2, 3};` lässt die Länge aus dem Initialisierer ableiten. Indizes beginnen bei null; gültig sind `a[0]` bis `a[2]`. `sizeof(a)` liefert in diesem Kontext die Anzahl Bytes des gesamten Arrays, also drei, wenn ein `char` ein C-Byte belegt. Bei `int c[10] = {};` werden alle zehn Elemente nullinitialisiert. Ein **designated initializer** wie `int c[] = {[8] = 8, 9};` legt einen Eintrag an Index 8 auf 8 und den nachfolgenden an Index 9 auf 9; die vorherigen Elemente werden nullinitialisiert. Der Compiler ermittelt hier ein Array von zehn Elementen.

Eine typische Längenberechnung für ein tatsächliches lokales Array lautet `sizeof(c)/sizeof(c[0])`; sie liefert die Anzahl Elemente, solange `c` wirklich einen Arraytyp besitzt und nicht bereits als Zeiger behandelt wird. **Die Grenzen werden beim Zugriff nicht automatisch geprüft.** Bei einem Array der Länge 8 liegen die gültigen Indizes zwischen 0 und 7; `a[9]` überschreitet den Bereich und führt zu undefiniertem Verhalten. Die auf der Folie gezeigten Zufallszahlen beim Lesen eines nicht initialisierten bzw. außerhalb gelegenen Elements sind daher keine reproduzierbaren Ergebnisse. Speicherprüfer sollen solche Zugriffe auffinden, aber sie ersetzen nicht die korrekte Berechnung und Prüfung der Grenzen.

Ein Stringliteral `"ABC"` beansprucht ein Array von vier `char`: `'A'`, `'B'`, `'C'`, `\0`. Eine C-Zeichenkette besitzt ihre Länge nur indirekt durch das abschließende Nullbyte. Deshalb muss bei Kopien ausreichend Speicher für **Nutzzeichen plus Terminator** vorhanden sein. Fehlt der Terminator oder wird über das Ende geschrieben, entstehen Pufferüberläufe oder überlaufende Lesezugriffe. Insbesondere ist `sizeof(array)` nicht mit der Länge einer Zeichenkette gleichzusetzen und `strlen` nicht mit der allokierten Arraygröße.

Vertiefung · Schritt für Schritt

Ein Array ist ein eigenständiger Objekttyp mit fester Elementfolge. Für `int a[4]` sind genau die Indizes 0, 1, 2 und 3 gültig; der Ausdruck `a[4]` liegt bereits hinter dem Array. Der Standard führt bei einem gewöhnlichen Arrayindex keine automatische Laufzeitgrenzenprüfung aus. Dass ein unzulässiger Zugriff scheinbar einen Zahlenwert zurückgibt, beweist keine Korrektheit. In `a[i]` ist der Indexoperator definitionsgemäß äquivalent zu *(a+i); diese Gleichheit erklärt sowohl das Indexieren mit einem Zeiger als auch die Gefahr eines unbeabsichtigten Zugriffs hinter den vorgesehenen Elementbereich.

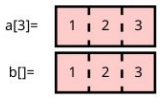
Eine C-Zeichenkette benötigt ein abschließendes Nullzeichen. Das Literal "ABC" enthält in der typischen 8-Bit-Zeichenumgebung vier `char`-Elemente: A, B, C und \0. Die Funktion `strlen` liefert dafür die logische Länge 3, `sizeof` eines echten lokalen Arrays `char` s[] = "ABC" liefert dagegen 4 Byte. Ein `char`-Zeiger enthält keine eigenständige Längeninformation. Werden Zeichen in einen zu kleinen Puffer geschrieben oder fehlt der Nullterminator, können Standardbibliotheksfunktionen über die Grenze hinaus lesen.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "ABC";
    size_t elements = sizeof s / sizeof s[0]; // 4
    size_t chars = strlen(s);                 // 3
    printf("%zu %zu\n", elements, chars);
    return 0;
}
```

Felder (*arrays*)

- zusammenhängende Folge von Werten gleichen Typs
- mit fixer Länger
- die zur Übersetzungszeit bekannt ist
- Definition mit eckigen Klammern (*brackets*) »`[]`«
- Zugriff mit Index-Operator `a[0]`
- Start bei Index Null



Array declaration - cppreference.com

```
char a[3] = {1, 2, 3};
SHOW(sizeof(a));

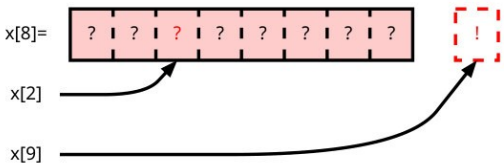
char b[] = {1, 2, 3}; // auto size
SHOW(sizeof(b));
```

```
sizeof(a) = 3
sizeof(b) = 3
```


Originalfolie 03, PDF-Seite 10: Felder (arrays)

Felder sind “*roh*”

- keine automatische Initialisierung
- keine Bereichsprüfung (*bounds check*)



- erlaubt höhere Performance
- birgt Risiken durch geringe Speichersicherheit

```
int x[8]; // XXX: not initialized
SHOW(x[2]);
SHOW(x[9]); // XXX: no bounds check
```

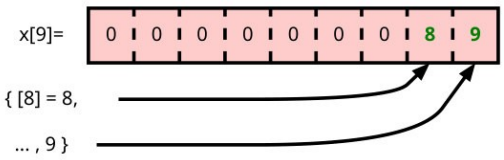
```
x[2] = 1753324224
x[9] = 32764
```

Ausgabe ohne Sanitizer

Originalfolie 03, PDF-Seite 11: Felder sind “roh”

Felder: Initialisierung

- Initialisieren und Zugriff mit Indexoperator, z.B. `c[0]`
- sobald Initialisierer angegeben »{}«
  - auch andere Indizes 0-initialisiert



- Array-Größe in Bytes kann (manchmal!) mit `sizeof` abgefragt werden
  - z.B. wenn die Größe zur Übersetzungszeit bereits feststeht

```
int c[10] = {};
SHOW(c[0]);
SHOW(c[9]);
```

```
c[0] = 0
c[9] = 0
```

```
int c[] = { [8] = 8, 9 };
SHOW(c[0]);
SHOW(c[8]);
SHOW(c[9]);
SHOW(sizeof(c)); // 10*sizeof(int)
SHOW(sizeof(c)/sizeof(c[0]));
```

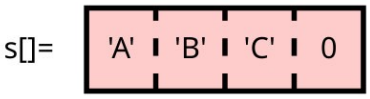
```
c[0] = 0
c[8] = 8
c[9] = 9
sizeof(c) = 40
sizeof(c)/sizeof(c[0]) = 10
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 11

Originalfolie 03, PDF-Seite 12: Felder: Initialisierung

Felder und C-Zeichenketten

- Felder sind Basis für C-Zeichenketten (*C strings*)
  - kennen eigene Länge nicht
  - Terminatorzeichen am Ende
    - ASCII Code `0`
- sehr häufige Fehlerquelle
  - z.B. Pufferüberlauf (*buffer overflow*)




```
char s[] = "ABC"; // + '\0'
SHOW(sizeof(s));
SHOW(s[0]);
SHOW((int)s[3]);
SHOW(s);
```

```
sizeof(s) = 4
s[0] = 'A'
(int)s[3] = 0
s = "ABC"
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 12

Originalfolie 03, PDF-Seite 13: Felder und C-Zeichenketten

3.6 Variable-length arrays und Array-to-pointer decay

Ein **VLA (variable-length array)** besitzt eine Länge, die erst zur Laufzeit bestimmt wird, zum Beispiel `size_t n = ...; int a[n];`. Solche Arrays werden seit C99 in bestimmten Zusammenhängen unterstützt; die Unterstützung ist je nach Standardversion und Implementierung eingeschränkt. Die Dimension muss gültig und **größer als null** sein. Eine unkontrollierte Größe kann den verfügbaren automatischen Speicher überfordern. Der in den Folien gezeigte Ausdruck `rand() % 20` kann auch 0 liefern; genau dieser Sonderfall muss überprüft werden. Ein VLA wird gewöhnlich mit Verlassen des entsprechenden Gültigkeitsbereichs automatisch freigegeben, ist aber kein Ersatz für eine sorgfältige Speicherstrategie.

In einem Funktionsparameter wie `void f(int a[])` wird der Arrayparameter zu einem **Zeigerparameter** angepasst. Es wird also **nicht das gesamte Array samt Längeninformation übergeben**. Innerhalb von `f` ist `sizeof(a)` die Größe des Zeigers, nicht die Größe des ursprünglichen Arrays. Deshalb schlägt die auf der Folie demonstrierte Längenrechnung `sizeof(a)/sizeof(a[0])` im Funktionskörper fehl. Die korrekte Variante übergibt `size_t n` ausdrücklich: `void f(size_t n, int a[static n])`. Der deklarierte Mindestumfang ist eine Voraussetzung für gültige Aufrufe und kann dem Compiler Diagnosen ermöglichen; er ergänzt keine echte Laufzeit-Längeninformation und ersetzt die tatsächliche Prüfung nicht.

Für ein Array `char a[] = {'X', 'Y', 'Z'};` gilt in den meisten Ausdrücken eine Umwandlung des Arrayausdrucks zu einem Zeiger auf das erste Element. Daher kann `char *b = a;` geschrieben werden. Die Zugriffe `a[1]`, `b[1]` und `*(b+1)` bezeichnen dasselbe zweite Element. Der Indexoperator entspricht dem Dereferenzieren des um einen Elementabstand verschobenen Zeigers. Die Addition `b+1` rechnet um `sizeof(*b)` Bytes, nicht pauschal um ein einzelnes Byte. **Arrays und Zeiger sind aber nicht identische Typen**: Ein Array ist ein zusammenhängendes Objekt festgelegter Elementanzahl, ein Zeiger ein eigenständiger Wert. Ausnahmen von Array-to-pointer decay sind zum Beispiel `sizeof`, das unäre `&` und bestimmte Initialisierungen. Zeigerarithmetik darf nur für den betreffenden Arraybereich (einschließlich der Ein-past-the-end-Position für Berechnungen, nicht Dereferenzieren) verwendet werden.

Vertiefung · Schritt für Schritt

Der Ausdruck `sizeof`(a)/`sizeof`(`a[0]`) kann die Elementzahl nur berechnen, solange a in diesem Ausdruck tatsächlich ein Array mit bekannter Gesamtgröße ist. Wird dasselbe Array in eine Funktion mit der Schreibweise `void f(int a[10])` übergeben, ist der Parameter in Wirklichkeit vom Typ `int` *; `sizeof`(a) liefert dort die Zeigergröße, nicht zehnmal die `int`-Größe. Das in den Folien demonstrierte Ergebnis zwei bei einem 8-Byte-Zeiger und 4-Byte-`int` ist daher keine Arraylänge. Eine Funktion benötigt entweder einen zusätzlich übergebenen count-Parameter oder einen anderen ausdrücklich vereinbarten Grenzmechanismus.

```c
#include <stddef.h>
#include <stdio.h>

void print_array(size_t count, const int a[static count]) {
    for (size_t i = 0; i < count; ++i) {
        printf("%d\n", a[i]);
    }
}

int main(void) {
    int values[] = {10, 20, 30};
    print_array(sizeof values / sizeof values[0], values);
    return 0;
}
```

Die Schreibweise `a[static count]` in einem Parameter ist ein Vertragsversprechen über mindestens count zugreifbare Elemente, kein Arrayobjekt und kein automatischer Laufzeitcheck. Wird diese Zusage verletzt, ist das Verhalten nicht zuverlässig. Für VLAs muss die Laufzeitlänge zudem größer als null sein; unkontrollierte Größen können den begrenzten automatischen Speicher erschöpfen. Variable-length arrays sind seit C99 vorgesehen, aber Compilerunterstützung und praktische Empfehlungen unterscheiden sich je nach Umgebung.

Felder mit variabler Länge

- Seit C99 auch Felder mit variabler Länge möglich (*variable-length arrays*, VLA)
  - Compiler erzeugt Code zum Initialisieren und Freigeben automatisch

```
srand(time(nullptr)); // initialize RNG
size_t size = rand() % 20; // random size
SHOW(size);
char a[size];
SHOW(sizeof(a));
```

```
size = 7
sizeof(a) = 7
```

was ist das problem damit?

Originalfolie 03, PDF-Seite 14: Felder mit variabler Länge

Felder zerfallen bei Parameterübergabe (i)

- Verfügbarkeit von VLAs irreführend, häufig ist Länge nicht bekannt
- bei Parameterübergabe zerfällt Array zu Zeiger (*decay*)
  - in Funktion somit nicht Länge von Array sondern Größe von `int`-Zeiger

```
/// NOTE: incorrect implementation
void show_length(int a[static 1]) {
  printf("--- inside function: ---\n");
  SHOW(sizeof(a)); // XXX: size of int pointer
  SHOW(sizeof(a[0])); // size of int
  SHOW(sizeof(a) / sizeof(a[0]));
}

int main() {
  int a[] = {1};
  SHOW(sizeof(a));
  SHOW(sizeof(a[0]));
  size_t len = sizeof(a)/sizeof(a[0]);
  SHOW(len);

  show_length(a);
  show_length(nullptr);
}
```

```
sizeof(a) = 4
sizeof(a[0]) = 4
len = 1
--- inside function: ---
sizeof(a) = 8
sizeof(a[0]) = 4
sizeof(a) / sizeof(a[0]) = 2
--- inside function: ---
sizeof(a) = 8
sizeof(a[0]) = 4
sizeof(a) / sizeof(a[0]) = 2
```

```
SHOW(sizeof(int*));
SHOW(sizeof(int));
```

```
sizeof(int*) = 8
sizeof(int) = 4
```


Originalfolie 03, PDF-Seite 15: Felder zerfallen bei Parameterübergabe (i)

Felder zerfallen bei Parameterübergabe (ii)

- korrekte Implementierung nur mit zusätzlichem Parameter möglich

```c
/// correct implementation
void show_length(size_t len, int a[static len]) {
  printf("--- inside function: ---\n");
  SHOW(len);
}

int main() {
  int a[] = {1};
  SHOW(sizeof(a));
  SHOW(sizeof(a) / sizeof(a[0]));
  show_length(sizeof(a) / sizeof(a[0]), a);
  show_length(0, nullptr);
}
```

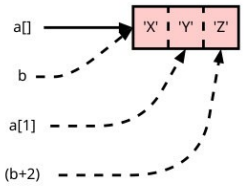
```
sizeof(a) = 4
sizeof(a) / sizeof(a[0]) = 1
--- inside function: ---
len = 1
--- inside function: ---
len = 0
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 15

Originalfolie 03, PDF-Seite 16: Felder zerfallen bei Parameterübergabe (ii)

Felder sind Zeiger

- meist sind Felder einfach Zeiger auf erstes Element
  - mit Typ-Information (also Elementgröße)
  - aber ohne Information über die Gesamtlänge des Feldes oder die Anzahl der Elemente
- Felder und Zeiger sind meist austauschbar
- Zeiger-Arithmetik is äquivalent zu Feldindexierung




```c
char a[] = {'X', 'Y', 'Z'};
char* b = a; // copy address
```

```c
SHOW(a[0]);
SHOW(*b);
```

```
a[0] = 'X'
*b = 'X'
```

```c
SHOW(a[1]);
SHOW(*(b+1));
```

```
a[1] = 'Y'
*(b+1) = 'Y'
```

```c
SHOW(*(a+2));
SHOW(b[2]);
```

```
*(a+2) = 'Z'
b[2] = 'Z'
```

```c
b[2] = 'C';
SHOW(a[2]);
```

```
a[2] = 'C'
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 16

Originalfolie 03, PDF-Seite 17: Felder sind Zeiger

3.7 `typedef` und Binärformat-Beispiel Ogg

Mit `typedef` erhält ein bereits existierender Typ einen neuen Namen; es wird **kein neuer inkompatibler Typ** geschaffen. Beispiel: `typedef struct { double x; double y; } point_t;` erlaubt `point_t p = {0.1, 0.2};`, ohne überall den Ausdruck `struct ...` zu schreiben. Die Folien zeigen sowohl benannte Aufzählungen als auch anonyme `struct`- und `enum`-Konstruktionen. Für Arraytypen kann ebenfalls ein Alias gebildet werden, etwa `typedef enum color_t color_array_t[];`.

Der Ogg-Header dient als zusammenhängendes Anwendungsbeispiel für `enum`, `struct`, `union` und Byte-Inspektion. Das verkürzte Format beginnt mit der Zeichenfolge `OggS` als **Magic Number**, enthält eine Version, Flagbits und eine in

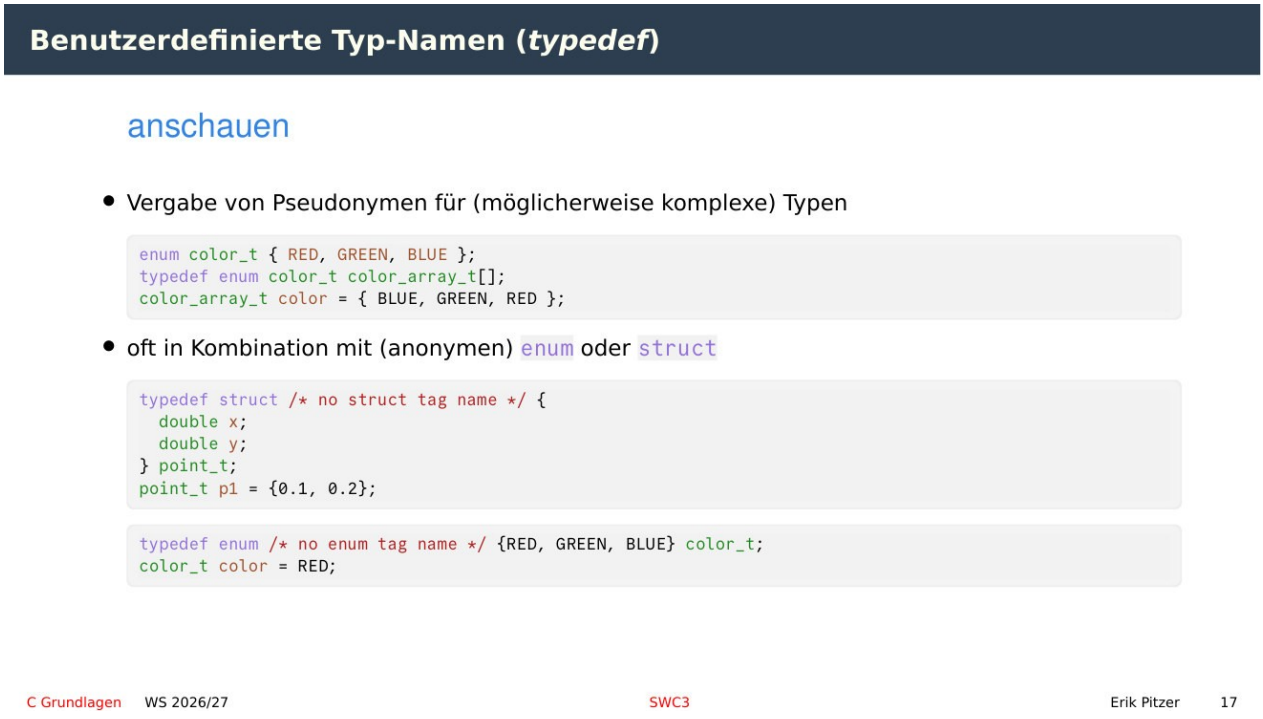
Little Endian codierte Position. Bitflags `CONTINUE = 1 << 0`, `BEGIN = 1 << 1` und `END = 1 << 2` entsprechen den Bitmustern 1, 2 und 4. Die Kombination `CONTINUE | END` ergibt 5. Die Folien definieren ein `header_t` mit 4-Byte-Magic, Version, Flags und `uint16_t granule_pos` und überlagern es in einer `union` mit `uint8_t bytes[8]`. Bei dem gezeigten Compiler sind Struktur und Union je 8 Bytes groß; in der Byteanzeige stehen am Ende `0xcd` und `0xab` für das Beispiel `0xABCD` in Little Endian.

Ergänzende Einordnung: Dieses direkt überlagerte Modell hängt vom Strukturpadding, der Bitbreite des Enumtyps und der Endianness ab. Ein portabler Parser liest externe Bytes und setzt Mehrbytewerte durch definierte Byteoperationen zusammen. Das Beispiel illustriert hervorragend das Speicherlayout, aber nicht automatisch eine auf allen Plattformen unveränderte binäre Datendeklaration.

Vertiefung · Schritt für Schritt

`typedef` erzeugt keinen neuen unabhängigen Speichertyp, sondern einen zusätzlichen Namen für einen bereits vorhandenen Typ. Beispielsweise bezeichnet `typedef struct { double` x, y; } point_t; denselben Strukturachverhalt wie eine entsprechende direkte Deklaration, während point_t anschließend als einfacher Typname verwendbar ist. Bei Zeigern ist Vorsicht geboten: `typedef int` *int_ptr; macht int_ptr zu einem Zeigertyp, sodass `const` int_ptr einen konstanten Zeiger und nicht einen Zeiger auf `const int` bezeichnet.

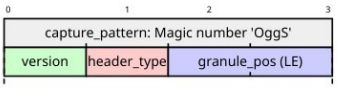
Der Ogg-Header kombiniert verschiedene Sprachmittel: Ein `enum` definiert benannte Bitflags; ein `struct` fasst zusammengehörende Felder zusammen; eine `union` erlaubt eine Byteansicht und eine Feldansicht desselben Speicherbereichs. In der im Diagramm gewählten Little-Endian-Anordnung ist die Folge CD AB die Darstellung des 16-Bit-Werts 0xABCD. Die hexadezimale Byteanzeige aus dem Diagnosemakro zeigt damit die reale Darstellung einer spezifischen Maschine. Für portable Parser darf man weder eine feste `struct`-Größe noch die Endian-Reihenfolge ungetestet annehmen.



Originalfolie 03, PDF-Seite 18: Benutzerdefinierte Typ-Namen (typedef)

Beispiel Ogg-Header (gekürzt)

anschauen




```c
typedef enum : uint8_t {
  CONTINUE = 1 << 0,
  BEGIN    = 1 << 1,
  END      = 1 << 2,
} flags_t;

typedef struct {
  char magic[4];
  uint8_t version;
  flags_t header_type;
  uint16_t granule_pos;
} header_t;

typedef union {
  header_t fields;
  uint8_t bytes[8];
} packet_t;
```

```c
int main() {
  packet_t ogg_header = {
    .fields = {
      {'O', 'g', 'g', 'S'},
      1, CONTINUE | END, 0xABCD
     }
  };

  SHOW(sizeof(ogg_header));
  SHOW(sizeof(header_t));
  SHOW(sizeof(packet_t));
  SHOW(ogg_header.fields.header_type);
  SHOW_HEX_CHARS(ogg_header.bytes);
}
```

```
sizeof(ogg_header) = 8
sizeof(header_t) = 8
sizeof(packet_t) = 8
ogg_header.fields.header_type = 5
 0: 0x4f 'O' |  1: 0x67 'g' |  2: 0x67 'g' |  3: 0x53 'S' |
 4: 0x01  ?  |  5: 0x05  ?  |  6: 0xcd  ?  |  7: 0xab  ?  |
```

RFC 3533 - Ogg Encapsulation Format v0

Originalfolie 03, PDF-Seite 19: Beispiel Ogg-Header (gekürzt)

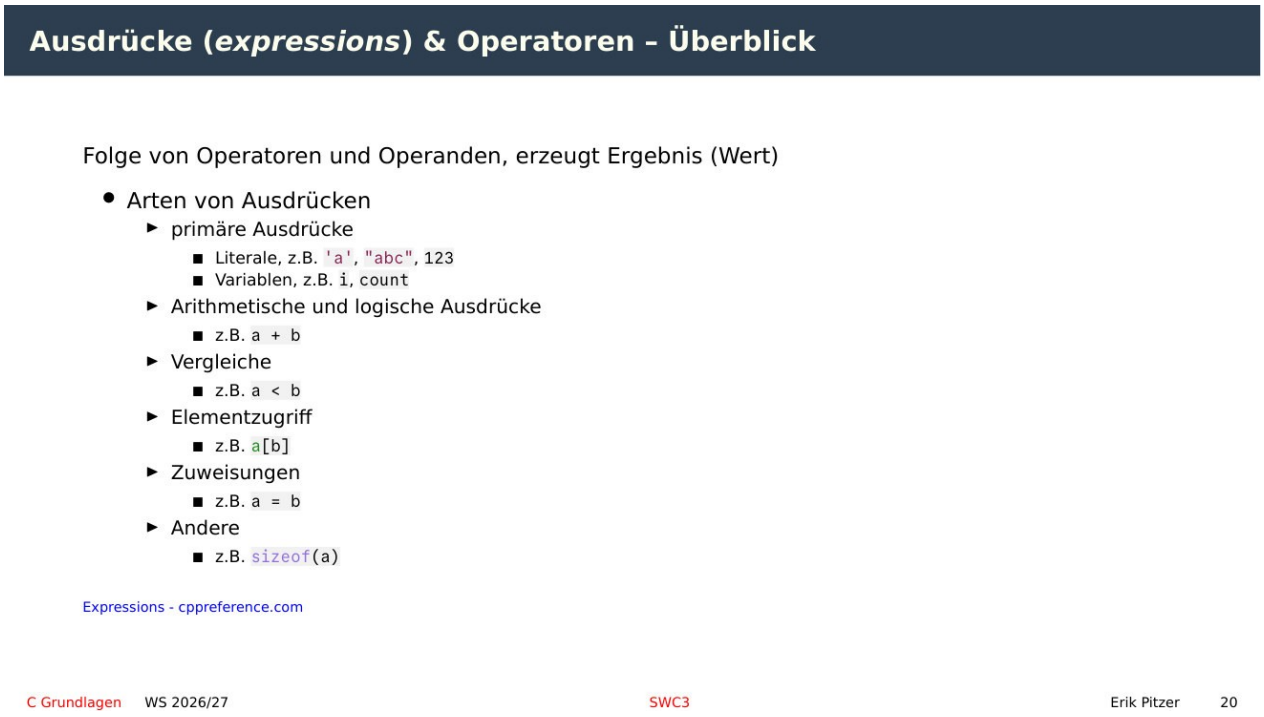
4 C-Grundlagen II: Ausdrücke, Typkonvertierungen und Kontrollfluss

4.1 Ausdrücke und die wichtigsten Operatoren

Ein **Ausdruck** kombiniert Operanden und Operatoren und kann einen Wert sowie Seiteneffekte erzeugen. Primäre Ausdrücke sind Namen und Literale, beispielsweise `count`, `123` oder `'a'`. Arithmetische Operatoren sind `+`, `-`, `*`, `/` und `%`. Bei ganzzahliger Division werden Nachkommastellen abgeschnitten (in Richtung null); `%` ergibt den Rest nach der ganzzahligen Division. Für einen Divisor null sind ganzzahlige Division und Restbildung undefiniert. Vergleiche wie `a == b`, `a != b`, `a < b`, `a <= b`, `a > b` und `a >= b` liefern logische Wahrheitswerte beziehungsweise im klassischen C die ganzzahligen Werte null oder eins.

Bei `a++` wird der alte Wert als Ausdrucksergebnis verwendet und der gespeicherte Wert erhöht; bei `++a` erfolgt die Erhöhung vor dem Gebrauch des Ausdruckswerts. `--` verhält sich entsprechend für die Verringerung. Bitweise Operationen `~`, `&`, `|`, `^`, `<<`, `>>` wirken auf Bitmuster ganzzahliger Werte; ihre Bedeutung unterscheidet sich von den logischen Operatoren `!`, `&&`, `||`. Das logische `&&` wertet die rechte Seite nur aus, wenn die linke wahr ist, und `||` nur, wenn die linke falsch ist (**Kurzschlussauswertung**). Deshalb bleibt der Aufruf `printf("logic")` in `true || printf("logic")` aus, während `true | printf("bitwise")` beide Operanden auswertet. Die Schreibweise `a += b` führt die entsprechende Zuweisungsoperation aus; sie bedeutet im Wesentlichen `a = a + b`, wertet ihren linken Operanden aber nur einmal aus.

Weitere Operatoren sind `a[b]` für einen Elementzugriff, `a.b` für das Mitglied einer Struktur, `p->m` als Abkürzung von `(*p).m`, `&a` für die Adresse, `*p` zum Dereferenzieren, `sizeof` für den Objektgrößenwert, `alignof` für die geforderte Ausrichtung, `(T)x` für Casts, `cond ? yes : no` als bedingter Ausdruck und der Kommaoperator `(a, b)`, der zunächst `a` auswertet und das Ergebnis von `b` liefert. **Achtung:** Das Komma als Trennzeichen in Argument- oder Initialisiererlisten ist nicht notwendigerweise der Kommaoperator.



Originalfolie 03, PDF-Seite 21: Ausdrücke (expressions) & Operatoren – Überblick

Ausdrücke – Arithmetik, Logik, Zuweisungen, Vergleiche

- Vorzeichen (`+a`, `-a`)
- Rechnen
  - `a * b`, `a + b`, `a - b`, `a * b`, `a / b`, `a % b`
  - Modulo-Operation / Rest der Division (`%`) nur für ganze Zahlen
- Inkrement und Dekrement
  - `a++`, `++a`, `a--`, `--a`
- Bit-Operationen
  - Negation `~a`
  - AND `a & b`, OR `a | b`, XOR, `a ^ b`
  - Left-Shift `a << b`, Right-Shift `a >> b`
- Logische Operationen
  - Verneinung `!a`, AND `a && b`, OR `a || b`
  - Auswertung im Kurzschlussverfahren

```
true || printf("logic"); // --> no output
true | printf("bitwise"); // --> "bitwise"
```

warum das?

- Einfache Zuweisung `a = b`
  - liefert zugewiesenen Wert
  - kann z.B. verkettet werden `a = b = c = d`
- Arithmetische Zuweisung
  - `a -= b`, `a *= b`, `a /= b`, `a %= b`, `a |= b`, `a ^= b`, `a <<= b`, `a >>= b`
  - verkürzte Schreibweise, kombiniert Operation und Zuweisung
    - z.B. `a += b` → `a = a + b`
- Vergleiche
  - `a == b`, `a != b`
  - `a < b`, `a <= b`, `a >= b`, `a > b`

Originalfolie 03, PDF-Seite 22: Ausdrücke – Arithmetik, Logik, Zuweisungen, Vergleiche

Ausdrücke – Weitere Operatoren

- Funktionsaufruf `printf()`
- Elementzugriffe
  - Indexoperator für Feld `a[b]`
  - Punktoperator für Verbund (*struct*) `a.b`
  - Dereferenzierung von Zeiger `*a`
  - Adresse von Bezeichner `&a`
  - Kombination von Dereferenzierung und Verbundzugriff `a->b` ⇔ `(*a).b`
- `sizeof(a)`
  - Größe eines Typs oder Ausdrucks in Byte
  - z.B. `sizeof(int_fast32_t)` → 8
- `alignof(a)`
  - Alignment eines Typs oder Ausdrucks
  - z.B. `alignof(char)` → 1, `alignof(long double)` → 16

was das bro

- Sequenz-Operator `a, b`
  - führt beide Ausdrücke aus
  - liefert Ergebnis des zweiten

```
int a;
(a = 123, 456) // --> 456
a // --> 123
```

  - selten benutzt
    - z.B. Intitialisierung von `for`-Schleife
- Typumwandlung (*cast*) `(type)a`
- Ternärer Operator `a ? b : c`
  - `if` als Ausdruck
  - falls `a` wahr ist liefert `b`
  - sonst `c`

```
10 > 5 ? "big" : "small" // --> "big"
```


Originalfolie 03, PDF-Seite 23: Ausdrücke – Weitere Operatoren

4.2 Operatorpräzedenz, Assoziativität und Auswertungsreihenfolge

Präzedenz bestimmt, wie Operatoren syntaktisch gruppiert werden. Beispielsweise wird `a + b * c` als `a + (b * c)` verstanden, weil Multiplikation stärker bindet als Addition. Bei gleicher Priorität ist die **Assoziativität** relevant: `a - b - c` gruppiert linksassoziativ als `(a-b)-c`, während `a = b = c` rechtsassoziativ als `a=(b=c)` gruppiert wird. Die vollständige Tabelle der Originalfolie listet von hoch nach niedrig unter anderem Postfixoperatoren, Präfixoperatoren, Multiplikation, Addition, Shifts, Vergleich, Gleichheit, bitweise Operatoren, logisches AND/OR, `?:`, Zuweisung und schließlich den Kommaoperator.

Von dieser rein **syntaktischen Gruppierung** ist die **zeitliche Reihenfolge der Auswertung** zu unterscheiden. Bei `f() + g()` ist normalerweise nicht vorgeschrieben, welche Funktion zuerst aufgerufen wird. Wenn beide Funktionen Ausgaben schreiben, kann deshalb einmal `fg` und einmal `gf` entstehen, ohne dass die Addition selbst schon undefiniert wäre. Die Folie kennzeichnet `printf("a") + printf("b")` pauschal als „undefined“; präziser ist: Die Reihenfolge der beiden Ausgaben ist **nicht festgelegt**, beide Aufrufe sind bei diesem Beispiel aber nicht allein deshalb undefiniertes Verhalten. Problematischer ist `a[i] = a[++i];`, weil Lesen und Ändern von `i` nicht in erforderlicher Weise sequenziert sind. Hier können sich echte Probleme des nicht sequenzierten Zugriffs ergeben. Solche Ausdrücke sollen in mehrere Anweisungen zerlegt werden, etwa `a[i] = a[i+1]; ++i;`, sofern dies die beabsichtigte Berechnung ist.

Die Faustregel der Folien ist daher begründet: komplexe Ausdrücke mit Klammern klar gruppieren und Seiteneffekte nicht unnötig kombinieren. Der Compiler darf einfache, sichere Schreibweisen entsprechend der *as-if rule* optimieren, sofern beobachtbares korrektes Programmverhalten erhalten bleibt.

Vertiefung · Schritt für Schritt

Präzedenz und Assoziativität beantworten zunächst eine syntaktische Frage: Wie werden Operatoren in einen Ausdruck gruppiert? Bei 2 + 3 * 4 wird wegen höherer Multiplikationspräzedenz die Teilrechnung 3 * 4 gruppiert, also das Ergebnis 14 erhalten. Sie legen aber im Allgemeinen nicht fest, welcher Operand zuerst evaluiert wird. Für `f()` + `g()` ist damit keine allgemeine Links-nach-rechts-Reihenfolge vorgegeben. Die Operatoren && und || evaluieren dagegen zuerst die linke Seite und lassen den rechten Ausdruck bei feststehendem logischem Ergebnis aus. Ein Kommaoperator sequenziert beide Teilausdrücke; ein Komma in der Argumentliste einer Funktion ist nicht automatisch dieser Operator.

```
int main(void) {
    int x = 0;
    if (0 && ++x) { /* unerreichbar */ }
    // Die rechte Seite wird nicht ausgewertet: x bleibt 0.
    int a = 2 + 3 * 4; // 14 wegen Präzedenz
    return (x == 0 && a == 14) ? 0 : 1;
}
```

Der Ausdruck `a[i]` = `a[++i]` aus der Originalfolie ist ein Beispiel für unsequenzierte, voneinander abhängige Auswertungen und damit undefiniertes Verhalten. Man sollte ihn in mehrere Anweisungen zerlegen, deren Reihenfolge klar ist. Zwei voneinander unabhängige Ausgaben wie `printf`("a") + `printf`("b") können hingegen je nach Auswertungsreihenfolge ab oder ba ausgeben; ihre Reihenfolge ist nicht festgelegt, aber diese bloße Reihenfolgefrage darf nicht pauschal mit undefiniertem Verhalten gleichgesetzt werden.

Reihenfolge der Operator-Auswertung (*precedence*)

Operatoren haben eine definierte Auswertungsreihenfolge

- Priorität 1 ⟶ (von Links nach Rechts)
  - suffix Inkrement/Dekrement `a++`, `a--`
  - Funktionsaufruf, Feldzugriff `a()`, `a[b]`
  - Verbundliterale `(point_t){1, 2}`
- Priorität 2 ⟵ (von Rechts nach Links)
  - Prefix Inkrement/Dekrement `++a`, `--a`
  - Vorzeichen & Verneinung `+a`, `-a`, `!a`, `~a`
  - Deref- & Adressoperator `*a`, `&a`
  - `sizeof(a)`, `alignof(a)`
- Priorität 3, 4 ⟶
  - Multiplikation & Division `a*b`, `a/b`
  - Addition & Subtraktion `a+b`, `a-b`
- Priorität 5 ⟶
  - Bitshift `a>>b`, `a<<b`
- Priorität 6, 7 ⟶
  - Ungleichheit `a<b`, `a<=b`, `a>=b`, `a>b`
  - Gleicheit `a==b`, `a!=b`
- Priorität 8, 9, 10 ⟶
  - Bitoperationen `a & b`, `a^b`, `a|b`
- Priorität 11, 12 ⟶
  - Logische Operationen `a && b`, `a || b`
- Priorität 13 ⟵
  - Ternärer Operator `a ? b : c`
- Priorität 14 ⟵
  - Zuweisungsoperatoren `a = b`, `a += b`, ...
- Priorität 15 ⟶
  - Sequenzoperator `a, b`

Tipp

Im Zweifelsfall Klammern verwenden

Originalfolie 03, PDF-Seite 24: Reihenfolge der Operator-Auswertung (precedence)

Reihenfolge der Auswertung von Operanden

Operanden auf gleicher Ebene haben keine definierte Auswertungsreihenfolge

- außer bei Kurzschlussevaluierung

```
printf("a") && printf("b"); // --> ab
printf("a") + printf("b"); // XXX: undefined
//    --> ab
// OR --> ba
a[i] = a[++i]; // XXX: undefined
//    --> a[i] = a[i+1]; i = i+1;
// OR --> a[i+1] = a[i]; i = i+1;
```

Seiteneffekte in Ausdrücken generell vermeiden, besser

```
a[i] = a[i+1];
++i;
```

- Optimierung dem Compiler überlassen[1]

[1] beide Versionen erzeugen den gleichen Maschinencode https://godbolt.org/z/e3zzx63Y8

Originalfolie 03, PDF-Seite 25: Reihenfolge der Auswertung von Operanden

4.3 Integer Promotions und übliche arithmetische Umwandlungen

Bei vielen arithmetischen Operatoren werden **schmale ganzzahlige Typen** wie `char`, `short` und `bool` vor der Operation zu `int` oder erforderlichenfalls `unsigned int` **promoted**. Deshalb hat `short x=1, y=2;` den Ausdruck `x+y` gewöhnlich vom Typ `int`. Auch `bool a=false, b=true; a+b` ergibt den numerischen Wert 1 eines Integerergebnisses; `a && b` liefert 0, weil mindestens einer der Operanden falsch ist. Das erklärt, warum die Größe des Ausdrucksergebnisses nicht automatisch der Größe jedes Operanden entspricht.

Bei gemischten Typen folgen die **usual arithmetic conversions**. Bei ganzzahligen Typen werden Rang und Vorzeichen berücksichtigt, sodass `long + int` häufig `long` ergibt und `long long + int` häufig `long long`. Bei binären

Fließkommatypen führt `float + double` zu `double` und `float + long double` zu `long double`; `float + float` bleibt grundsätzlich `float`, sofern keine andere Sprachregel eingreift. Das spätere Speichern eines Werts in einem schmaleren Typ, beispielsweise `float f = (long double)3;`, kann Präzision verlieren.

Besonders eindrücklich ist das Folienbeispiel, in dem `LONG_MAX` als `float` verarbeitet wird: der große Integerwert erscheint nur noch gerundet als etwa `9.22337e+18`. Ein `float` besitzt auf dem gezeigten System viel weniger signifikante Bits als dieser 64-Bit-Integer benötigt. Diese **Narrowing Conversion** muss nicht zu einem sichtbaren Programmabbruch führen, kann aber exakte Zählwerte oder Geld- und Zeitberechnungen verfälschen. Der GCC-Schalter `-Wconversion` hilft beim Auffinden verdächtiger stiller Umwandlungen. Präzision, Wertebereich und konkrete Zieltypgröße müssen getrennt betrachtet werden.

Vertiefung · Schritt für Schritt

Die üblichen arithmetischen Konversionen erfolgen in Stufen. Zuerst werden kleine Integer-Typen bei vielen Operatoren zu `int` beziehungsweise `unsigned int` befördert, sofern die Darstellung dies erfordert und zulässt. Danach wird zwischen den verbleibenden Operanden ein gemeinsamer Rechentyp bestimmt, etwa wegen eines `double`-Operanden der Typ `double`. Wird ein negativer `signed`-Wert mit einem hinreichend breiten `unsigned`-Typ zusammengeführt, kann ein scheinbar überraschend großer `unsigned`-Wert entstehen. Deshalb muss man Typ, Promotions, Operation und erst danach die Zuweisung getrennt betrachten.

```
#include <stdbool.h> // C17; unter C23 nicht nötig

int main(void) {
    unsigned int u = 1u;
    int i = -1;
    // Negatives signed wird für den Vergleich zu unsigned.
    bool result = (i < u); // false auf üblichen 32-Bit-int-Systemen
    return result ? 1 : 0;
}
```

Besonders tückisch ist die Mischung von Integer- und Fließkommawerten. Eine Umwandlung großer ganzer Zahlen in `float` verliert gegebenenfalls niedrige Bits, während umgekehrt die Umwandlung eines zu großen Fließkommawerts nach `int` außerhalb des darstellbaren Bereichs undefiniert ist. Die in den Folien gezeigten Werte sind daher Ergebnisse eines konkreten Compilers und sollen nicht die Prüfregeln des Standards ersetzen.

Arithmetische Ausdrücke mit schmalen Datentypen

- sogenannte schmalen Datentypen (*narrow types*)
  - `bool`, `unsigned char`, `unsigned short`
  - `char`, `signed char`, `short`
- werden vor der Berechnung befördert (*promoted*)
  - das Ergebnis ist meist vom Typ `int`

```
short x = 1;
short y = 2;
SHOW_TYPE(x);
SHOW_TYPE(y);
SHOW_TYPE(x + y);
```

```
x = 1 (short int)
y = 2 (short int)
x + y = 3 (int)
```

```
bool a = false;
bool b = true;
SHOW_TYPE(a);
SHOW_TYPE(b);
SHOW_TYPE(a + b);
SHOW_TYPE(a && b);
```

```
a = 0 (bool)
b = 1 (bool)
a + b = 1 (int)
a && b = 0 (int)
```


Originalfolie 03, PDF-Seite 26: Arithmetische Ausdrücke mit schmalen Datentypen

Arithmetische Ausdrücke mit unterschiedlichen Datentypen (i)

```
int i = 1;
long l = 2;
long long ll = 3;
SHOW_TYPE(l+i);
SHOW_TYPE(i+ll);
i = ll + l; // NOTE: narrowing conversion
SHOW_TYPE(i);
```

```
l+i = 3 (long int)
i+ll = 4 (long long int)
i = 5 (int)
```

```
float f = 1;
double d = 2;
long double ld = 3;
SHOW_TYPE(f+d);
SHOW_TYPE(f+ld);
f = d + ld; // NOTE: narrowing conversion
SHOW_TYPE(f);
SHOW_TYPE(f+f);
```

```
f+d = 3 (double)
f+ld = 4 (long double)
f = 5 (float)
f+f = 10 (float)
```

- Compilerwarnung für implizite Konvertierungen, z.B. GCC mit `-Wconversion`

Originalfolie 03, PDF-Seite 27: Arithmetische Ausdrücke mit unterschiedlichen Datentypen (i)

Arithmetische Ausdrücke mit unterschiedlichen Datentypen (ii)

```
float f = 1;
long int i = LONG_MAX;
SHOW_TYPE(i);
SHOW_TYPE(f+i); // NOTE: narrowing conversion (long int -> float)
```

```
i = 9223372036854775807 (long int)
f+i = 9.22337e+18 (float)
```

- implizite Typumwandlung generell vermeiden

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 27

Originalfolie 03, PDF-Seite 28: Arithmetische Ausdrücke mit unterschiedlichen Datentypen (ii)

4.4 Anweisungen, Blöcke und Verzweigungen

Eine **Anweisung** steuert die Programmausführung. Die einzelne leere Anweisung `;` hat keine Wirkung. Ein Ausdruck gefolgt von Semikolon ist eine Ausdrucksanweisung: `a = b;` weist zu, `f(a);` ruft eine Funktion auf, während `a + b;` ohne weitere Seiteneffekte nur ein ungenutztes Ergebnis berechnet. Ein **Block** `{ ... }` fasst mehrere Deklarationen und Anweisungen zu einer zusammengesetzten Anweisung zusammen. Variablen wie `b` und `c`, die in getrennten Blöcken deklariert sind, sind danach außerhalb dieser Blöcke nicht mehr sichtbar.

`if (bedingung) ... else ...` wählt genau den relevanten Zweig. In C gilt eine Bedingung als wahr, wenn ihr skalarer Wert **ungleich null** ist; null ist falsch. Eine Formulierung auf einer Folie, bei der „ungleich 0 (= false)“ steht, vertauscht daher offensichtlich die Bedeutung: **ungleich null bedeutet true**, nicht false. `switch (ausdruck)` prüft einen ganzzahligen Ausdruck gegen konstante `case`-Werte. Ohne `break`, `return` oder vergleichbare Kontrollflussänderung läuft die Ausführung vom gefundenen `case` in die folgenden Anweisungen weiter (**Fall-through**). `default` ist der optionale Fall für keine passende Marke. Die syntaktische Schachtelung und die Frage, welchem `if` ein `else` zugeordnet wird, sollte durch Klammern eindeutig gemacht werden.

Vertiefung · Schritt für Schritt

Ein Block in geschweiften Klammern ist eine eigene Anweisung und kann lokale Deklarationen sowie mehrere Unteranweisungen enthalten. Ein `if` interpretiert eine skalare Bedingung als falsch, wenn sie mit null gleichwertig ist, und sonst als wahr. Ein `else` gehört syntaktisch zum nächstliegenden noch nicht abgeschlossenen `if`; durch Klammern wird die gewünschte Zugehörigkeit eindeutig. `switch` wertet einen ganzzahligen Steuerwert aus und springt zu einem passenden `case`-Label oder default. Ohne `break` läuft die Ausführung in den folgenden `case`-Abschnitt weiter (fallthrough), was bewusst eingesetzt, aber meist markiert werden sollte.

Einfache Anweisungen

- leere Anweisung »`;`«
- Ausdrucksanweisung »`expression;`«

```
a + b; // no effect
a = b;
f(a);
```

- Blockanweisung durch Klammerung »`{ /* block */ }`«
  - kann eigene Deklarationen und Definitionen enthalten
  - kann beliebige Anweisungen enthalten
  - kein Semikolon am Ende

```
int a = 1;
{
  int b = a + 1;
}
{
  int c = a + 2;
}
```


Originalfolie 03, PDF-Seite 30: Einfache Anweisungen

Verzweigungen

- Binäre Verzweigung mit `if`
  - wenn Bedingung (*condition*) ungleich `0` (=false) wird Konsequenz ausgeführt
  - ansonsten die Alternative (falls mit `else` angegeben)

```
if (condition_expr)
  consequence_statement
else
  alternative_statement
```

- Mehrfachverzweigung mit `switch` und `case`
  - Entscheidungsausdruck muss ganzzahlig sein (ein `int`-Typ)

```
switch(expr) {
  case 1: statement1; ...; // fall-through to case 2
  case 2: statement2; ...; break;
    ...
  default: default_statement;
}
```


Originalfolie 03, PDF-Seite 31: Verzweigungen

4.5 Schleifen, `break`, `continue`, `goto` und `longjmp`

Die `while`-Schleife prüft eine Bedingung **vor** jedem Durchlauf, `do ... while` erst **nach** dem ersten Durchlauf. Daher wird `do` mindestens einmal ausgeführt. Eine `for`-Schleife besitzt in typischer Form `for (initialisierung; bedingung; fortschritt) { ... }`. Die Initialisierung erfolgt einmal, die Bedingung vor jeder Wiederholung und der Fortschritt nach jedem vollständig durchlaufenen Körper beziehungsweise nach `continue`. Bei `for (;;) { ... }` sind alle drei Teile weggelassen; ohne anderen Abbruch entsteht eine potenziell unendliche Schleife.

`break` beendet die **innerste** umgebende Schleife oder den aktuellen `switch`, während `continue` die nächste Iteration der innersten Schleife vorbereitet. Im Originalbeispiel `for (size_t i=0; i<10; i++) { if (i<5)`

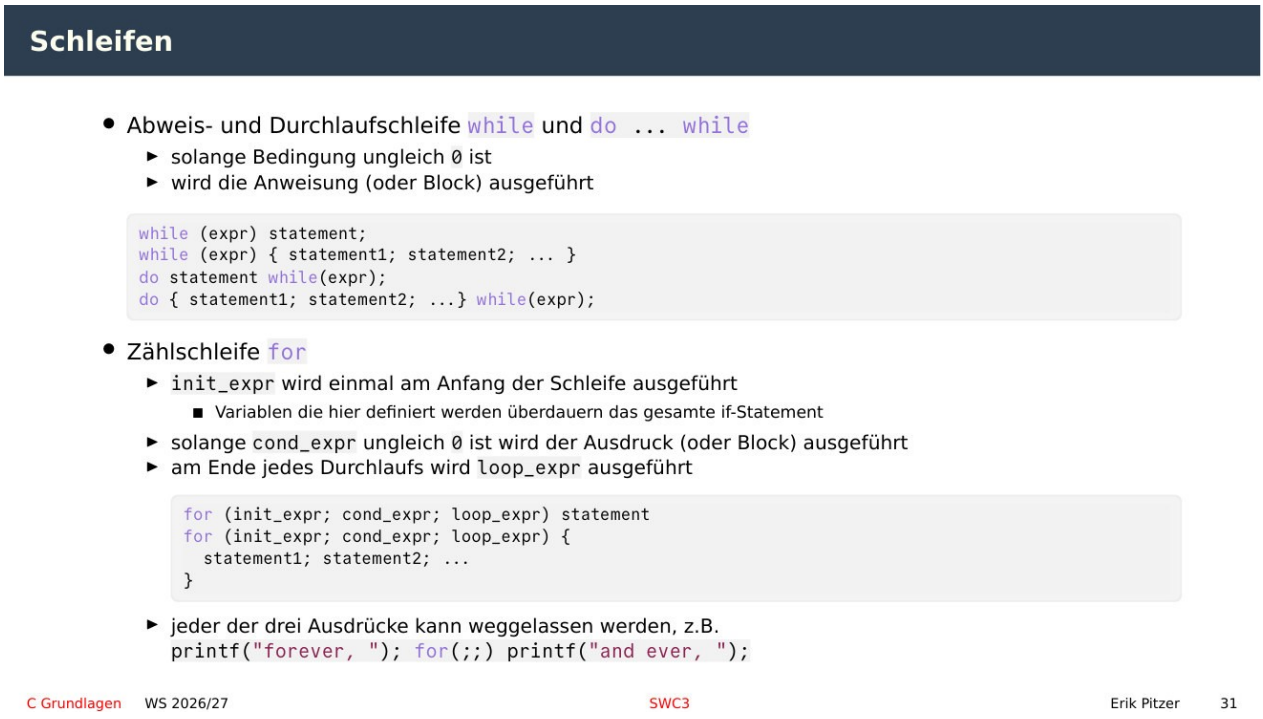
`continue; if (i>7) break; printf(...); }` werden folgerichtig nur die Werte 5, 6 und 7 ausgegeben. **Hinweis zur Folie:** Bei einem `size_t`-Wert ist `%zu`, nicht `%ld`, der passende `printf`-Platzhalter.

`goto label;` springt zu einer Marke in derselben Funktion. Das Beispiel mit den Marken `hello`, `world` und `end` erzeugt `Hello, World!`, ist aber gewöhnlich schwerer zu verfolgen als ein normaler Ablauf. `setjmp` und `longjmp` aus `<setjmp.h>` ermöglichen einen **nichtlokalen Kontrollflusswechsel**: `setjmp` speichert einen Rückkehrkontext, und `longjmp` bewirkt, dass die Ausführung dort mit einem vom ersten Rückgabewert abweichenden Wert fortgesetzt wird. Die Folie illustriert dies durch rekursive Aufrufe bis zur Tiefe 4. **Ergänzende Einordnung:** Nach `longjmp` haben bestimmte geänderte automatische lokale Variablen, die nicht `volatile` sind, indeterminierte Werte; außerdem kann man nicht gefahrlos in eine bereits beendete Funktion springen. Solche Verfahren sind daher selten und komplexe Alternativen für die Ablaufsteuerung.

Vertiefung · Schritt für Schritt

Eine `while`-Schleife prüft ihre Bedingung vor jedem Durchlauf. Eine do-`while`-Schleife führt den Rumpf dagegen mindestens einmal aus, weil sie erst danach prüft. Die for-Schleife bündelt Initialisierung, Test und Iterationsausdruck; `continue` springt bei for vor der nächsten Bedingungsprüfung noch zum Iterationsausdruck, bei `while` unmittelbar zur Bedingungsprüfung. `break` beendet die innerste Schleife oder die umgebende `switch`-Anweisung. Diese Unterschiede sind für die genaue Reihenfolge von Seiteneffekten wesentlich.

`goto` springt zu einem Label innerhalb derselben Funktion; es kann sinnvoll sein, um bei mehreren aufeinanderfolgenden Ressourcenzuteilungen zentral aufzuräumen. `setjmp` und `longjmp` aus `<setjmp.h>` erlauben einen nichtlokalen Kontrollfluss. Nach `longjmp` gelten jedoch besondere Einschränkungen für automatische lokale Variablen, deren Werte seit dem `setjmp`-Aufruf geändert wurden; außerdem werden dazwischen liegende Funktionsaufrufe nicht normal zu Ende geführt. Für Anfänger ist daher gewöhnlicher Rückgabewert-basierter Fehlerfluss meist klarer und sicherer.



Originalfolie 03, PDF-Seite 32: Schleifen

Schleifenabbruch

- mit Hilfe von `break` wird eine Schleife sofort abgebrochen
- mit Hilfe von `continue` wird sofort zur nächsten Iteration gesprungen

```c
for (size_t i = 0; i<10; i++) {
  if (i < 5) continue;
  if (i > 7) break;
  printf("i = %ld\n", i);
}
```

```
i = 5
i = 6
i = 7
```


Originalfolie 03, PDF-Seite 33: Schleifenabbruch

Sprünge mit `goto`

- mit Hilfe von `goto` kann innerhalb einer Funktion gesprungen werden

```c
goto hello;

world: printf("World!");
       goto end;

hello: printf("Hello, ");
       goto world;

end:
```

```
Hello, World!
```

- in der Praxis kaum mehr verwendet bzw. notwendig

Originalfolie 03, PDF-Seite 34: Sprünge mit goto

Sprünge mit `longjmp()`

- Mit Hilfe von `longjmp()` kann zwischen Funktionen gesprungen werden

```c
#include <setjmp.h>
jmp_buf target;

[[noreturn]] void f(size_t depth) {
  printf("recursion %ld\n", depth);
  if (depth > 3) {
    longjmp(target, 1); // (4) jump to saved target
  } //                                        |
  f(depth+1); // (3) infinte recursion        |
} //                                          |
int main() { //                               |
  // (1) mark jump target with setjmp         |
  int result = setjmp(target); // <-----------'
  if (result == 0 ) {
    printf("starting...\n");
    f(0); // (2) call f
  } else {
    printf("result = %d", result);
  }
}
```

```
starting...
recursion 0
recursion 1
recursion 2
recursion 3
recursion 4
result = 1
```

- wird selten direkt verwendet
- erlaubt Implementieren eigener Ablaufsteuerung, z.B.
  - Ausnahmen (*exceptions*)
  - Koroutinen
    - Generatoren
    - kooperatives Multitasking

Originalfolie 03, PDF-Seite 35: Sprünge mit longjmp()

5 C-Grundlagen III: Funktionen, Parameter und Fehlerverhalten

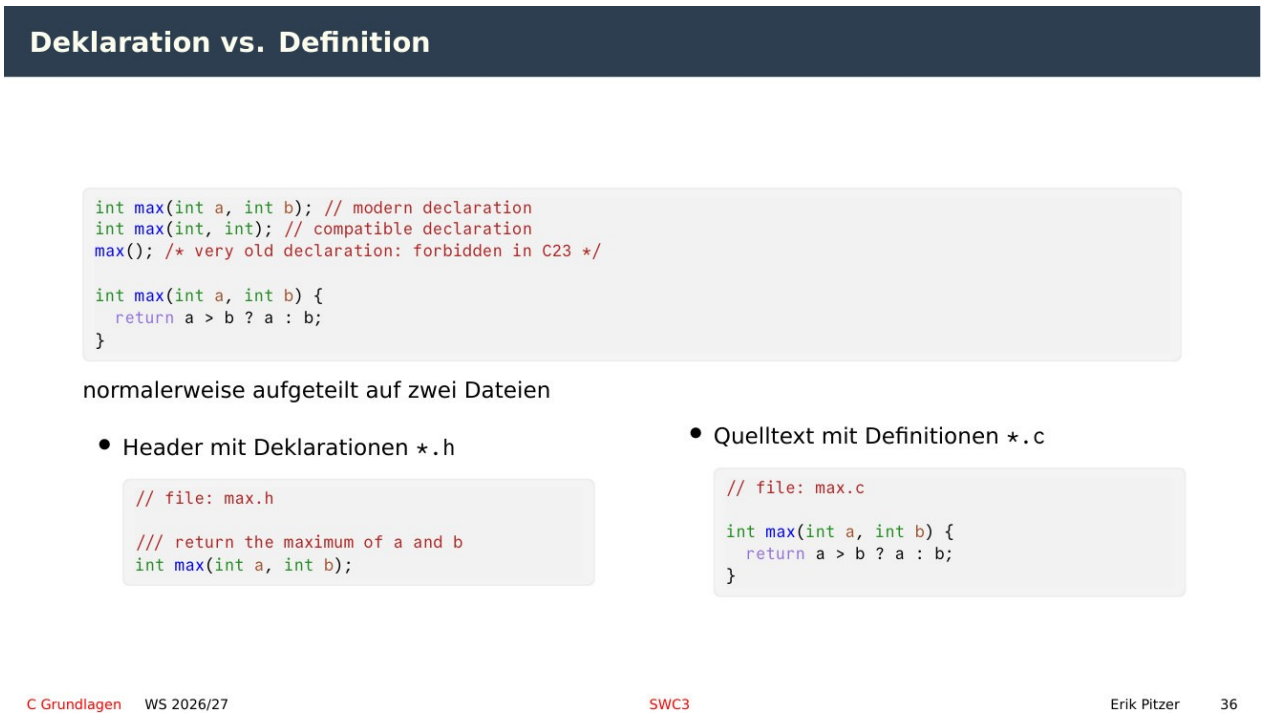
5.1 Deklarationen, Definitionen und Funktionsprototypen

Eine Funktion fasst einen Programmabschnitt mit einem Namen, formalen Parametern und einem möglichen Rückgabewert zusammen. Beispiel: `int max(int a, int b);` ist ein **Funktionsprototyp** und erklärt dem Compiler Typen und Anzahl der Argumente. `int max(int a, int b) { return a > b ? a : b; }` definiert die Funktion. Der Name und die Parametertypen müssen zwischen Deklaration und Definition kompatibel sein. Das Folienbeispiel erklärt zudem die Aufteilung auf `max.h` mit dem öffentlichen Prototyp und `max.c` mit dem Funktionskörper. Der Prototyp darf mehrfach konsistent erscheinen; die externe Definition darf nicht mehrfach miteinander kollidieren.

In alten C-Dialekten wurden Formen ohne aussagekräftige Parameterliste verbreitet verwendet. Der C23-Stand zieht hier engere Grenzen und soll alte nicht-prototypisierte Deklarationen beseitigen. `int main(void)` ist die eindeutige Schreibweise einer Hauptfunktion ohne Argumente, während die folienübliche Kurzform `int main()` im C23-Kontext entsprechend zu verstehen ist. C-Funktionen sind keine Methoden eines Objekts; eine Funktion kann allerdings Zeiger auf Strukturen erhalten und dadurch deren Zustand verändern.

Vertiefung · Schritt für Schritt

Ein Funktionsprototyp stellt eine überprüfbare Signatur bereit: `int maximum(int a, int b);` benennt Rückgabetyp und Parametertypen. Bei einem Aufruf kann der Compiler damit die Anzahl und Typverträglichkeit der Argumente prüfen. `int maximum(int, int);` ist eine äquivalente Deklaration ohne Parameternamen, während eine Definition den Funktionskörper hinzufügt. Die Aussage, dass eine Parameterübergabe „per Referenz“ stattfinde, ist in C nur als Beschreibung der Wirkung von explizit übergebenen Zeigern zulässig: Die Zeigerwerte selbst werden weiterhin kopiert.



Originalfolie 03, PDF-Seite 37: Deklaration vs. Definition

5.2 Call-by-value und Änderungen über Zeiger

C verwendet **ausnahmslos call-by-value** für Funktionsargumente. Wird `int x=0; increment(x);` aufgerufen und der Parameter innerhalb von `increment(int x)` erhöht, verändert die Funktion ihre eigene Parameterkopie, nicht das ursprüngliche `x` im Aufrufer. Erst eine explizite Rückgabe `x = increment(x);` übernimmt den neuen Wert. Für die Änderung eines bestehenden Objekts kann seine Adresse übergeben werden: `void increment(int *x) { ++(*x); }`. Bei `increment(&counter);` ist das Argument der **Wert der Adresse**; durch `*x` wirkt die Änderung auf `counter` im Aufrufer.

Soll eine Funktion den referenzierten Wert lediglich lesen, ist `const int *p` eine sinnvolle Parameterdeklaration. Das teilt dem Aufrufer und dem Compiler mit, dass durch diesen Zugriffspfad keine Änderung erlaubt ist. Ein `const`-Zeigerparameter kann trotzdem null sein, falls seine Schnittstelle das zulässt: Für sich allein garantiert `const` keine Gültigkeit oder Mindestlänge. Arrays, die als Argumente übergeben werden, werden gewöhnlich zu Zeigern auf ihr erstes Element umgewandelt. In `void f(size_t n, int a[static n])` wird deshalb weiterhin ein Zeiger übergeben; `[static n]` drückt eine mindestens erforderliche Anzahl gültiger Elemente aus und kann Compilerwarnungen auslösen, ist aber keine automatisch erzwungene Prüfung.

Parameterübergabe (i)

- Parameterübergabe immer per Value

```c
int increment(int x) {
  x = x + 1; // copy is incremented (2)
  return x;  //   and returned
}

int main() {
  int x = 0;
  increment(x); // x is copied (1)
  SHOW(x); // x in main is unaffected
  x = increment(x); // x is set to return value
  SHOW(x);
}
```

```
x = 0
x = 1
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 37

Originalfolie 03, PDF-Seite 38: Parameterübergabe (i)

Parameterübergabe (ii)

- in altem C-Code daher häufig Übergabe per Pointer
  - wenn Referenzsemantik gewünscht wird

```c
void increment(int *x) {
  // x -points-to-> counter
  ++(*x); // *x == counter
}

int main() {
  int counter = {};
  increment(&counter); // address of counter is copied
  SHOW(counter);
}
```

```
counter = 1
```


Originalfolie 03, PDF-Seite 39: Parameterübergabe (ii)

Parameterübergabe (iii)

- falls Pointer übergeben wird aber keine Änderung gemacht wird → Parameter mit `const` qualifizieren

```c
void print(const int *x) { // const => reference not changed
  SHOW(x);
  SHOW(*x);
}
int main() {
  int counter = {};
  print(&counter); // reference is transferred
}
```

```
x = 0x2ef0020
*x = 0
```

- erlaubt bessere Einschätzung beim Aufrufer
- erlaubt Compiler-Optimierungen

Originalfolie 03, PDF-Seite 40: Parameterübergabe (iii)

Parameterübergabe (iv)

- Arrays zerfallen (*decay*) zu einfachen Zeigern bei der Parameterübergabe
  - C23 ermöglicht Angabe von Mindestgröße
  - erzeugt aber *nur Warnung*

```c
void array_length(size_t n, int array[static n]) {
  SHOW(sizeof(array)); // --> warning sizeof-array-argument)
}

void not_null(int pointer[static 1]) {
  SHOW(sizeof(pointer)); // --> warning sizeof-array-argument
}

int main() {
  int a[] = {1, 2, 3};
  const size_t len = sizeof(a)/sizeof(a[0]);
  array_length(len, a); // --> OK: size matches
  array_length(10, a);  // --> warning stringop-overflow
  not_null(a);       // --> OK: a is not null
  not_null(nullptr); // --> warning nonnull
}
```

```
sizeof(array) = 8
sizeof(array) = 8
sizeof(pointer) = 8
sizeof(pointer) = 8
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 40

Originalfolie 03, PDF-Seite 41: Parameterübergabe (iv)

5.3 Variadische Funktionen und Rückgabewerte

Eine Funktion mit **variabler Argumentzahl** hat einen festen benannten Parameterteil und am Ende `...`, zum Beispiel `double sum(int count, ...);`. Die Verarbeitung erfolgt mit `<stdarg.h>`: `va_list` hält den Traversierungszustand; `va_start(args, count)` startet die Auswertung nach dem letzten festen Parameter, `va_arg(args, double)` liest den nächsten Wert des angegebenen Typs und `va_end(args)` beendet die Verarbeitung. Der Aufruf `sum(2, 1.5, 2.5)` summiert zwei `double`-Argumente zu 4.0. Die Funktion hat keine eingebaute Information über die Typen und Anzahl der variablen Argumente. Der Aufrufer muss den Vertrag daher genau einhalten, sonst entstehen falsche Zugriffe oder undefiniertes Verhalten.

Bei variadischen Aufrufen gelten **default argument promotions**: `float` wird zu `double`, und kleinere Integer-Typen werden über Integer Promotions verarbeitet. Deshalb wird ein Zeichenargument in der gezeigten `showf`-Funktion über `va_arg(args, int)` gelesen, während ein Fließkommaargument mit `va_arg(args, double)` gelesen werden muss. Formatzeichen wie `i`, `d` und `c` können Typen kodieren, aber ein falsches Format ist eine erhebliche Fehlerquelle. Das ist derselbe Grund, warum die Typzuordnung bei `printf` exakt stimmen muss.

Funktionen können zusammengesetzte Typen **direkt als Wert zurückgeben**. `point_t scale(point_t p, double factor)` kann etwa `(point_t){.x = p.x * factor, .y = p.y * factor}` liefern; das Resultat wird einem neuen Objekt im Aufrufer zugewiesen. Eine ältere Variante verändert stattdessen die übergebene Struktur über `point_t *p` und verwendet den `int`-Rückgabewert als Statuscode. Beide Varianten sind zulässig und haben unterschiedliche Schnittstellenverträge. **Wichtig:** Der Zeiger auf eine automatische lokale Variable darf nicht als dauerhaft gültiger Rückgabewert verwendet werden; nach Verlassen der Funktion endet deren Lebensdauer.

Vertiefung · Schritt für Schritt

Die Ellipse … hinter einem festen Parameter signalisiert eine variable Argumentzahl. Eine Funktion wie `sum(int count, ...)` muss die Anzahl der folgenden Argumente durch einen zusätzlichen Vertrag kennen: Hier gibt count sie vor. `va_start` initialisiert die Leseposition, jeder Aufruf `va_arg` entnimmt das nächste Argument mit dem vereinbarten nach Promotion gültigen Typ, und `va_end` beendet den Zugriff. Ein fehlendes Argument oder falscher `va_arg`-Typ kann undefiniertes Verhalten verursachen. Bei `printf` übernimmt stattdessen der Formatstring die Rolle des Typ- und Anzahlvertrags.

```c
#include <stdarg.h>

double sum_double(int n, ...) {
    va_list args;
    va_start(args, n);
    double result = 0.0;
    for (int i = 0; i < n; ++i) {
        result += va_arg(args, double);
    }
    va_end(args);
    return result;
}
```

Variable Parameteranzahl

<stdarg.h> und »`...`« für *variadische* Funktionen

```c
double sum(int count, ...) {
  double sum = {};
  va_list args; va_start(args, count);
  for (int i = 0; i < count; i++) {
    sum += va_arg(args, double);
  }
  va_end(args);
  return sum;
}

int main() {
  SHOW(sum(0));
  SHOW(sum(2, 1.5, 2.5));
  SHOW(sum(3, 1.1, 2.2, 3.3));
}
```

```
sum(0) = 0
sum(2, 1.5, 2.5) = 4
sum(3, 1.1, 2.2, 3.3) = 6.6
```

```c
void showf(const char fmt[static 1], ...) {
  va_list args;
  for (va_start(args, fmt); *fmt != '\0'; fmt++) {
    switch (*fmt) {
      case 'i': SHOW(va_arg(args, int)); break;
      case 'd': SHOW(va_arg(args, double)); break;
      case 'c': SHOW(va_arg(args, int)); break;
      default: printf("unknown format '%c'", *fmt);
    }
  }
  va_end(args);
}
int main() {
  showf("idcx", 1, 2.2, 'c', "unknown");
}
```

```
va_arg(args, int) = 1
va_arg(args, double) = 2.2
va_arg(args, int) = 99
unknown format 'x'
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 41

Originalfolie 03, PDF-Seite 42: Variable Parameteranzahl

Rückgabe von Werten

```c
typedef struct { double x; double y; } point_t;
```

- modernes C: direkter Rückgabewert
  - auch bei komplexerem Typ

```c
point_t scale(point_t p, double length) {
  return (point_t){ // return another copy
    .x = p.x * length,
    .y = p.y * length
  };
}
int main() {
  point_t p1 = {1., 1.};
  point_t p2 = scale(p1, 2); // copy p
  SHOW(p2.x);
  SHOW(p2.y);
}
```

```
p2.x = 2
p2.y = 2
```

- älteres C: Rückgabe oft über Zeiger
  - als Referenzparameter
  - Rückgabewert für Fehlercode

```c
int scale(point_t* p_ref, double length) {
     p_ref->x *= length; // modify p in main
     p_ref->y *= length; //   via reference
     return 2; // OK
}
int main() {
  point_t p = {1., 1.};
  if (scale(&p, 2)) { // <-- pass address with `&'
    SHOW(p.x);
    SHOW(p.y);
  }
}
```

```
p.x = 2
p.y = 2
```

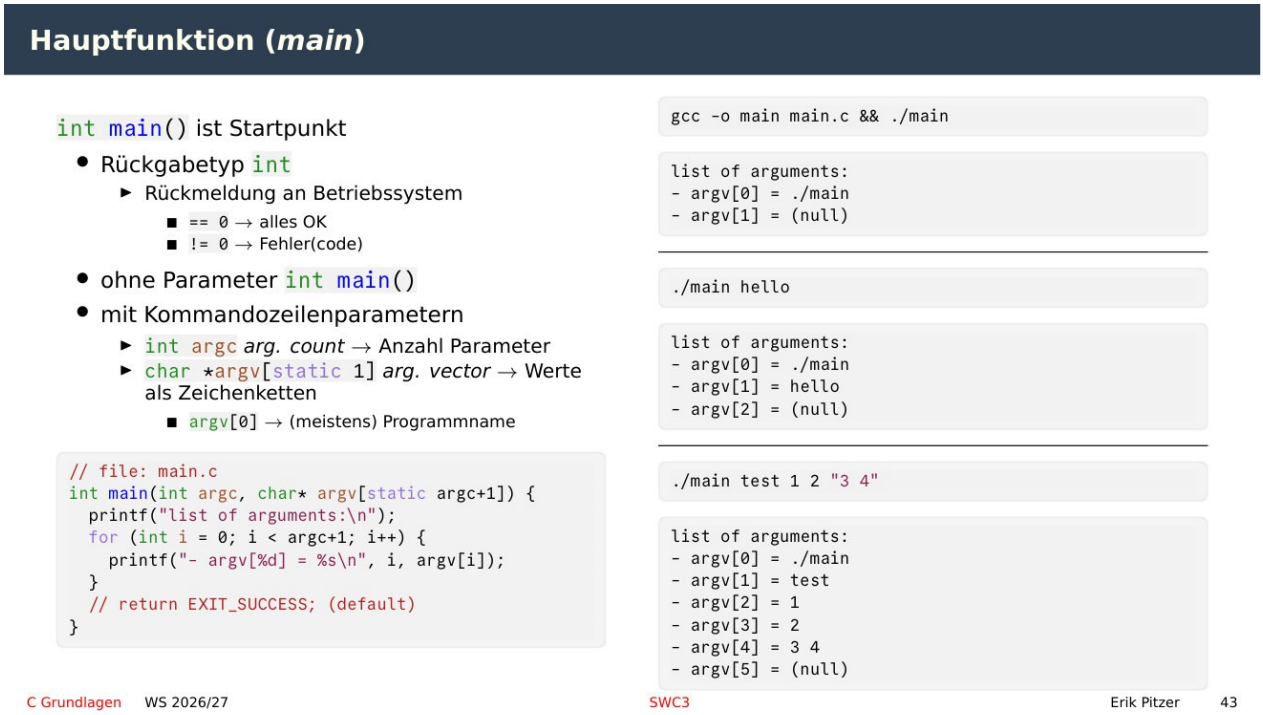
C Grundlagen WS 2026/27 SWC3 Erik Pitzer 42

Originalfolie 03, PDF-Seite 43: Rückgabe von Werten

5.4 Die Hauptfunktion `main` und Programmargumente

Der Startpunkt eines typischen C-Programms ist `main`. Ein Rückgabewert null steht für erfolgreiche Beendigung; von null verschiedene Statuswerte kennzeichnen üblicherweise einen Fehler, wobei die genaue Interpretation dem Betriebssystem beziehungsweise der Umgebung folgt. `EXIT_SUCCESS` und `EXIT_FAILURE` aus `<stdlib.h>` stellen entsprechende portable Konstanten bereit. Eine gebräuchliche Argumentform ist `int main(int argc, char *argv[])`. `argc` zählt die übergebenen Argumente einschließlich `argv[0]`, das normalerweise den Programmaufruf bezeichnet. `argv[1]` bis `argv[argc-1]` sind die weiteren Argumente, während `argv[argc]` ein Nullzeiger ist.

Beim Beispiel `./main test 1 2 "3 4"` ist `argv[1]` die Zeichenkette `test`, `argv[2]` ist `1`, `argv[3]` ist `2` und `argv[4]` enthält `3 4` als **ein** Argument, weil die Anführungszeichen durch die Shell zur Gruppierung verwendet wurden. Programmargumente liegen zunächst als Zeichenketten vor; selbst wenn dort Ziffern stehen, entsteht nicht automatisch ein C-Integerwert. Für numerische Eingaben braucht man eine geeignete Konvertierungsfunktion und Fehlerprüfung, etwa mit `strtol`.



Originalfolie 03, PDF-Seite 44: Hauptfunktion (main)

5.5 `printf`: Formatzeichen, Breite und Genauigkeit

Die Standardbibliotheksfunktion `printf` ist in `<stdio.h>` deklariert und erwartet als ersten Parameter eine **Formatzeichenkette**. Darin stehen wörtliche Zeichen und Platzhalter, die mit `%` beginnen. Ein Platzhalter hat vereinfacht die Form `%[flags][breite][.genauigkeit][länge]typ`. Wesentliche Umwandlungen sind `%%` für ein Prozentzeichen, `%c` für ein Zeichen, `%s` für eine nullterminierte Zeichenkette, `%d` beziehungsweise `%i` für vorzeichenbehaftete Dezimalwerte, `%u` für vorzeichenlose, `%x`/`%o` für hexadezimale/oktale Darstellungen, `%f`, `%e`, `%g` für Fließkommawerte und `%p` für einen Zeiger. Beim `%p`-Konversionsspecifier wird ein Wert vom Typ `void *` erwartet; `printf("%p", (void *)p)` ist daher das passende allgemeine Objektzeigerbeispiel.

Die Flags `-`, `+` und `0` bewirken unter geeigneten Bedingungen Linksbündigkeit, explizite Vorzeichen und führende Nullen. Eine Breite wie `%5.2f` bedeutet bei `f`: Mindestbreite fünf Zeichen, zwei Stellen nach dem Dezimalpunkt; der Wert π wird als `3.14` dargestellt. `%e` benutzt Exponentialschreibweise, `%g` wählt je nach Magnitude eine kompakte Darstellung und interpretiert die Genauigkeit überwiegend als Anzahl signifikanter Ziffern. Die Kombination `%+010.5g` fügt z. B. Vorzeichen, Mindestbreite, Nullen und eine signifikante Genauigkeit zusammen. Längenmodifikatoren sind für korrektes Ausgeben unverzichtbar: `%hd` für `short`-Werte nach den erforderlichen Promotions, `%ld` für `long`, `%lld` für `long long`, `%zu` für `size_t`, `%Lf` für `long double`.

Eine Breite und Genauigkeit kann zur Laufzeit über `*` angegeben werden: `printf("%*.*f", width, prec, value)` erwartet in genau dieser Reihenfolge `int width`, `int prec` und ein zum Platzhalter passendes Fließkommaargument. Bei `width=10` und `prec=4` wird beispielsweise die Zahl `12345.12345` gerundet dargestellt; falls die Darstellung schon mindestens zehn Zeichen benötigt, findet keine Abschneidung statt. **Die Formatzeichenkette ist ein Typvertrag.** Die Folie demonstriert absichtlich `printf("%+d", "hello")` als Falschangabe. Das ist nicht bloß eine unschöne Ausgabe, sondern kann **undefiniertes Verhalten** auslösen, weil tatsächlich ein Zeiger übergeben wird, obwohl `int` erwartet wird. Auch bei variadischen Funktionen ist der Compiler ohne passende Prüfungen nicht in der Lage, einen solchen Aufruf automatisch sicher zu machen.

Vertiefung · Schritt für Schritt

Formatangaben sind nicht nur Dekoration: Sie beschreiben, wie `printf` die nachfolgenden, nicht eigens typgeprüften Argumentwerte interpretiert. Die Variable vom Typ `size_t` wird mit %zu ausgegeben; ein `long` mit %ld; ein `double`

mit `%f`, `%e` oder `%g`. Bei `%f` meint .2 zwei Nachkommastellen, bei `%g` dagegen zwei signifikante Stellen. `%5.2f` definiert eine Mindestbreite, keine maximale Länge; nötigenfalls wird das Feld breiter. Für `%s` muss die angegebene Adresse auf eine gültige nullterminierte Zeichenfolge zeigen und bei `%p` ist ein geeignet zum `void *` konvertierter Objektzeiger zu verwenden.

```c
#include <stdio.h>
#include <stddef.h>

int main(void) {
    size_t n = 12;
    double x = 3.14159;
    printf("n=%zu; x=%8.3f; g=%.4g\n", n, x, x);
    // n=12; x=   3.142; g=3.142
    return 0;
}
```

Ausgabefunktion `printf` (i)

```c
#include <stdio.h>

int printf(const char *restrict format, ...)
```

- formatierte Ausgabe
- variable Anzahl von Parametern
- Format-Zeichenkette mit eingebetteten Formatierungshinweisen und Escape-Zeichen
  - beginnen mit »%«
  - optionale Flags, Optionen und Längenangabe
  - abgeschlossen durch Umwandlungstyp

```c
printf("char: %c, string: %s, int: %i, hex: %x, pointer: %p",
       'c', "test", 123, 0x123, printf);
```

```
char: c, string: test, int: 123, hex: 123, pointer: 0x7fadd9056a64
```

cppreference – printf

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 44

Originalfolie 03, PDF-Seite 45: Ausgabefunktion printf (i)

Ausgabefunktion `printf` (ii)

- wichtigste Typumwandlungen und Optionen

Umwandlungstyp	Formatierung
%%	Prozentzeichen
%c	char
%s	Zeichenkette
%d / %i	dezimal int
%o %x	oktal hex int
%u	unsigned int
%f, %e, %g	double
%p	Zeiger *void

Option	Wirkung
-	Ausrichtung links
+	Vorzeichen immer
0	führenden Nullen

```
printf("%+d\n", 123);
printf("%+d", "hello"); // <- XXX: wrong type
```

```
+123
+1266475104
```


Originalfolie 03, PDF-Seite 46: Ausgabefunktion printf (ii)

Ausgabefunktion `printf` (iii)

- Längenangabe
  - minimale Länge
  - optionale Genauigkeit (».p«)

```
double pi = 2*acos(0);
printf("%f\n", pi);
printf("%5f\n", pi);
printf("%5.2f\n", pi);
printf("%e\n", pi);
printf("%+010.5g\n", pi);
```

```
3.141593
3.141593
 3.14
3.141593e+00
+0003.1416
```

- Typ-Modifikatoren, z.B.
  - h → half
  - l → long
  - L → long double
  - z → size_t

```
short s = 123;
long l = 123;
size_t n = 123;
printf("short: %hi, long: %li, size_t: %zu",
       s, l, n);
```

```
short: 123, long: 123, size_t: 123
```


Originalfolie 03, PDF-Seite 47: Ausgabefunktion printf (iii)

Ausgabefunktion `printf` (iv)

- Minimale Breite und Genauigkeit kann auch mit »`*`« angegeben werden
  - dann erwartet `printf()` jeweils einen zusätzlichen Parameter

```c
for (int width = 10; width < 15; width += 4) {
    for (int prec = 0; prec < width; prec += 4) {
        printf("width %2d, prec %2d   ===>  ", width, prec);
        printf("'%*.*f'\n", width, prec, 12345.12345);
        //        ^ ^         |     |
        //        | '---------+-----'
        //        '-----------'
    }
}
```

```
width 10, prec  0   ===>  '     12345'
width 10, prec  4   ===>  '12345.1234'
width 10, prec  8   ===>  '12345.12345000'
width 14, prec  0   ===>  '         12345'
width 14, prec  4   ===>  '    12345.1234'
width 14, prec  8   ===>  '12345.12345000'
width 14, prec 12   ===>  '12345.123449999999'
```

C Grundlagen WS 2026/27 SWC3 Erik Pitzer 47

Originalfolie 03, PDF-Seite 48: Ausgabefunktion printf (iv)

5.6 Funktionszeiger und Callback-Funktionen

Ein **Funktionszeiger** erlaubt, eine Funktion als Wert zu speichern und später über diesen Wert aufzurufen. `typedef int (*fp_t)(int);` definiert `fp_t` als Zeiger auf eine Funktion, die ein `int` übernimmt und ein `int` zurückgibt. `fp_t p = &twice;` speichert die Funktion `twice`; `p(2)` führt sie aus und liefert wie `twice(2)` den Wert 4. Der Funktionsbezeichner wird bei vielen Ausdrücken automatisch in einen passenden Funktionszeiger umgewandelt. Im Gegensatz dazu ist `&p` die Adresse des Variablenobjekts, das den Funktionszeiger enthält.

Der wichtige praktische Anwendungsfall ist ein **Callback**: Eine Bibliotheksfunktion erhält einen Funktionszeiger und ruft die Funktion bei Bedarf auf. `qsort` aus `<stdlib.h>` sortiert ein Array und erwartet eine Vergleichsfunktion mit dem Typ `int (*compar)(const void *, const void *)`. Die Folien zeigen `point_t`-Objekte, die einmal nach `x` und einmal nach `y` verglichen und entsprechend sortiert werden. Ein negativer Vergleichswert bedeutet „erstes Objekt davor“, null „gleichwertig“ und ein positiver „danach“. **Ergänzende Einordnung:** Die einfache Rückgabe `a->x - b->x` kann bei größeren ganzzahligen Werten selbst überlaufen; der Ausdruck `(a->x > b->x) - (a->x < b->x)` vermeidet diese Gefahr. Außerdem ist `qsort` nicht stabil, sodass die relative Reihenfolge bei gleichem Schlüssel nicht garantiert wird.

Vertiefung · Schritt für Schritt

Eine Deklaration wie `int` (*callback)(`int`) bedeutet: callback ist ein Zeiger auf eine Funktion mit `int`-Parameter und `int`-Ergebnis. Die Klammern sind entscheidend, denn `int *callback(int)` würde eine Funktion deklarieren, die einen `int`-Zeiger zurückgibt. Typaliasnamen über `typedef` machen solche Signaturen lesbarer. Im `qsort`-Beispiel muss die Vergleichsfunktion zwei `const void *`-Argumente entgegennehmen, passend in Zeiger auf den tatsächlichen Elementtyp umwandeln und entsprechend negativ, null oder positiv zurückgeben. Sie darf bei ihrem Vergleich weder auf ungültige Speicherbereiche zugreifen noch durch einfache Subtraktion unbeabsichtigt einen Integerüberlauf erzeugen.

Funktionszeiger

- Funktionsbezeichner sind Zeiger auf Beginn von Maschinencode

```c
int twice(int a) {
    return a * 2;
}

typedef int (*fp_t)(int);
fp_t also_twice = &twice;

int main() {
    SHOW(twice);
    SHOW(&twice);
    SHOW(also_twice);
    SHOW(&also_twice);
    SHOW(twice(2));
    SHOW(also_twice(2));
}
```

```
twice = 0x86ec8199
&twice = 0x86ec8199
also_twice = 0x86ec8199
&also_twice = 0x86ecb0e0
twice(2) = 4
also_twice(2) = 4
```

https://godbolt.org/z/6h5dahe96

```c
typedef struct { short x; short y; } point_t;
point_t* as_point(void *p) { return (point_t*)p; }

int cmp_x(const void* a, const void* b) {
    return as_point(a)->x - as_point(b)->x;
}
int cmp_y(const void* a, const void* b) {
    return as_point(a)->y - as_point(b)->y;
}
int main() {
    point_t points[] = {{0, 10}, {1, 20}, {2, 30}, {3, 10}};
    constexpr size_t N = sizeof(points)/sizeof(point_t);

    qsort(points, N, sizeof(point_t), cmp_x); // < compare by x
    SHOW_POINTS_N(points, N);

    qsort(points, N, sizeof(point_t), cmp_y); // < compare by y
    SHOW_POINTS_N(points, N);
}
```

```
points = {(0,10), (1,20), (2,30), (3,10)}
points = {(0,10), (3,10), (1,20), (2,30)}
```

https://en.cppreference.com/c/algorithm/qsort

Originalfolie 03, PDF-Seite 49: Funktionszeiger

5.7 Undefiniertes, nicht spezifiziertes und implementationsdefiniertes Verhalten

Undefiniertes Verhalten (undefined behavior, UB) bedeutet, dass der C-Standard für einen bestimmten Programmsachverhalt keinerlei Anforderungen an das Ergebnis stellt. Typische Ursachen sind Zugriff außerhalb eines Arrays, Dereferenzierung ungültiger Zeiger, Verwendung bestimmter nicht initialisierter Werte, vorzeichenbehafteter Integerüberlauf und die Übergabe unpassender Typen an `printf`. Schon das Auftreten von UB entzieht weiteren Schlussfolgerungen über den Programmlauf die Grundlage. Compiler optimieren oft unter der Annahme, dass korrektes Programmverhalten kein UB erzeugt; deshalb kann „es funktioniert bei meinem Test“ niemals als Sicherheitsbeweis dienen.

Das muss von **nicht spezifiziertem Verhalten (unspecified behavior)** unterschieden werden: Der Standard lässt mehrere erlaubte Resultate zu, etwa bei der Reihenfolge zweier unabhängiger Funktionsaufrufe. **Implementationsdefiniertes Verhalten (implementation-defined behavior)** bedeutet, dass die Implementierung eine bestimmte Wahl treffen und dokumentieren muss. Die Folie nennt zahlreiche Fundstellen in der C-Norm und erläutert, dass UB unter anderem bestimmte effiziente Übersetzungen ermöglicht, weil beispielsweise keine generelle Bereichsprüfung vorgeschrieben ist. AddressSanitizer, UndefinedBehaviorSanitizer und Compilerwarnungen können Fehler entdecken, sie können aber keine allgemeine Korrektheit beweisen.

Undefiniertes Verhalten (*undefined behavior*)

- Warum gibt es im C-Standard undefiniertes Verhalten?
  - 64 Vorkommen von *“undefined behavior”*
  - § J.1 Unspecified Behavior: 63 Unterpunkte
  - § J.2 Undefined Behavior: 221 Unterpunkte
- erlaubt Compiler in bestimmten Situationen *irgendetwas*™ zu machen
  - ermöglicht damit bestimmte Optimierungen (in anderen Situationen), z.B.
    - keine Überprüfung ob Index innerhalb von Array
    - keine Überprüfung ob Zeiger gültig
    - keine Überprüfung auf Integer-Overflow
- als Programmierer immer vermeiden
  - z.B. mit Hilfe von Undefined Behavior Sanitizer

As-if rule - cppreference.com
Undefined behavior - cppreference.com
n3220

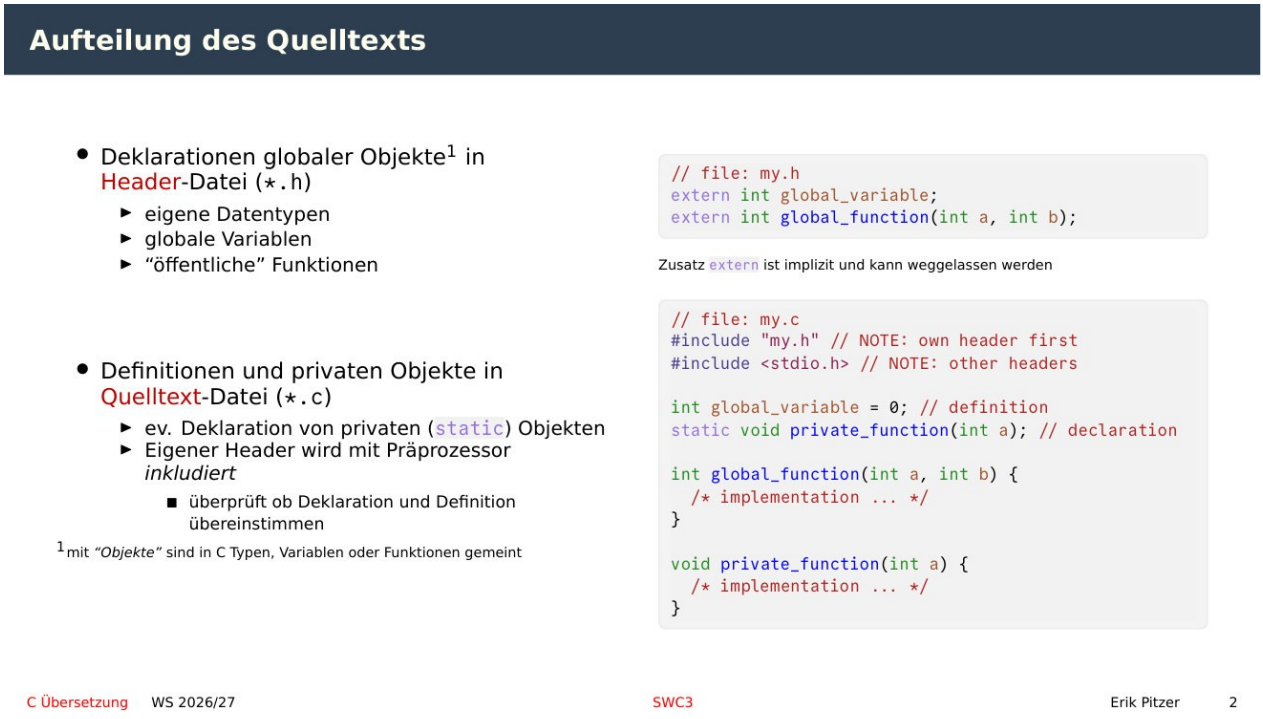
Originalfolie 03, PDF-Seite 50: Undefiniertes Verhalten (undefined behavior)

6 Übersetzungsprozess: Präprozessor, Compiler und Linker

6.1 Header und Implementierungseinheiten

Ein größeres C-Programm besteht gewöhnlich aus mehreren **Translation Units**, die sich aus jeweils einer Quelltextdatei und den durch Präprozessoranweisungen eingebundenen Inhalten ergeben. Eine Headerdatei enthält öffentliche Deklarationen von Typen, Variablen und Funktionen. Die zugehörige `.c`-Datei enthält deren Definitionen sowie gegebenenfalls **dateilokale** Hilfsfunktionen und Variablen mit `static`. In `my.h` kann daher `extern int global_variable;` und `int global_function(int a,int b);` stehen; in `my.c` steht dann `#include "my.h"` und die Definition `int global_variable=0;` neben dem eigentlichen Funktionskörper.

Wird der eigene Header im Implementierungscode ebenfalls eingebunden, prüft der Compiler, ob die Definition mit ihrer öffentlichen Deklaration übereinstimmt. Die Originalfolie enthält außerdem einen privaten Funktionsprototyp mit `static`; wenn die Implementierung intern bleiben soll, muss die Definition dazu passende interne Linkage besitzen. Ein bloßes `void private_function(int a) { ... }` ohne zuvor wirksames `static` kann in manchen Konstellationen durch eine vorherige `static`-Deklaration interne Linkage erben; aus Gründen der Verständlichkeit ist eine übereinstimmende Kennzeichnung sinnvoll.



Originalfolie 04, PDF-Seite 3: Aufteilung des Quelltexts

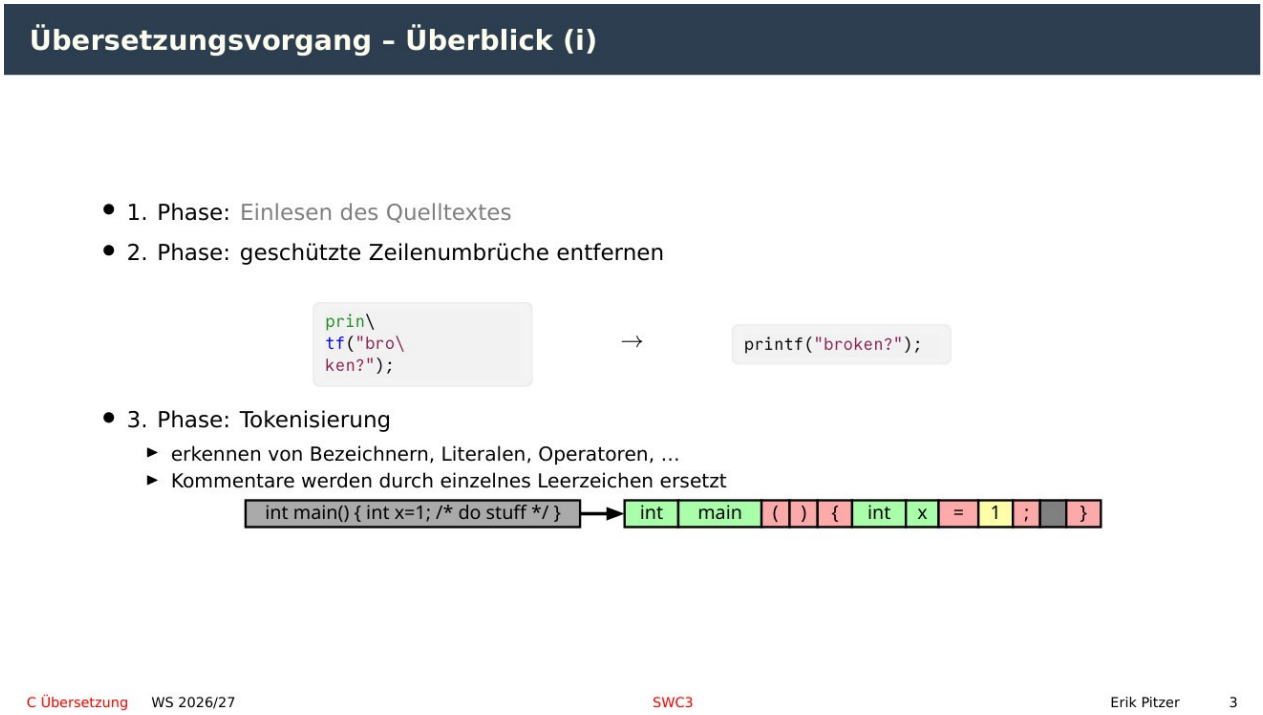
6.2 Acht logische Übersetzungsphasen

Die Norm beschreibt die Übersetzung in Phasen, die ein Compiler nicht zwingend als acht separate Programme implementieren muss. Zuerst wird der Quelltext eingelesen und einer definierten Zeichenverarbeitung unterzogen. **Backslash-Newline** verbindet physische Quelltextzeilen, sodass die Folie aus `prin\` am Zeilenende und `tf(...)` wieder `printf(...)` entstehen lässt. Danach werden aus der Zeichenfolge Vorverarbeitungstoken erkannt und Kommentare durch Leerzeichen ersetzt. Anschließend verarbeitet der Präprozessor Direktiven und Makros, die Quell- und Ausführungszeichensätze werden nach Regeln der Implementierung berücksichtigt, benachbarte Stringliterale wie `"This" "is" "a" "single" "string"` werden zusammengefügt, und die eigentliche Übersetzung erzeugt eine maschinennahe **Objektdatei**. Schließlich verbindet der **Linker** die Objektdateien und nötigen Bibliotheken zu einer ausführbaren Datei oder Bibliothek.

Das Schaubild der Folie unterscheidet besonders die zwei Arten von Bibliotheksintegration: `#include <stdio.h>` fügt **Deklarationen** in die Übersetzungseinheit ein, dagegen verbindet die Linkeroption `-lc` die Implementierung der C-Bibliothek. Nach schematischer Vorverarbeitung entsteht aus `main.c` ein Zwischenergebnis `main.i`, nach Kompilierung eine Objektdatei `main.o`, nach Linken ein ausführbares Image. Auf Linux wird `libc` meist standardmäßig eingebunden, sodass `-lc` normalerweise nicht manuell nötig ist.

Vertiefung · Schritt für Schritt

Die acht Übersetzungsphasen sind ein logisches Normmodell, keine Pflicht, acht getrennte Programme auszuführen. Vor dem eigentlichen C-Parser verändern bereits Zeilenverkettung, Tokenisierung und Präprozessor den sichtbaren Text. Eine über einen Backslash vor dem Zeilenende fortgesetzte Makrodefinition bleibt logisch eine Anweisung; Kommentare werden nicht erst nach dem vollständigen C-Parsing erkannt. Benachbarte Zeichenkettenliterale werden nach der Makroverarbeitung zusammengesetzt, sodass `printf`("abc" "def") dieselbe Zeichenfolge wie `printf`("abcdef") erhält. Erst danach werden der vollständige Quelltext interpretiert, Code erzeugt und externe Referenzen aufgelöst.



Originalfolie 04, PDF-Seite 4: Übersetzungsvorgang – Überblick (i)

Übersetzungsvorgang – Überblick (ii)

- 4. Phase: Präprozessor führt Textersetzung durch
- 5. Phase: Umwandeln von *source* in *execution character set*
- 6. Phase: Zeichenketten verknüpfen

```
printf("This" "is" "a" "single" "string"); // -> Thisisasinglestring
```

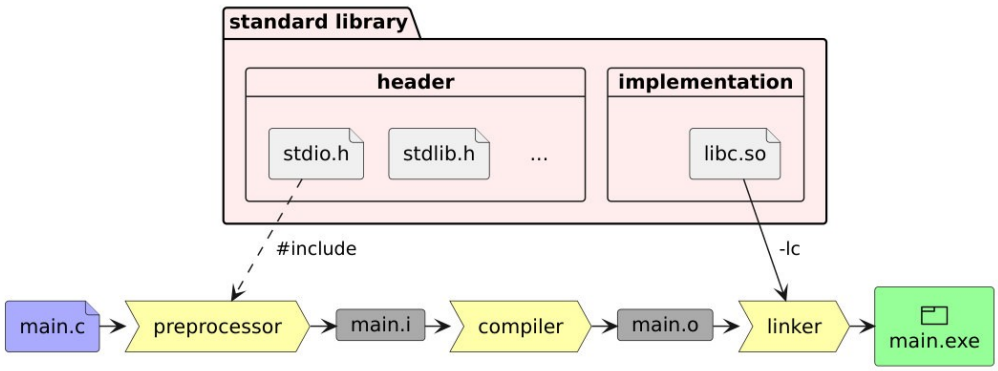
- 7. Phase: Übersetzung in Maschinensprache (sogenannter »*Objektcode*«)
  - tatsächliche Übersetzung (*compilation*)
- 8. Phase: Binden (*linking*)
  - Objektcode von mehreren Dateien wird verknüpft
  - Objektcode von Bibliotheken wird verknüpft
  - Ergebnis ist Speicherabbild (*image*)
    - ausführbare Datei (*executable*)
    - Bibliothek (*library*)

Phases of translation - cppreference.com

Originalfolie 04, PDF-Seite 5: Übersetzungsvorgang – Überblick (ii)

Übersetzungsvorgang Schematisch

- Einbinden von Headern mit `#include` Anweisung
  - werden zuerst hinein "kopiert" und dann selbst vorverarbeitet
- Binden von Bibliotheken mit Compiler-Parameter (`-l`)
  - enthalten Implementierung von vorhandenen Funktionen
  - z.B. `-lc` bindet Standardbibliothek (`libc.so`) ein[1]



[1] Binden mit der Standardbibliothek mit `-lc` passiert meist automatisch

Originalfolie 04, PDF-Seite 6: Übersetzungsvorgang Schematisch

6.3 Präprozessor: Textersetzung, Makros und Konditionen

Der Präprozessor arbeitet vor der eigentlichen semantischen C-Übersetzung mit **Makroersetzung** und Direktiven. `#include <stdio.h>` sucht in den für das System eingerichteten Include-Pfaden, `#include "my.h"` berücksichtigt üblicherweise zusätzlich den Pfad zur eigenen Datei. `#define PI 3.1415` definiert eine Makroersetzung ohne abschließendes Semikolon; `#undef PI` entfernt die Definition. Bedingte Bereiche wie `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else` und `#endif` bestimmen, welche Quelltextteile aktiv sind. `#pragma` stellt implementierungsspezifische Anweisungen bereit; `#line` kann Quellpositionen etwa für generierten Code beeinflussen.

Funktionsartige Makros besitzen formale Parameter: `#define ARRAY_LEN(X) (sizeof(X)/sizeof((X)[0]))` kann bei einem echten Array die Elementzahl liefern. Auch hier gilt: Wird ein Zeiger statt eines Arrays übergeben, ist das Ergebnis nicht die Elementzahl. Das Makro `#define REPEAT(X) (X); (X); (X); (X); (X);` ruft einen übergebenen Ausdruck fünfmal auf; `REPEAT(x++)` verändert `x` fünfmal. Gerade daran zeigt sich die Gefahr der Textersetzung: Das Argument kann mehrfach ausgewertet werden und die expandierten Semikola können die Struktur eines umgebenden `if` zerstören. Sorgfältiges Einklammern und die übliche `do { ... } while (0)`-Form für mehrteilige Statement-Makros verbessern die Sicherheit, ersetzen aber keine Typprüfung.

Mit dem Operator `#` wird ein Makroargument zu einem Stringliteral gemacht (**Stringification**). `#define SHOW_INT(X) printf("%s -> %d\n", #X, (X))` gibt zum Beispiel bei `SHOW_INT(five * 10)` zuerst die Zeichenfolge `five * 10` und danach das berechnete Integerergebnis 50 aus. Das Pascal-ähnliche Folienbeispiel definiert `PROCEDURE`, `FOR`, `TO`, `DO`, `BEGIN`, `END` als Textmakros und zeigt mit `gcc -E`, wie daraus C-Quelltext wird. Dies demonstriert die Mächtigkeit, aber auch die fehlende syntaktische und typbezogene Kontrolle von Makros.

Vertiefung · Schritt für Schritt

Eine Makrodefinition ist keine normale Funktion. Ihre Argumente werden als Token eingesetzt und können beim Ersetzen mehrfach vorkommen. Das gefährliche Makro `#define SQUARE(x)` x * x ergibt bei `SQUARE(1+2)` den Wert 5 statt 9. Mit `#define SQUARE(x)` ((x)*(x)) ist die Gruppierung zwar korrigiert; `SQUARE(i++)` bleibt aber problematisch, weil i zweimal verändert würde. Eine inline-Funktion hat Typprüfung und wertet übergebene Argumente für den Funktionsaufruf nach dessen Regeln aus. Makros sollen daher sparsam und bewusst eingesetzt werden.

```
#define SQUARE_UNSAFE(x) x * x
#define SQUARE_GROUPED(x) ((x) * (x))

int a = SQUARE_UNSAFE(1 + 2);  // wird zu 1 + 2 * 1 + 2 = 5
int b = SQUARE_GROUPED(1 + 2); // wird zu 9
// SQUARE_GROUPED(i++) bleibt unsicher: zweimaliges i++
```

Präprozessor (i)

- einfache (stupide) Textersetzung

wichtigste Präprozessor Direktiven

- `#include`
  - “Kopieren” des Inhalts von anderen Dateien, Einbinden von Deklarationen
  - spitze Klammern für externe Header, z.B. `<stdio.h>`
  - doppelte Hochkommata für eigene Header, z.B. `"utils/myutils.h"`
    - sucht nach Header-Datei auch im aktuellen Verzeichnis
- `#define` und `#undef`
  - Definition von Makros (auch mit Parametern)
- `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif` und `#endif` für bedingte Übersetzung
  - ermöglicht einzelne Abschnitt nur unter bestimmten Voraussetzungen zu übersetzen
- `#pragma` für Compiler-Interaktion
  - z.B. deaktivieren von Warnungen für bestimmte Zeilen
- `#line` um Dateiname und Zeile anzugeben, z.B. für generierten Quelltext

C Übersetzung WS 2026/27 SWC3 Erik Pitzer 6

Originalfolie 04, PDF-Seite 7: Präprozessor (i)

Präprozessor: Konstanten und Makros (i)

- Achtung: kein Strichpunkt am Ende
- Konstanten

```c
#define PI 3.1415
#define NULL ((void*)0)

SHOW(PI);
SHOW(NULL);
```

```
PI = 3.1415
NULL = 0x0
```


Originalfolie 04, PDF-Seite 8: Präprozessor: Konstanten und Makros (i)

Präprozessor: Konstanten und Makros (ii)

- Makros mit Parametern

```c
#define ARRAY_LEN(X) (sizeof(X) / sizeof((X)[0]))

int a[] = {1, 2, 3, 4};
int b[] = {1, 2, 3};

SHOW(ARRAY_LEN(a));
SHOW(ARRAY_LEN(b));
```

```
ARRAY_LEN(a) = 4
ARRAY_LEN(b) = 3
```

```c
#define REPEAT(X) (X); (X); (X); (X); (X);

REPEAT(printf("!"));
int x = 0;
REPEAT(x++);
SHOW(x);
```

```
!!!!!x = 5
```


Originalfolie 04, PDF-Seite 9: Präprozessor: Konstanten und Makros (ii)

Präprozessor: Konstanten und Makros (iii)

- mit Hilfe von »#« + Parametername, Zugriff auf Parameter als Text möglich

```c
#define SHOW_INT(X) printf("%s -> %d\n", #X, (X))

SHOW_INT(5);

int five = 5;
SHOW_INT(five);

SHOW_INT(five * 10);

SHOW_INT(3 * 2 + 10 - 21 / 123);
```

```
5 -> 5
five -> 5
five * 10 -> 50
3 * 2 + 10 - 21 / 123 -> 16
```


Originalfolie 04, PDF-Seite 10: Präprozessor: Konstanten und Makros (iii)

Präprozessor: PASCAL → C

PASCAL-Code in gültigen C-Code umwandeln

```c
// file: preprocessor-pascal.c
#define PROCEDURE int
#define FOR for (size_t
#define TO ; i <
#define DO ; i++)
#define BEGIN {
#define END }
#define PRINT(X) printf("%zu", (X));

#include <stdio.h>

PROCEDURE main() BEGIN
  FOR i = 0 TO 10 DO BEGIN
    PRINT(i)
  END
END
```

```
0123456789
```

Compiler-Switch -E führt nur Präprozessor aus

```
gcc -E preprocessor-pascal.c
```

```c
# 0 "preprocessor-pascal.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3
# 0 "<command-line>" 2
# 1 "preprocessor-pascal.c"
# 10 "preprocessor-pascal.c"
# 1 "/usr/include/stdio.h" 1 3
// [... contents of stdio.h ... ]
extern int printf (const char *__restrict __format, ...);
// [...]
int main() {
  for (size_t i = 0 ; i < 10 ; i++) {
    printf("%zu", (i));
  }
}
```


Originalfolie 04, PDF-Seite 11: Präprozessor: PASCAL →C

6.4 Bedingte Übersetzung, eingebaute Makros und Include-Guards

Mit `#ifdef`, `#elif defined(...)` und `#else` können unterschiedliche Betriebssystemzweige ausgewählt werden. Die Folie nutzt beispielhaft `__APPLE__`, `__linux__`, `__unix__` sowie ein Windows-Makro. Die konkreten Plattformmakros sind **Compiler- beziehungsweise Systemkonventionen**, nicht durchgängig Teil des ISO-C-Standards. `__DATE__` und `__TIME__` geben Übersetzungsdatum und -uhrzeit als Zeichenketten an, `__STDC_VERSION__` kann zur Abfrage des C-Standardstands dienen. Die konkret gezeigte Ausgabe `202311` ist eine normbezogene Kennzahl aus der Folie und von der eingestellten Implementierung abhängig.

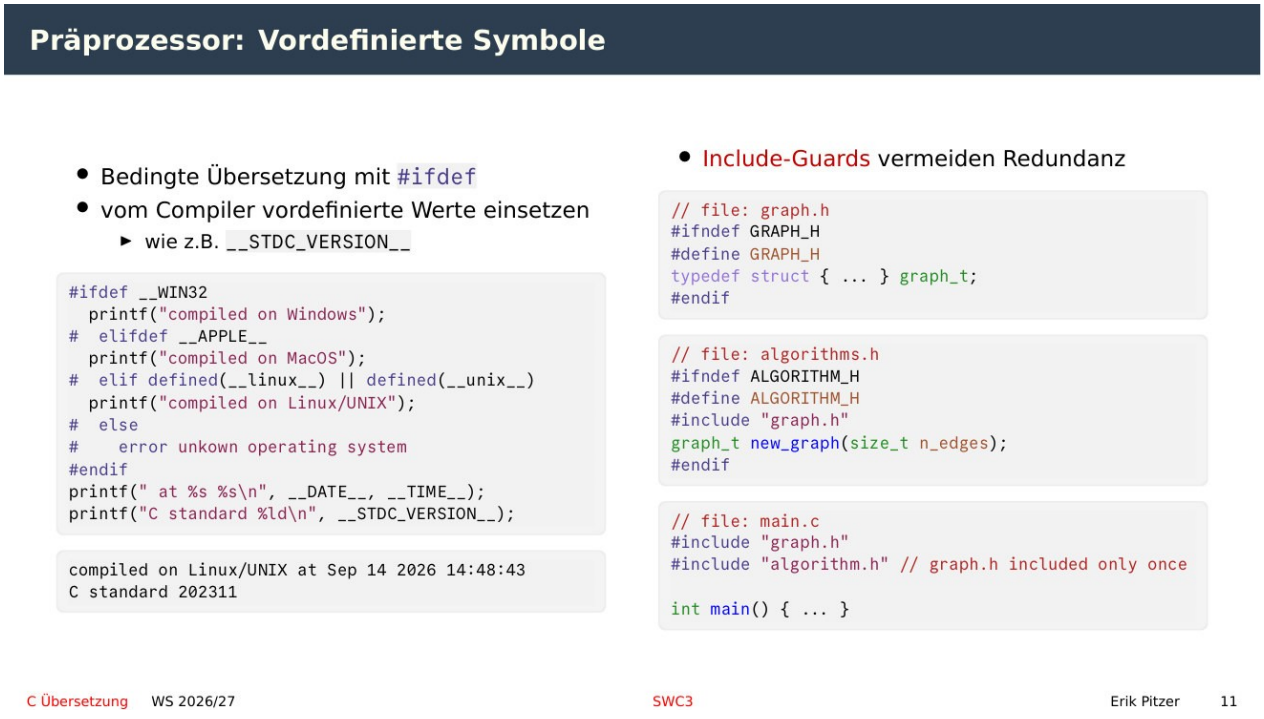
Ein **Include-Guard** schützt einen Header davor, innerhalb derselben Translation Unit mehrfach mit denselben Deklarationen verarbeitet zu werden. Typisches Muster:

```
#ifndef GRAPH_H
#define GRAPH_H

typedef struct graph graph_t;
graph_t *new_graph(size_t n_edges);

#endif
```

Das erste Einbinden setzt `GRAPH_H`. Beim zweiten Versuch ist die Bedingung von `#ifndef` falsch und der Inhalt wird übersprungen. In der gezeigten Kombination von `graph.h`, `algorithms.h` und `main.c` verhindert dies doppelte Typdefinitionen. Ein Include-Guard verbindet jedoch **keine** Bibliothek und ersetzt keine konsistente Trennung von Deklaration und Definition. Der Header muss außerdem alle benötigten Typnamen durch geeignete Standardheader oder Vorwärtsdeklarationen selbst kenntlich machen.



Originalfolie 04, PDF-Seite 12: Präprozessor: Vordefinierte Symbole

6.5 Objektdateien, Symboltabellen und Linken

Der Aufruf `gcc -c demo.c` erstellt aus einer Quelle die **Objektdatei** `demo.o`, ohne sie mit allen benötigten Implementierungen zu einem fertigen Programm zu verbinden. Sie kann übersetzten Maschinencode (**Text-Segment**), Daten für globale Objekte, Relokationsinformationen und eine **Symboltabelle** enthalten. Programme wie `nm`, `readelf` und `objdump` untersuchen diese Informationen. Die Folie zeigt `nm demo.o` mit Symbolklassen `U` für noch undefinierte externe Referenzen, `T` für global sichtbaren Code, `t` für lokalen Code sowie `B`/`b` für Objekte im Bereich der uninitialisierten beziehungsweise nullinitialisierten globalen Daten (**BSS**). Groß- und Kleinbuchstaben zeigen in diesem Beispiel globale bzw. lokale Sichtbarkeit an.

Der **Linker** löst die Referenzen zwischen Übersetzungseinheiten und Bibliotheken auf. Wenn `main.o` eine Funktion `printf` verwendet, aber deren Maschinencode nicht enthält, muss beim Linken eine passende Bibliotheksimplementation gefunden werden. Bei statischer Bindung können relevante Bibliotheksbestandteile in die ausführbare Datei integriert werden. Die Folien zeigen dies als Verbindung von `hello.o` und `libc.a` zu einer

ausführbaren Datei, in der die passenden Symbole zusammengeführt beziehungsweise aufgelöst werden. Eine Bibliothek darf dagegen selbst noch bestimmte offene Referenzen enthalten, die erst beim späteren Einbinden geklärt werden.

Beim praktischen Compileraufruf empfiehlt die Lehrveranstaltung unter anderem

```
gcc -std=c23 -pedantic \
    -Wall -Wextra -Wconversion -Wshadow \
    -o hello hello.c
```

`-std=c23` wählt den beabsichtigten Sprachstandard, `-pedantic` meldet bestimmte Nichtstandard-Konstrukte, `-Wall` und `-Wextra` aktivieren verbreitete Warnungen, `-Wconversion` weist auf verdächtige Typumwandlungen und `-Wshadow` auf überschattene Namen hin. `-o hello` bestimmt den Zieldateinamen. Der Compiler-Treiber führt gewöhnlich mehrere Schritte intern aus; für die Untersuchung können sie getrennt werden, etwa mit `-E` für Präprozessorausgabe und `-c` für eine Objektdatei.

Vertiefung · Schritt für Schritt

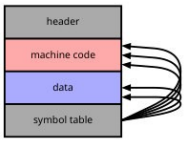
Ein Linkerfehler wie „undefined reference“ bedeutet typischerweise nicht, dass der Compiler den Funktionsaufruf nicht syntaktisch versteht. Er kann auf Grundlage eines Headers wissen, dass eine Funktion existieren soll, obwohl noch keine passende Definition für das ausführbare Programm gefunden wurde. Die Symboltabelle einer Objektdatei hält dafür Namen und Referenzen bereit. nm markiert undefinierte Referenzen mit U, Code typischerweise mit T/t und nullinitialisierte globale Daten oft mit B/b. Kleinbuchstaben kennzeichnen in den gezeigten nm-Ausgaben interne beziehungsweise lokale Symbole. Das konkrete Binärformat und die Symbolcodierung hängen jedoch vom Toolchain-System ab.

Compiler (i)

- Compiler übersetzt Resultat des Präprozessors in Maschinensprache
- kann durch Kommandozeilenparameter konfiguriert werden
- Beispiel: Übersetzen einer C-Datei `demo.c` in Objekt-Datei mit GCC

```
gcc -c demo.c
```

- Ergebnis ist Objektdatei `demo.o`
  - Maschinensprache der Funktionen
  - Initialwerte für globalen Variablen
  - Symboltabelle mit Adressen von
    - Funktionen
    - Variablen



C Übersetzung WS 2026/27 SWC3 Erik Pitzer 12

Originalfolie 04, PDF-Seite 13: Compiler (i)

Compiler (ii)

```c
// file: demo.c

extern int extern_variable;
extern int extern_function();

int my_variable = {};
void my_function(int x) {
  extern_function();
  extern_variable = 0;
}
static int my_private_function() {
  static int my_private_global = {};
}

int main() {}
```

- Analyse der erzeugten Objektdatei
  - z.B. mit `readelf`, `nm` oder `objdump`

```
nm demo.o
```

```
                 U extern_function
                 U extern_variable
0000000000000024 T main
0000000000000000 T my_function
000000000000001d t my_private_function
0000000000000004 b my_private_global.0
0000000000000000 B my_variable
```

- U … undefiniert
- T … im Text-Segment (Maschinensprache)
- B … im BSS-Segment[1] (globale Variablen)
- Kleinbuchstabe … lokale Definition

[1] Block Started by Symbol: Speicherbereich (normalierweise) mit Null initialisiert wird

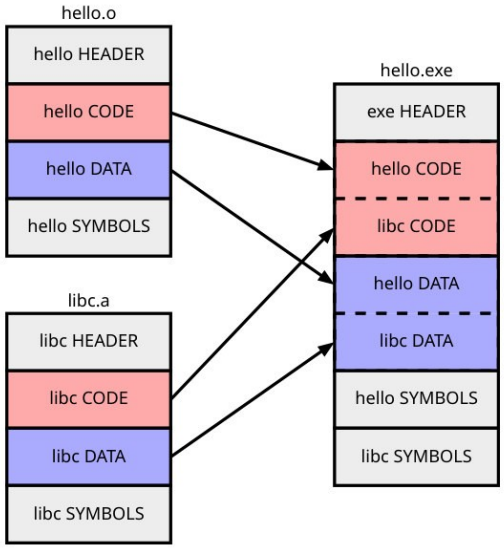
Originalfolie 04, PDF-Seite 14: Compiler (ii)

Binden (*linking*)

- Binder kombiniert mehrere Objekt-Dateien und Bibliotheken
  - entweder zu ausführbare Datei (*executable*)
    - fügt automatisch Standardbibliothek (libc) hinzu
    - prüft ob alle Symbole definiert sind
  - oder zu Bibliothek (*library*)
    - prüft nicht ob alle Symbole definiert sind

```
gcc -c hello.c # -> hello.o
gcc -static -o hello -lc hello.o # -> hello(.exe)
./hello
```

```
Hello, World!
```



Originalfolie 04, PDF-Seite 15: Binden (linking)

Beispiel Gnu Compiler Collection (GCC)

- Beispiel Übersetzen einer C-Datei in ausführbare Datei (*executable*)

```
gcc -std=c23 -pedantic \
    -Wall -Wextra -Wconversion -Wshadow \
    -o hello \
    hello.c
```

- `-o` Namen der resultierenden Datei
- `-std=c23` C23 Standard verwenden
  - `-pedantic` Warnung bei Nicht-Standard-Features
- `-W` Warnungen aktivieren
  - `-Wall` gängigste Warnungen
  - `-Wextra` zusätzliche Warnungen
  - `-Wconversion` fragwürdige Konvertierungen
  - `-Wshadow` Überschatten von Bezeichnern
- Zwischenschritt über Objektdatei wird im Hintergrund ausgeführt
  - direkte Übersetzung (`-c`) und Bindung (`-lc`) in einem Schritt

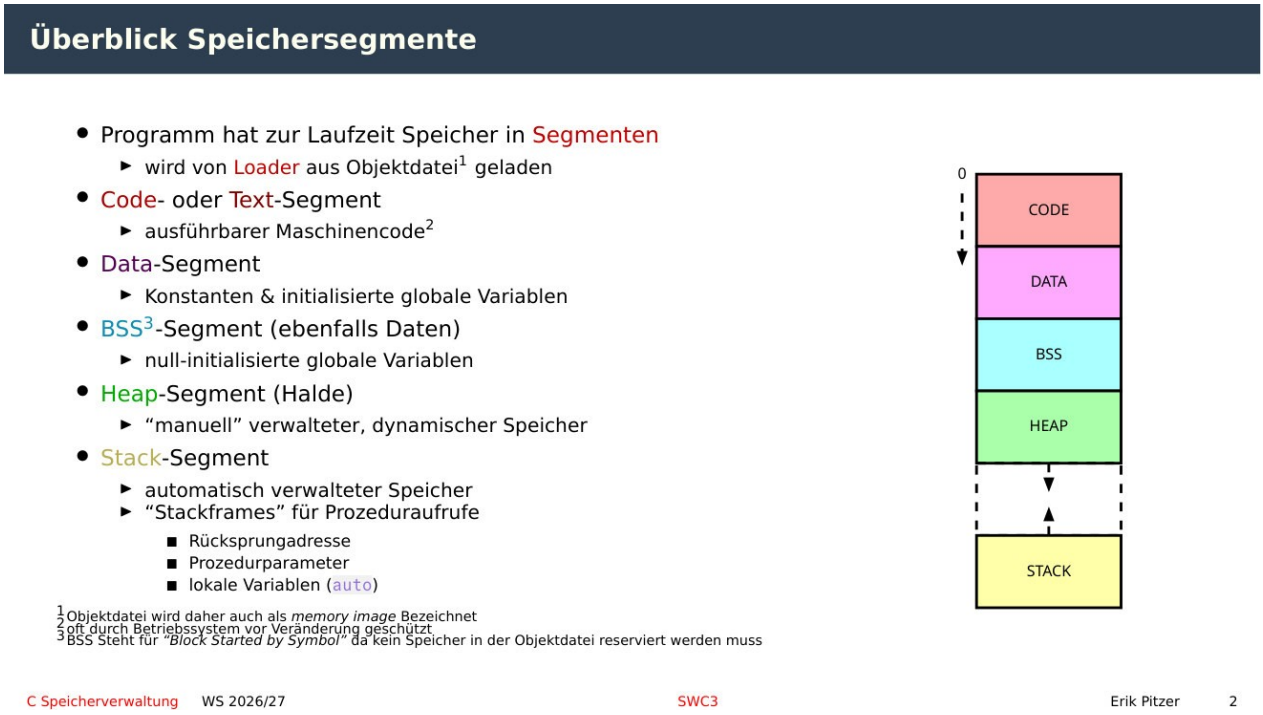
Originalfolie 04, PDF-Seite 16: Beispiel Gnu Compiler Collection (GCC)

7 Speicherverwaltung: Speicherbereiche, Lebensdauer und sichere Zugriffe

7.1 Überblick über Code, Data, BSS, Heap und Stack

Ein laufendes Programm arbeitet typischerweise mit unterschiedlichen Speicherbereichen. Das **Code- beziehungsweise Text-Segment** enthält Maschinenanweisungen und ist üblicherweise nicht beschreibbar. Das **Data-Segment** enthält unter anderem initialisierte Objekte mit statischer Speicherdauer; ein **BSS-Bereich** wird für nicht ausdrücklich mit Nichtnullwerten initialisierte globale und `static`-Objekte verwendet. Diese Einteilung beschreibt typische Executable- und Laufzeitsysteme, nicht zwingend eine festgelegte Speichergrafik jeder C-Implementierung. Der **Heap** dient üblicherweise dynamischen Speicherzuweisungen, während der **Stack** oft automatische lokale Objekte und Funktionskontexte trägt.

Die Originalgrafik stellt die Bereiche untereinander dar und deutet unterschiedliche Wachstumsrichtungen von Heap und Stack an. Es handelt sich um eine **schematische Darstellung eines üblichen Prozessadressraums**: Reale moderne Systeme nutzen virtuelle Adressen, mehrere Mappings, Schutzmechanismen, ASLR (Address Space Layout Randomization), Thread-Stacks und Optimierungen. C selbst schreibt nicht vor, dass jeder automatische lokale Wert auf einem Hardware-Stack liegt oder dass jede dynamische Zuweisung aus einem einzigen zusammenhängenden Heap-Segment stammen muss. Für die Programmlogik ist vielmehr die festgelegte **Speicherdauer und Gültigkeit** entscheidend.



Originalfolie 05, PDF-Seite 3: Überblick Speichersegmente

7.2 Ausführbarer Code und gemeinsam genutzte Bibliotheken

Ein Funktionsname kann in einem geeigneten Ausdruck in einen **Funktionszeiger** umgewandelt werden. In den Folien werden Adressen von `f`, `main` und `printf` angezeigt, um zu zeigen, dass ausführbare Maschinenanweisungen an bestimmten Stellen im Prozessadressraum liegen. Bei statisch gebundenem Code können Aufrufe im selben allgemeinen Codebereich erscheinen; bei **dynamisch geladenen Bibliotheken** wie einer `libc.so` liegt die

Implementation von `printf` typischerweise in einem separaten Memory Mapping. Das Beispiel zeigt deshalb für `printf` einen stark von den beiden eigenen Funktionen abweichenden Adressbereich.

Diese Werte sind Momentaufnahmen eines konkreten Prozesses. ASLR verändert häufig Adressen bei jedem neuen Prozessstart. Ein C-Funktionszeiger ist nicht grundsätzlich mit einem `void *`-Objektzeiger gleichzusetzen; ihre Repräsentation und erlaubte Umwandlung sind keine allgemeine ISO-C-Garantie. Die Folienmakros zur Ausgabe solcher Adressen sind deshalb primär **plattformgebundene Demonstrationswerkzeuge**, nicht Vorlage für portable Zeigerformatierung.

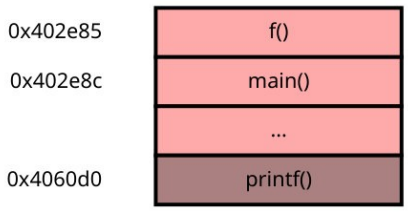
Code Segment

- enthält Maschinensprache
- kann direkt ausgeführt werden
- Funktionsname ist Zeiger ins Code-Segment

```c
int f() { }

int main() {
  SHOW_TYPE(f);
  SHOW_TYPE(main);
  SHOW_TYPE(printf);
}
```

```
f = 0x402e85 (function)
main = 0x402e8c (function)
printf = 0x4060d0 (library function)
```



Originalfolie 05, PDF-Seite 4: Code Segment

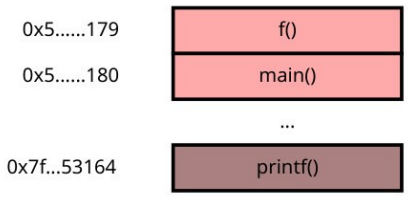
Code Segmente von gemeinsamen Bibliotheken

- Bibliotheksfunktionen oft gemeinsam mit anderen Programmen genutzt (shared library)
- Zeiger von `printf()` daher in anderes Segment

```c
int f() { }

int main() {
  SHOW_TYPE(f);
  SHOW_TYPE(main);
  SHOW_TYPE(printf);
}
```

```
f = 0x557b4a1ec179 (function)
main = 0x557b4a1ec180 (function)
printf = 0x7f8c30e56a64 (library function)
```

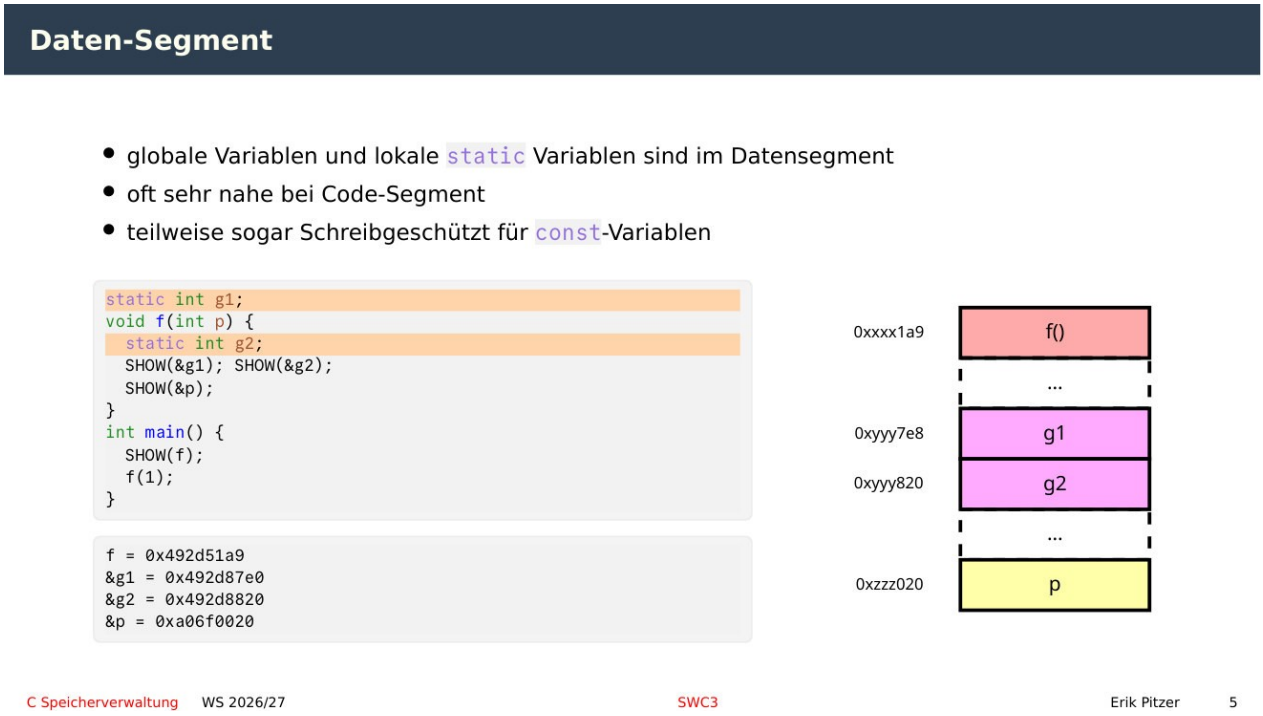


Originalfolie 05, PDF-Seite 5: Code Segmente von gemeinsamen Bibliotheken

7.3 Globale Objekte und lokale statische Variablen

Eine globale Variable oder eine lokale `static`-Variable hat **statische Speicherdauer**: Sie existiert während der gesamten Programmausführung und behält ihren gespeicherten Wert zwischen Funktionsaufrufen. Die Folie vergleicht die Adressen einer globalen Variablen `g1`, einer innerhalb von `f` definierten statischen Variablen `g2` und eines Funktionsparameters `p`. In der beispielhaften Laufzeit liegen `g1` und `g2` im allgemeinen Datenbereich, `p` dagegen an einer Adresse, die dem automatischen Funktionskontext zugeordnet ist. Der Sichtbarkeitsbereich von `g2` bleibt trotz statischer Speicherdauer auf die deklarierende Funktion beschränkt.

Das ist der Unterschied zwischen **Sichtbarkeit, Linkage und Speicherdauer**: Eine Datei-`static`-Variable ist nur innerhalb dieser Translation Unit extern sichtbar, eine lokale `static`-Variable ist nur innerhalb ihres Blocks benennbar, beide leben aber lange. `const` kann dazu beitragen, dass unveränderliche Daten in schreibgeschützten Bereichen abgelegt werden; der konkrete Schutz und die Platzierung sind implementierungsabhängig. Auch die BSS-Unterteilung ist eine Beschreibung üblicher Linker, nicht eine zusätzliche C-Speicherklasse.



Originalfolie 05, PDF-Seite 6: Daten-Segment

7.4 Stackframes, Funktionsaufrufe und ABI

Beim Aufruf einer Funktion muss der Prozessor beziehungsweise die Laufzeitumgebung den Rückweg und eventuell Registerinhalte, lokale Werte und Argumente verwalten. Ein **Stackframe** ist der konzeptuelle Kontext eines bestimmten Funktionsaufrufs. Das Stackdiagramm zeigt eine Rücksprungadresse `RET`, einen vorigen Basiszeiger `old BP`, Funktionsparameter und lokale Variablen. SP und BP stehen dort für Register, die einen Stack beziehungsweise einen Frame verwalten können. Bei mehreren ineinander verschachtelten Aufrufen existieren entsprechend mehrere solche Funktionskontexte.

Eine **Calling Convention (Aufrufkonvention)** regelt unter anderem, welche Register für welche Argumente und Rückgabewerte benutzt werden, welche Register der Aufrufer oder die aufgerufene Funktion sichern muss, welche Alignment-Regeln gelten und wie der Stack organisiert wird. Zusammen mit weiteren Binärregeln bildet sie eine **Application Binary Interface (ABI)**. Die **Application Programming Interface (API)** beschreibt demgegenüber die Programmierschnittstelle auf Quelltextniveau, also insbesondere Namen, Parametertypen und Verwendungsregeln. Ein C-API kann auf mehreren Systemen gleich aussehen, während die dazugehörige ABI unterschiedlich ist. Gerade für

die Anbindung externer Bibliotheken und anderer Programmiersprachen müssen beide Seiten der konkreten ABI entsprechen.

Die Folie zur Parameterübergabe zeigt vereinfacht, wie `y = inc(y)` abgearbeitet wird: `main` besitzt zunächst `y=1`, die aufgerufene Funktion erhält eine Kopie für `x`, erzeugt einen temporären Wert `temp=1`, erhöht diesen auf 2, liefert 2 zurück und `main` speichert das Ergebnis in `y`. Die Illustration zeigt Parameter „über Stack“ und Rückgabe „im Register AX“. **Ergänzende Einordnung:** Das ist eine didaktische Modellkonvention; auf verbreiteten x86-64-ABIs werden viele Argumente tatsächlich in Registern übergeben, und optimierter Compiler-Code kann lokale Variablen vollständig in Registern halten. Das logische Ergebnis der Call-by-value-Semantik bleibt dennoch gleich.

Vertiefung · Schritt für Schritt

Der Ablauf beim Aufruf `inc(y)` wird verständlich, wenn man zwischen abstrakter C-Semantik und Maschinencode unterscheidet. Abstrakt wird der Wert von y ausgewertet und in den formalen Parameter x kopiert; im Funktionsrumpf wird aus x ein neues Ergebnis berechnet und beim Rücksprung an den Aufrufer geliefert. Maschinenabhängig kann der Parameter dafür im Stack oder in einem Register liegen. Die in der Folie gezeigten Register AX, BP und SP entsprechen einer bestimmten Modellarchitektur; auf x86-64 werden viele Argumente regulär in Registern übergeben. Die Unterscheidung ist zentral, weil das C-Sprachmodell call-by-value garantiert, nicht aber eine konkrete Reihenfolge und Position aller Stackeinträge.

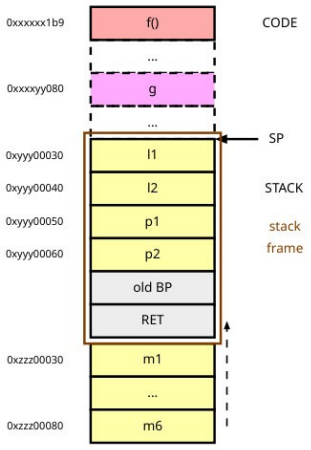
Stack-Segment (i)

- Funktionsparameter und lokale Variablen liegen im Stack-Segment

```c
int g = 1;
void f(int p1, int p2) {
  int l1, l2;
  SHOW2(&l1, &l2);
  SHOW2(&p1, &p2);
}
int main() {
  SHOW(&f);
  SHOW(&g);
  int m1, m2, m3, m4, m5, m6;
  f(1, 2);
  SHOW2(&m1, &m2);
  SHOW2(&m3, &m4);
  SHOW2(&m5, &m6);
}
```

- Parameter und lokale Variablen eines Funktionsaufrufs bilden ein Stack-Frame am Stack

```
&f = 0x10e111c9
&g = 0x10e14080
&l1 = 0x66bf0030;  &l2 = 0x66bf0040
&p1 = 0x66bf0050;  &p2 = 0x66bf0060
&m1 = 0x66cf0030;  &m2 = 0x66cf0040
&m3 = 0x66cf0050;  &m4 = 0x66cf0060
&m5 = 0x66cf0070;  &m6 = 0x66cf0080
```



C Speicherverwaltung WS 2026/27 SWC3 Erik Pitzer 6

Originalfolie 05, PDF-Seite 7: Stack-Segment (i)

Stack-Segment (ii)

- Aufrufkonventionen (*calling conventions*) definieren
  - wo Parameter abgelegt werden, z.B. am Stack
  - wo Rückgabewert abgelegt wird, z.B. im Register AX
  - wer welche Register wiederherstellen muss
- gemeinsam mit weiteren Parametern bilden sie Application Binary Interface (ABI)
  - beschreibt Interoperabilität zur Laufzeit
  - erlaubt Interoperabilität zwischen Prozeduren
    - aus unterschiedlichen Programmiersprachen
    - von verschiedenen Compiler
- im Gegensatz dazu Application Programming Interface (API)
  - beschreibt Interoperabilität zur Compile-Zeit

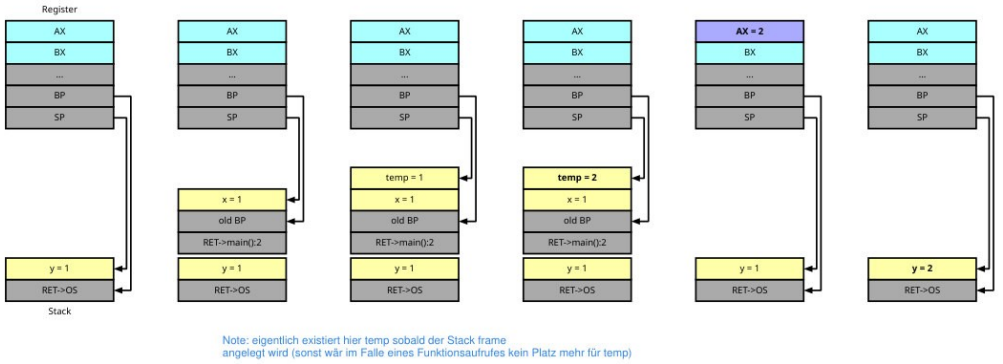
Originalfolie 05, PDF-Seite 8: Stack-Segment (ii)

Parameterübergabe

- Parameter über Stack übergeben
- Rückgabewert über Register AX
- Stackframe nur kurz gültig

```c
int inc(int x) {
    int temp = x;
    temp++;
    return temp;
}
```

```c
int main() {
    int y = 1;
    y = inc(y);
    SHOW(y);
}
```



Originalfolie 05, PDF-Seite 10: Parameterübergabe

7.5 Lebensdauer lokaler Variablen, Dangling Pointer und Rekursion

Eine automatische lokale Variable existiert für die Dauer ihres jeweiligen Ausführungsblocks. Nach Rückkehr aus der Funktion endet ihre Lebensdauer; ein Zeiger auf diese Variable wird zum **dangling pointer**. Im Folienschema erzeugt `int *make_array() { int a[10]; return a; }` einen Zeiger auf ein lokales Array und gibt ihn zurück. Nach dem Rücksprung darf der Aufrufer dieses Array nicht mehr verwenden. Das Beispiel beobachtet, dass GCC die ungültige Rückgabe in einer konkreten Variante als Nulladresse ausgibt; das ist eine Compilerreaktion, kein vorgeschriebenes C-Verhalten. Die richtige Lösung ist beispielsweise, Speicher mit passender Lebensdauer vom Aufrufer bereitstellen zu lassen oder dynamisch zu reservieren und die Verantwortung zur Freigabe zu dokumentieren.

Auch mehrfaches Aufrufen einer Funktion kann denselben physischen Stackbereich erneut nutzen. Daraus folgt aber nicht, dass eine automatische Variable beim zweiten Aufruf ihren alten Wert garantieren würde. Die auf der Folie gezeigten wiederholten Werte stammen aus zufällig vorhandenen Speicherinhalten bzw. Implementierungsdetails. **Nicht initialisierte lokale Variablen dürfen nicht gelesen werden, bevor ein gültiger Wert zugewiesen wurde.**

Bei **Rekursion** ruft eine Funktion sich selbst direkt oder indirekt auf. Die Folie demonstriert eine rekursive Funktion ohne Abbruchbedingung, die bei zunehmender Tiefe schließlich mit einem Speicherfehler wie `SIGSEGV` endet. Jeder nicht wegoptimierte rekursive Aufruf benötigt zusätzliche Ressourcen; die maximale Tiefe hängt von Framegröße, Compiler, Optimierung und Prozesskonfiguration ab. Eine sinnvolle rekursive Funktion benötigt eine **Abbruchbedingung** und eine Fortschrittsbedingung, die diese schließlich erreicht. Tail-Call-Optimierung kann den Speicherbedarf mancher Rekursionen reduzieren, ist aber nicht generell durch C vorgeschrieben.

Vertiefung · Schritt für Schritt

Ein Pointer wird nicht dadurch gültig, dass sein numerischer Adresswert nach Verlassen der Funktion noch existiert. Entscheidend ist die Lebensdauer des Objekts, auf das der Zeiger zeigte. Ein Rückgabewert vom Typ `int *` darf beispielsweise auf eine dynamisch reservierte Allocation oder ein vom Aufrufer übergebenes Array verweisen, aber nicht auf ein gewöhnliches lokales `int a[10]`. Deshalb behandelt die Folie die vermeintlich gleiche Adresse bei mehreren Aufrufen nur als zufällige Beobachtung. Gleiches gilt für anfangs „sichtbare“ Zahlen in uninitialisierten lokalen Variablen: Daraus darf niemals ein Startwert abgeleitet werden.

Stack-Segment (iii)

- lokale Variablen nur in Funktion gültig
  - neuer Aufrufe → gleiches Stack-Frame[1]

```c
int f() {
  int x;
  SHOW(x);
  x = 2;
  SHOW(x);
  SHOW(&x);
}
int main() {
  f(); f();
}
```

```
x = 0
x = 2
&x = 0xd871c8e4
x = 2
x = 2
&x = 0xd871c8e4
```

- Rückgabe von Zeiger auf Stack
  - nach Funktionsaufruf ungültig
  - GCC ersetzt Rückgabewert durch `0`

```c
int* make_array() {
    int a[10];
    SHOW(a);
    return a; // NOTE: compiler warning
}
int main() {
    int* a1 = make_array();
    int* a2 = make_array();
    SHOW2(a1, a2);
}
```

```
a = 0x397b6b90
a = 0x397b6b90
a1 = 0x0;  a2 = 0x0
```

[1] in GCC verhindert, Beispiele übersetzt mit `-fno-stack-clash-protection`

C Speicherverwaltung WS 2026/27 SWC3 Erik Pitzer 8

Originalfolie 05, PDF-Seite 9: Stack-Segment (iii)

Rekursion

- jeder rekursive Funktionsaufruf benötigt weiteres Stack-Frame
  - Rekursionstiefe daher in der Praxis eingeschränkt

```c
void count_recursive(size_t n) {
  fprintf(stderr, "n = %zu\n", n);
  count_recursive(n+1); // NOTE: infinite recursion (warning)
}
int main() {
  count_recursive(0);
}
```

```
n = 0
n = 1
n = 2
...
n = 312763
n = 312764
n = 312765
Job 1, './count_recursive' terminated by signal SIGSEGV (Address boundary error)
```

C Speicherverwaltung WS 2026/27 SWC3 Erik Pitzer 10

Originalfolie 05, PDF-Seite 11: Rekursion

7.6 Dynamischer Speicher mit `malloc`, `calloc` und `free`

Dynamisch zugewiesener Speicher wird mit Funktionen aus `<stdlib.h>` verwaltet. `malloc(bytes)` reserviert einen zusammenhängenden Speicherbereich der angegebenen Bytegröße und liefert bei Erfolg einen geeigneten Objektzeiger; bei Misserfolg liefert es einen Nullzeiger. Der Speicher ist **nicht initialisiert**. `calloc(n, size)` reserviert Speicher für `n` Elemente zu jeweils `size` Bytes und setzt alle Bytes des Speicherbereichs auf null. Für die üblichen Integer- und Gleitkommaobjekte bedeutet das beim gezeigten System einen numerischen Nullwert. **Ergänzende Einordnung:** Byteweises Nullsetzen ist nicht für jeden denkbaren Typ und jede Implementierung gleichbedeutend mit einer gültigen semantischen Nullinitialisierung, insbesondere nicht ohne Weiteres für jedes Zeiger- oder Spezialformat.

Ein korrektes Muster ist:

```c
#include <stdlib.h>
#include <stddef.h>

int *make_int_array(size_t n) {
    if (n > SIZE_MAX / sizeof(int)) return nullptr; // Overflow vermeiden
    return calloc(n, sizeof(int));
}

int main(void) {
    size_t n = 10;
    int *a = make_int_array(n);
    if (a == nullptr) return EXIT_FAILURE;
    a[0] = 123;
    free(a);
    a = nullptr;
    return EXIT_SUCCESS;
}
```

SIZE_MAX ist in <stdint.h> definiert. Die Multiplikationsprüfung verhindert, dass n * sizeof(int) im Typ size_t vor der Allokation überläuft. free hebt eine erfolgreich reservierte Speicherzuordnung auf. Ein Speicherleck entsteht, wenn allokierter Speicher nicht mehr erreichbar bleibt und deshalb nicht mehr freigegeben werden kann. Nach free(a) enthält a möglicherweise einen nun ungültigen Zeigerwert, daher sollte man ihn nicht mehr verwenden; eine anschließende Zuweisung a = nullptr macht die Absicht klar. free(nullptr) ist zulässig und hat keine Wirkung.

Das Folienbeispiel `make_int_array(10)` verwendet `calloc`, gibt den Zeiger zurück, liest ein zu null gesetztes Element und gibt den Block schließlich frei. Es reserviert anschließend einen neuen Block für 20 Elemente und gibt auch diesen frei. Die gezeigten unterschiedlichen Zeigeradressen sind illustrative Ausgaben. Die Faustregel „zu jedem `malloc`/`calloc` gehört ein `free`“ ist als **Eigentumsregel** zu verstehen: Jede erfolgreiche Reservierung muss irgendwann **genau einmal** korrekt freigegeben werden; das muss nicht in derselben Funktion geschehen.

Die Folie zum Größenvergleich stellt ein lokales großes Array mit etwa 3 Mi Elementen einem noch größeren Heapblock mit etwa 1 Gi Elementen gegenüber. Dabei ist zu beachten: Die aufgeführten Größen in MiB beziehen sich nicht automatisch auf den belegten Speicher, wenn das Array Elemente vom Typ `int` besitzt. Bei `sizeof(int)=4` benötigen 3 Mi Integer ungefähr **12 MiB** und 1 Gi Integer ungefähr **4 GiB**, nicht 3 bzw. 1024 MiB. Auch die erfolgreiche `calloc`-Reservierung garantiert nicht, dass ein Betriebssystem alle physischen Seiten sofort zuordnet. Die konkrete Ausführung, ihre Ausgaben und die verfügbare Speichermenge sind umgebungsabhängig.

Vertiefung · Schritt für Schritt

Die drei wichtigsten Eigentumsfehler sind: Speicherleck (die einzige erreichbare Adresse zu einem noch allokierten Block wird verloren), Double Free (dieselbe Allocation wird zweimal freigegeben) und Use After Free (nach erfolgreichem `free` erfolgt ein weiterer Zugriff). Eine gute Schnittstelle dokumentiert daher, wer den Block besitzt und wann er freizugeben ist. Mit `malloc`(n * `sizeof` *p) wird die Elementgröße aus dem Zieltyp abgeleitet. Vor der Multiplikation muss geprüft werden, ob n > `SIZE_MAX` / `sizeof` *p gilt, sonst könnte der berechnete Byteumfang überlaufen. Ein fehlgeschlagenes `malloc`/`calloc` darf nicht dereferenziert werden.

Bei `calloc` ist die Unterscheidung zwischen bitweiser Nullsetzung und sprachlicher Nullinitialisierung wesentlich: `calloc` schreibt Nullbits in die reservierten Bytes. Auf gewöhnlichen Plattformen entspricht das einer Null für Integer, Fließkommazahlen und Nullzeiger, aber der ISO-C-Standard garantiert für Zeiger und Fließkommazahlen nicht generell dieselben Bitmuster. Soll ein Array von Zeigern garantiert auf Nullzeiger gesetzt werden, kann man die Einträge ausdrücklich zuweisen. Bei `malloc` ist ohnehin der gesamte zurückgegebene Speicher zunächst uninitialisiert.

Heap-Segment und manuelle Speicherverwaltung (i)

- Speicher im Heap-Segment wird zur Laufzeit manuell (dynamisch) verwaltet
- Funktionen im Header `<stdlib.h>`
  - `malloc()` (*memory allocate*) Speicher reservieren
    - returniert `nullptr` falls nicht möglich
  - `free()` Speicher frei gegeben
- Verwendung wie normales Array, z.B.
  - für größere Datenmengen
  - Rückgabewerte von Prozeduren
- manuelle Speicherverwaltung ist historisch größte Hürde für C-Programmierung

```c
int* a = malloc(sizeof(int)*10);
if (a != nullptr) {
  SHOW(a[0]); // NOTE: content not initialized
  a[0] = 123;
  SHOW(a[0]);
  free(a);
}

SHOW(a); // address of dynamic memory
SHOW(&a); // address of variable on stack
```

```
a[0] = -1094795586
a[0] = 123
a = 0x2bfe0010
&a = 0x2aaf0020
```


Originalfolie 05, PDF-Seite 12: Heap-Segment und manuelle Speicherverwaltung (i)

Heap-Segment und manuelle Speicherverwaltung (ii)

- Beispiel Rückgabe von Array
  - `calloc()` initialisiert Speicher mit `0`
  - manuell reservierter Speicher wird mit `free()` wieder freigegeben
    - ansonsten → Speicherleck (memory leak)

```c
int* make_int_array(size_t n) {
    int* result = calloc(n, sizeof(int)); // <---- allocate ---.
    return result;                        //                   |
}                                         //                   |
int main() {                              //                   |
    int* a = make_int_array(10);          //                   |
    SHOW(a[0]);                           //                   +
    SHOW(a);                              //                  /|
    free(a); // <---- free ---------------------------------'  |
    a = make_int_array(20); //                                 |
    SHOW(a);                //                                 |
    free(a); // <---- free ------------------------------------'
}
```

```
a[0] = 0
a = 0x25e0010
a = 0x25e0020
```

Faustregel: *Für jeden Aufruf von `malloc()` oder `calloc()` muss es einen entsprechenden Aufruf von `free()` geben.*

Originalfolie 05, PDF-Seite 13: Heap-Segment und manuelle Speicherverwaltung (ii)

Heap-Segment und manuelle Speicherverwaltung (iii)

Beispiel: Verarbeitung von großen Datenmengen nur am Heap möglich

- Speicher am Stack (3 MiB)
  - Programm bricht zur Laufzeit ab

```
constexpr size_t MAX = 3 * 1024 * 1024;
void huge_stack() {
    int a[MAX] = {};
    SHOW(a[MAX-1]);
}
int main() {
    huge_stack();
}
```

```
stack-overflow on address 0x7fff5efbf300
    #0 0x5596cca641c2 in huge_stack
    #1 0x5596cca642e4 in main
    #2 0x7f9be0e27674 libc
    #3 0x7f9be0e27728 in __libc_start_main
    #4 0x5596cca640d4 in _start
```

- Speicher auf Heap (1024 MiB)
  - kein Problem

```
constexpr size_t MAX = 1024 * 1024 * 1024;

void huge_heap() {
    int* a = calloc(MAX, sizeof(int));
    SHOW(a[MAX-1]);
    free(a);
}

int main() {
    huge_heap();
}
```

```
a[MAX-1] = 0
```

C Speicherverwaltung WS 2026/27 SWC3 Erik Pitzer 13

Originalfolie 05, PDF-Seite 14: Heap-Segment und manuelle Speicherverwaltung (iii)

7.7 Sanitizer und Memory-Profiling

Sanitizer sind Prüfverfahren, bei denen Compiler zur Übersetzungszeit zusätzliche Überwachung in das Programm einbauen. Der **AddressSanitizer (ASan)** erkennt viele ungültige Speicherzugriffe, insbesondere Pufferüberläufe und use-after-free; der **MemorySanitizer (MSan)** konzentriert sich auf Verwendungen nicht initialisierter Daten; der **UndefinedBehaviorSanitizer (UBSan)** instrumentiert bestimmte Kategorien undefinierten Verhaltens. Die Folie nennt ungefähre Performance-Mehrkosten als Beispiele, keine plattformunabhängigen Garantiewerte. Typischer GCC-Aufruf ist `gcc -fsanitize=address,undefined -g ...`; die genaue Unterstützung hängt von Compiler, Toolchain und Plattform ab.

Valgrind Memcheck führt eine andere, meist stärker überwachte Programmanalyse zur Laufzeit durch. Ein Aufruf wie `valgrind --leak-check=yes ./hello` liefert unter anderem eine Heap Summary mit Anzahl allokierter und freigegebener Blöcke sowie Informationen über noch erreichbaren oder verlorenen Speicher. Im Folienbeispiel heißt „All heap blocks were freed“, dass bei dieser Programmausführung keine gemeldeten Heaplecks verblieben sind. Weder Sanitizer noch Profiler prüfen automatisch alle möglichen Programmpfade; für eine zuverlässige Beurteilung müssen Testfälle, Compilerwarnungen, klare Schnittstellenverträge und korrekte Eigentumsregeln zusammenspielen.

Vertiefung · Schritt für Schritt

Sanitizer prüfen unterschiedliche Fehlerklassen. AddressSanitizer instrumentiert beispielsweise Speicherzugriffe und entdeckt häufig Pufferüberschreitungen, Use After Free und bestimmte ungültige Stackzugriffe. UndefinedBehaviorSanitizer erfasst ausgewählte Fälle undefinierten Verhaltens wie `signed` Integerüberlauf. MemorySanitizer kann die Verwendung uninitialisierter Werte verfolgen, wird aber nicht mit jeder Compilerumgebung angeboten. Valgrind Memcheck arbeitet anders: Es überwacht Speicherzugriffe und Allokationen während der Ausführung und ist typischerweise erheblich langsamer. Eine fehlerfreie Prüfung eines Testlaufs beweist keine Fehlerfreiheit der nicht getesteten Pfade.

Sanitizer & Memory Profiler

- Sanitizer erzeugen zusätzlichen Code der zur Laufzeit die “Vernünftigkeit” (*sanity*) des Programms überprüfen
  - Address Sanitizer (ASan)
    - use after free/return/scope
    - heap/stack/global buffer overflow
    - memory leaks
    - reduziert Geschwindigkeit um ca. 50%
  - Memory Sanitizer (MSan)
    - uninitilized memory access
    - reduziert Geschwindigkeit um ca. 75%
  - Undefined Behavior Sanitizer (UBSan)
    - erkennt undefiniertes Verhalten ~40 Prüfungen
    - kaum Laufzeitoverhead
  - können durch Compiler-Parameter aktiviert werden,
    - z.B. `gcc -fsanitize=address,undefined ...`

- Speicher-Profiler, wie, z.B. Valgrind Memcheck
  - exaktes Verfolgen von gültigem Speicherzugriffen
  - Rückschlüsse auf Speicherlecks
  - langsamer aber gründlicher als Sanitizer
  - z.B.

```
gcc -o hello hello.c
valgrind --leak-check=yes ./hello
```

```
HEAP SUMMARY:
    in use at exit: 0 bytes in 0 blocks
  total heap usage: 1 allocs, 1 frees,
                    4,096 bytes allocated

All heap blocks were freed
    -- no leaks are possible
```


Originalfolie 05, PDF-Seite 15: Sanitizer & Memory Profiler

8 Bibliotheken, Linkverfahren und Interoperabilität

8.1 Aufbau einer Bibliothek

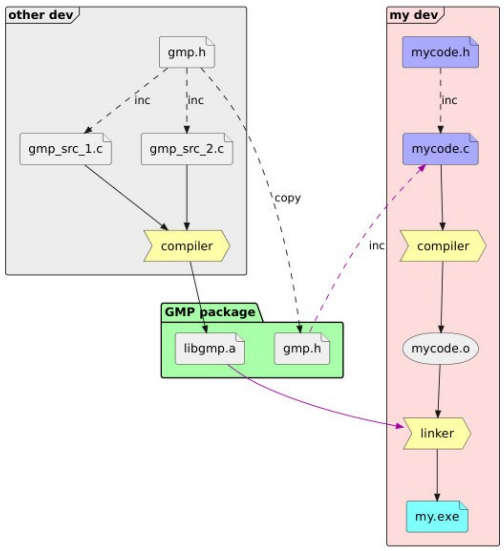
C-Bibliotheken bestehen gewöhnlich aus einer **Quelltextschnittestelle** und einer **Implementierung**. Die Schnittstelle wird über einen oder mehrere Header (`.h`) bereitgestellt. Die Implementation kann direkt als `.c`-Quellcode vorliegen, in eine **statische Bibliothek** (`.a` beziehungsweise plattformabhängig `.lib`) verpackt oder als **dynamische Bibliothek** (`.so` unter vielen Unix-Systemen, `.dll` unter Windows) ausgeliefert sein. Der Compiler benötigt die Deklarationen, um die Quelltextaufrufe zu prüfen und passende Aufrufsequenzen zu erzeugen. Der Linker benötigt die dafür nötigen Symbole und Implementierungen. Das Architekturdiagramm zeigt, wie eine fremde GMP-Bibliothek zusammen mit eigenem `mycode.c` zu `my.exe` verbunden wird.

Die **GNU Multiple Precision Arithmetic Library (GMP)** ermöglicht Rechnungen mit ganzen Zahlen jenseits der fest eingebauten Größenbeschränkungen. Der Typ `mpz_t` repräsentiert eine Ganzzahl mit dynamisch angepasster Genauigkeit. In der Originalfolge wird ein Programm `mathdemo.c` erstellt, das zwei sehr große Dezimalzahlen aus den Kommandozeilenparametern liest, sie multipliziert und das Ergebnis als Zeichenkette ausgibt. Der Wert der Basiskonstante `BASE=10` bedeutet, dass die Eingaben und die Ausgabe im Dezimalsystem interpretiert werden.

Bibliotheken

- in der Praxis häufig Aufbau auf bestehende Funktionalität
- C Bibliotheken bestehen aus zwei Teilen
  - Header Dateien (`*.h`) mit Deklarationen
  - Implementierung
    - entweder als Quelltext (`*.c`)
    - als statische Bibliothek oder "Archiv" von Objektcode (`*.lib` oder `*.a`)
    - als dynamische Bibliothek, z.B. shared object (`*.so`) oder dynamic-link library (`*.dll`)
- Compiler benötigt Header-Dateien
- Linker benötigt Implementierung
- Beispiel: "GNU Multiple Precision Arithmetic Library" (GMP)[1]

[1] https://gmplib.org/



C Bibliotheken & Interoperabilität WS 2026/27 SWC3 Erik Pitzer 2

Originalfolie 06, PDF-Seite 3: Bibliotheken

8.2 GMP-Praxisbeispiel: Initialisieren, Multiplizieren, Ausgabe und Aufräumen

Das Beispiel bindet `<gmp.h>` ein und legt `mpz_t x, y, result;` an. `mpz_init_set_str(x, argv[1], 10)` initialisiert `x` und verarbeitet das erste Zahlenargument; analog `y`. `mpz_init(result)` bereitet einen Ergebniswert vor. `mpz_mul(result, x, y)` schreibt das Produkt der beiden beliebig großen Ganzzahlen in `result`. Schließlich erzeugt `mpz_get_str(nullptr, 10, result)` eine dezimale Zeichenkette, die `printf` ausgeben kann. Die Folie zeigt für `32189045321 × 95639067` das Resultat `3078530262121155507`; das zweite Beispiel enthält entsprechend wesentlich längere Eingaben und Resultate.

Wichtige Ergänzung zur abgekürzten Folie: Das Programm muss die Argumentzahl ausreichend prüfen, den Rückgabestatus von `mpz_init_set_str` kontrollieren und seine Ressourcen freigeben. Für die Eingabe von

Zahlargumenten reichen bei zwei Operanden mindestens `argc == 3` (`argv[0]` ist der Programmname). Nach Verwendung müssen die drei GMP-Werte über `mpz_clear` freigegeben werden. Der von `mpz_get_str(nullptr, ...)` neu reservierte String muss gemäß GMP-Allocator-Regeln freigegeben werden; bei Standardallocator gewöhnlich mit `free`, sofern keine eigenen GMP-Speicherfunktionen konfiguriert wurden. Die Folie lässt die Bereinigung als `// cleanup ...` absichtlich offen. Ohne diesen Schritt wäre das Programm als allgemeines langlebiges Muster unvollständig.

Vertiefung · Schritt für Schritt

Der GMP-Funktionsablauf ist eine vollständige Ressourcenlebensdauer. Zunächst prüft das Programm, ob mindestens zwei Zahlzeichenfolgen als Argumente bereitstehen. `mpz_init_set_str` versucht, die Zeichenketten in interne Ganzzahlobjekte umzuwandeln; diese Funktion liefert einen Status, der geprüft werden sollte. `mpz_init` initialisiert das Ergebnisobjekt, `mpz_mul` multipliziert die beiden Operanden, und `mpz_get_str` erzeugt bei Übergabe eines Nullzeigers eine neu allokierte Zeichenkette. Diese muss anschließend passend freigegeben werden; unabhängig davon benötigt jedes zuvor erfolgreich initialisierte `mpz_t` ein `mpz_clear`. Das „cleanup …“ auf der Originalfolie ist somit kein optionaler Nebenschritt.

Einbinden von Bibliotheken: Beispiel GNU MP (i)

```c
// file: mathdemo.c
#include <stdio.h>
#include <stdlib.h>
#include <gmp.h>
int main(int argc, char* argv[static argc+1]) {
  if (argc <= 2) exit(1);

  constexpr int BASE = 10;
  mpz_t x, y, result;
  mpz_init_set_str(x, argv[1], BASE);
  mpz_init_set_str(y, argv[2], BASE);
  mpz_init(result);

  mpz_mul(result, x, y);

  char *result_str =
    mpz_get_str(nullptr, BASE, result);
  printf("%s * %s = %s",
         argv[1], argv[2], result_str);
  // cleanup ...
}
```

- Rechnen mit beliebiger Genauigkeit
- Bibliothek *GNU multiple precision arithmetic library* (gmp)
  - Header `/usr/include/gmp/gmp.h`
  - Bibliothek `/usr/lib/gmp/libgmp.so`
- `mpz_t` ist ganze Zahl mit beliebiger Genauigkeit
- Einlesen der Kommandozeilenparameter `argv[1]` und `argv[2]`
  - z.B. `./mathdemo 32189045321 95639067`

```
32189045321 * 95639067 = 3078530262121155507
```

C Bibliotheken & Interoperabilität WS 2026/27 SWC3 Erik Pitzer 3

Originalfolie 06, PDF-Seite 4: Einbinden von Bibliotheken: Beispiel GNU MP (i)

8.3 Header-Suchpfad, Bibliothekssuchpfad und Linkerfehler

Der erste Build-Schritt lautet in den Folien `gcc -I/usr/include/gmp -c mathdemo.c`. `-I` fügt einen zusätzlichen Suchpfad für Header hinzu; `-c` stoppt nach der Objektdatei `mathdemo.o`. Das erfolgreiche Übersetzen belegt nur, dass der Compiler die Deklarationen kennt. Der zweite Versuch `gcc -o mathdemo mathdemo.o` scheitert beim Linken, weil Referenzen auf die GMP-Funktionen wie `__gmpz_init_set_str`, `__gmpz_mul` und `__gmpz_get_str` **noch nicht aufgelöst** werden konnten. Dieser Fehler ist etwas anderes als eine fehlende Headerdatei oder ein Syntaxfehler im C-Quelltext.

Die erfolgreiche Linkvariante ist `gcc -L/usr/lib/gmp -o mathdemo mathdemo.o -lgmp`. `-L` ergänzt den Suchpfad des Linkers, `-lgmp` fordert die Bibliothek GMP an. Der Linker sucht unter passenden Konventionen Dateien namens `libgmp.so` oder `libgmp.a`. Auf vielen GNU-Linkern ist auch die **Reihenfolge** der Objektdateien und Bibliotheksoptionen relevant; die Verwendung `mathdemo.o -lgmp` ist deshalb zweckmäßig. Das konkrete Dateisystemlayout `/usr/include/gmp` und `/usr/lib/gmp` gilt nicht für jede Installation. Auf anderen Systemen

liefern Paketsysteme oder `pkg-config` die korrekten Optionen. Das Beispiel ist als reproduzierbarer Ablauf zu verstehen: Deklarationen suchen → kompilieren → externe Symbole auflösen → Programm ausführen.

Vertiefung · Schritt für Schritt

Für den Compiler sind der Deklarationspfad und der Linkerpfad verschiedene Fragen: Die Option -I fügt einen Suchpfad für Header beim Übersetzen hinzu; -L fügt einen Suchpfad für Bibliotheken beim Linken hinzu; -lgmp fordert eine passende Bibliothek mit dem logischen Namen gmp an. Wird nur gmp.h gefunden, aber -lgmp vergessen, kann die Übersetzung der Quelldatei trotzdem erfolgreich sein, während der Linker später undefined reference auf GMP-Symbole meldet. Die Reihenfolge von Objektdateien und -l-Optionen kann bei statischen Archiven auf bestimmten Linkern relevant sein.



Originalfolie 06, PDF-Seite 5: Einbinden von Bibliotheken: Beispiel GNU MP (ii)

Einbinden von Bibliotheken: Beispiel GNU MP (iii)

- 2a. Binden der Objektdatei schlägt fehl
  - Bibliothek und darin enthaltene Funktionen nicht gefunden

```
gcc -o mathdemo mathdemo.o 2>&1
```

```
/usr/bin/ld: mathdemo.o: in function `main':
mathdemo.c:(.text+0x50): undefined reference to `__gmpz_init_set_str'
/usr/bin/ld: mathdemo.c:(.text+0x6f): undefined reference to `__gmpz_init_set_str'
/usr/bin/ld: mathdemo.c:(.text+0x7b): undefined reference to `__gmpz_init'
/usr/bin/ld: mathdemo.c:(.text+0x92): undefined reference to `__gmpz_mul'
/usr/bin/ld: mathdemo.c:(.text+0xa8): undefined reference to `__gmpz_get_str'
collect2: error: ld returned 1 exit status
```


Originalfolie 06, PDF-Seite 6: Einbinden von Bibliotheken: Beispiel GNU MP (iii)

Einbinden von Bibliotheken: Beispiel GNU MP (iv)

- 2b. Angabe des Suchpfades mit `-L` und der Bibliothek mit `-l` erforderlich
  - Präfix `lib` und Suffix `.so` oder `.a` wird automatisch verwendet
  - Parameter `-lgmp` → Dateiname `libgmp.so`

```
gcc -L/usr/lib/gmp -o mathdemo mathdemo.o -lgmp
ls -nh mathdemo*
```

```
-rwxr-xr-x 1 1000 1000  16K 14. Sep 14:48 mathdemo
-rw-r--r-- 1 1000 1000  468 14. Sep 14:14 mathdemo.c
-rw-r--r-- 1 1000 1000 2.1K 14. Sep 14:48 mathdemo.o
```

- Testlauf

```
./mathdemo 78903490364390378904  437830126749047038790432
```

```
78903490364390378904 * 437830126749047038790432 = 3454632518718325131640205959344470232984652 8
```


Originalfolie 06, PDF-Seite 7: Einbinden von Bibliotheken: Beispiel GNU MP (iv)

8.4 Statische und dynamische Bindung

Beim **statischen Linken** werden die benötigten Bestandteile der verwendeten Bibliotheken in das ausführbare Programm integriert. Das erzeugte Image ist deshalb häufig größer und enthält potentiell Bibliothekscode mehrfach in verschiedenen Programmen. Vorteil ist, dass die benötigten integrierten Funktionen nicht als separate Shared Library vorliegen müssen und bestimmte Laufzeitabhängigkeiten entfallen. Das Folienbeispiel nutzt `gcc -static -o linking-static linking.c` und erhält auf dem gezeigten System ungefähr **816 K** Dateigröße.

Beim **dynamischen Linken** enthält das Programm vor allem seinen eigenen Code sowie Metadaten, die eine passende Shared Library zur Lade- oder Laufzeit auffinden lassen. Mehrere Prozesse können bestimmte schreibgeschützte

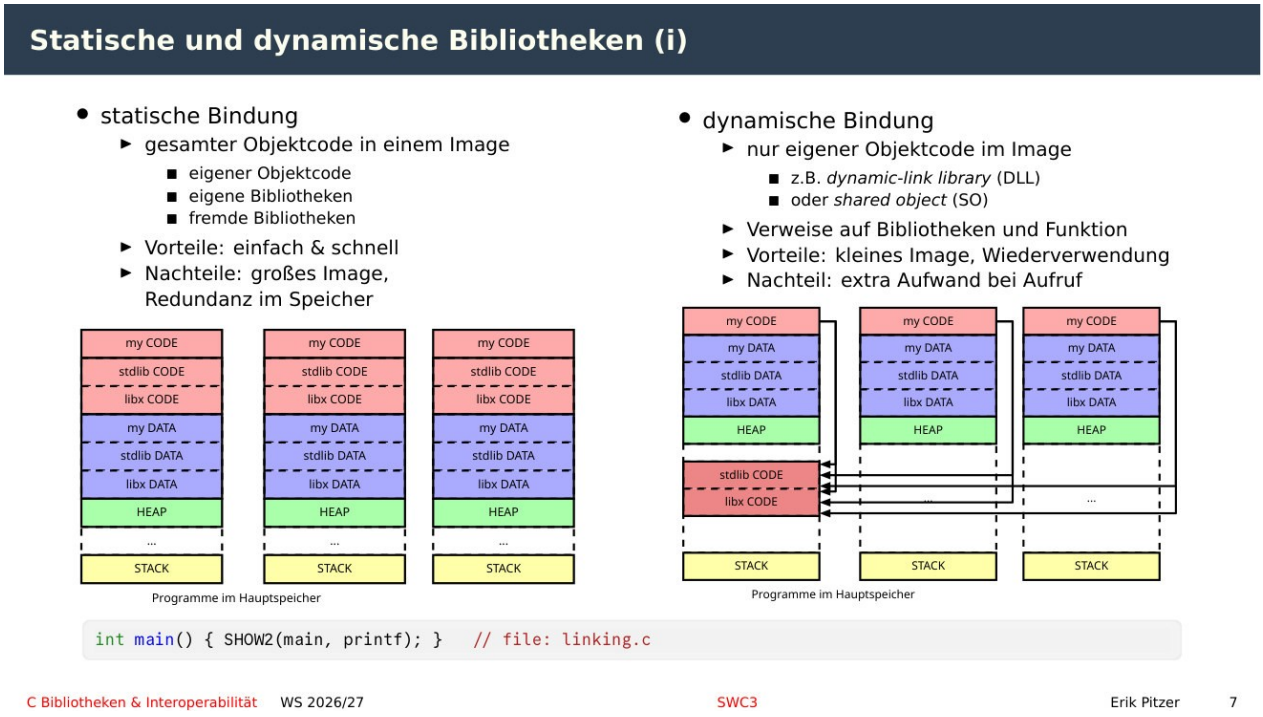
Bibliotheksseiten gemeinsam verwenden; die Programme sind meist kleiner. Im Folienbeispiel entsteht mit `gcc -o linking-dynamic linking.c` eine Datei von ungefähr **16 K**. Die Grafiken zeigen mehrere Programme, die auf gemeinsame Bibliothekscodebereiche verweisen. Dynamische Bindung erfordert passende Bibliotheksversionen und korrektes Laden der Bibliotheksdateien; Funktionsadressen können wegen ASLR zwischen Aufrufen verschieden sein.

Die Folie charakterisiert statische Bindung als tendenziell schnelleren Funktionsaufruf und dynamische als möglicherweise minimal aufwendiger. **Ergänzende Einordnung:** Das ist keine universelle Laufzeitregel: Indirektionskosten hängen von Architektur, Optimierungen und Lazy Binding ab und sind oft im Gesamtprogramm vernachlässigbar. Statische Linkverfahren können ebenfalls Position Independent Executables erzeugen, deren Adressen sich zwischen Prozessen ändern. Entscheidend sind hier die begrifflichen Unterschiede zwischen integriertem Objektcode, extern bereitgestellten Implementierungen, dem Ladeprozess und der Laufzeit-Symbolauflösung.

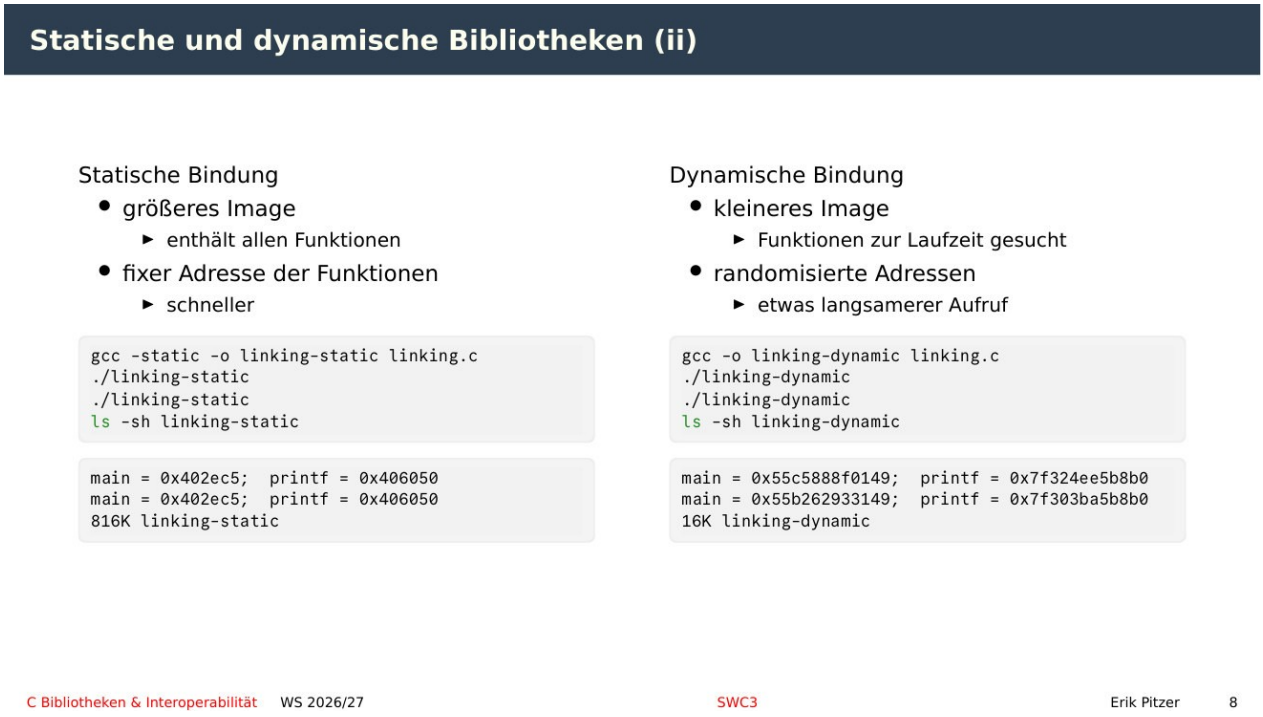
Vertiefung · Schritt für Schritt

Bei statischem Linken werden zur Linkzeit benötigte Bibliotheksanteile in die ausführbare Datei integriert. Dadurch können Programme unabhängig von einer später installierten passenden Shared Library starten, allerdings wachsen Dateigröße und Wartungsaufwand, und Programme nutzen nicht automatisch gemeinsam denselben Bibliothekscode. Beim dynamischen Linken enthält die ausführbare Datei dagegen Referenzen auf eine externe Bibliothek, die beim Laden oder beim ersten Aufruf aufgelöst werden. Das verringert häufig die Dateigröße und erlaubt gemeinsame Speicherabbildungen, führt aber zu Abhängigkeiten von einer kompatiblen Laufzeitbibliothek und ihren Suchpfaden.

Die Folie zeigt unterschiedliche Prozessadressen bei erneuten Programmläufen. Das ist eine anschauliche Wirkung von Adressraumrandomisierung (ASLR), aber kein inhärentes Merkmal von „dynamisch“ allein: Auch statisch gelinkte position-independent executables können randomisierte Ladeadressen besitzen. Ebenso entscheidet die verwendete ABI über den zusätzlichen Indirektionsmechanismus eines Funktionsaufrufs. Die Geschwindigkeitsaussage „statisch grundsätzlich schneller“ ist deshalb eine Vereinfachung des konkreten Beispiels.



Originalfolie 06, PDF-Seite 8: Statische und dynamische Bibliotheken (i)



Originalfolie 06, PDF-Seite 9: Statische und dynamische Bibliotheken (ii)

8.5 C als Sprachbrücke: Java Native Interface (JNI)

C hat für fremde Sprachen eine verbreitete **Application Binary Interface** und wird deshalb oft als Zwischenebene verwendet. Das Folienbeispiel zeigt das **Java Native Interface (JNI)**: In Java wird eine Methode als `private static native int getTime();` deklariert. Das Schlüsselwort `native` bedeutet, dass der Methodenrumpf nicht in Java steht, sondern eine fremde Implementierung bereitgestellt wird. `javac -h . CallC.java` erzeugt einen C-Header mit einer Deklaration in der Art `JNIEXPORT jint JNICALL Java_CallC_getTime(JNIEnv *, jclass);`. Dabei bestimmen `JNIEXPORT` und `JNICALL` Export- und Aufrufkonventionen. `JNIEnv*` erlaubt über die JNI-Schnittstelle die Interaktion mit der JVM; bei einer statischen Methode repräsentiert das zweite Argument die aufrufende Klasse.

In `callc.c` wird der Header `CallC.h` eingebunden und die exakt benannte Implementierungsfunktion definiert. Die Folie lässt sie `printf("Hello C!\n");` aufrufen und als `jint` einen aus `time(nullptr)` stammenden Zeitwert zurückgeben. Das gezeigte Build-Kommando verwendet `gcc -shared`, fügt mit `-I $JAVA_HOME/include` und `-I $JAVA_HOME/include/linux` die JNI-Headerpfade ein und erzeugt `libcallc.so`. In Java lädt `System.loadLibrary("callc")` die Bibliothek; der Aufruf der nativen Methode erfolgt dann wie ein gewöhnlicher Java-Methodenaufruf. Der Programmstart setzt mit `-Djava.library.path=.` das aktuelle Verzeichnis als Bibliothekssuchort und mit `--enable-native-access=ALL-UNNAMED` die entsprechende Java-Konfiguration.

Zwei präzise Ergänzungen: Erstens hängt die tatsächliche Reihenfolge von `Hello Java`, `Hello C!` und `C time is ...` wegen unterschiedlicher Ausgabepuffer von Umgebung und Flush-Verhalten ab; die physische Ausgabereihenfolge in den Folien ist deshalb kein garantierter Ablaufbeweis. Zweitens ist `time_t` nicht grundsätzlich auf den Wertebereich von `jint` beschränkt. Bei einer API mit dauerhafter Zeitstempelverwendung wäre ein passender breiter Java-Typ wie `long` mit entsprechender JNI-Rückgabe zu erwägen. Die Folie demonstriert vorrangig den Weg **Java-Deklaration → generierter Header → C-Implementierung → Shared Library → Java-Laden und Aufruf**.

Vertiefung · Schritt für Schritt

JNI verbindet Quelltext und Binärschnittstelle in einer festen Reihenfolge. Zuerst deklariert die Java-Klasse eine native-Methode, danach erzeugt javac -h eine Headerdeklaration mit `JNIEXPORT`, `JNICALL` sowie `JNIEnv*` und jclass für eine statische Methode. Die C-Implementierung muss exakt diese Signatur einschließlich ihrer Calling Convention verwenden. Beim Übersetzen entsteht mit -shared eine ladbare Bibliothek wie libcallc.so, die Java durch

System.`loadLibrary("callc")` lädt. Die Java-Laufzeit findet sie nur, wenn ihr Bibliothekssuchpfad passend eingerichtet ist. Beim Aufruf wechselt die Ausführung durch die JNI-Bindung zur implementierten C-Funktion und kehrt mit einem `jint` zurück.

Zusätzliche Vorsicht gilt den Datentypen: JNI verwendet definierte Brückentypen wie `jint`; ein C-`time_t` ist nicht automatisch derselbe Typ und kann einen größeren Bereich haben. Ein Aufruf `return time(nullptr);` kann deshalb bei sehr großen Zeitwerten einen Informationsverlust verursachen. Die konkreten Ausgaben und Zeitstempel der Originalfolie sind Laufzeitbeispiele, keine festgelegten Ergebnisse. Für ein reales Projekt gehören außerdem Fehlerbehandlung, Ressourcenfreigabe und eine Prüfung der Java-Native-Access-Einstellungen zur Implementierung.

Interoperabilität (i)

- C wird oft als Brücke zwischen verschiedenen Programmiersprachen genutzt
  - einfache Sprache
  - sehr weit verbreitet
- Beispiel Aufruf von C aus Java mit Hilfe von `native`-Methode

```java
// file: CallC.java
class CallC {
  private static native int getTime();
}
```

- erzeugen der Schnittstelle für C mit Java-Compiler

```
javac -h . CallC.java # -> CallC.h
```

```
// file CallC.h
JNIEXPORT jint JNICALL Java_CallC_getTime(JNIEnv*, jclass);
```

C Bibliotheken & Interoperabilität WS 2026/27 SWC3 Erik Pitzer 9

Originalfolie 06, PDF-Seite 10: Interoperabilität (i)

Interoperabilität (ii)

- Erstellen der Implementierung in C

```c
// file: callc.c
#include "CallC.h" // generated by 'java -h'

#include <stdio.h>
#include <time.h>
#include <jni.h>

// NOTE: ensure Java calling conventions
JNIEXPORT jint JNICALL Java_CallC_getTime(
    [[maybe_unused]] JNIEnv* env,
    [[maybe_unused]] jclass c) {
  printf("Hello C!\n");
  return time(nullptr);
}
```

- Erstellen einer shared library aus C-Code (GCC Option `-shared`)

```
gcc -std=c23 -pedantic -Wall -Wextra \
    -I $JAVA_HOME/include \
    -I $JAVA_HOME/include/linux \
    -shared \
    -o libcallc.so \
    callc.c
```

- Ergebnis ist Datei `libcallc.so`

C Bibliotheken & Interoperabilität WS 2026/27 SWC3 Erik Pitzer 10

Originalfolie 06, PDF-Seite 11: Interoperabilität (ii)

Interoperabilität (iii)

- Laden der shared Library in Java
- Aufruf wie normale Methode

```java
class CallC {

  private static native int getTime();

  static {
    System.loadLibrary("callc");
  }

  public static void main(String[] args) {
    System.out.println("Hello Java");
    int time = getTime();
    System.out.println("C time is " + time);
  }
}
```

- Angabe des Suchpfads für Bibliotheken (`java.library.path`) bei Aufruf[1]

```
javac CallC.java
java --enable-native-access=ALL-UNNAMED \
    -Djava.library.path=. \
    CallC
```

```
Hello Java
C time is 1789390126
Hello C!
```

[1] aktuelles Verzeichnis ist »`.`«

Originalfolie 06, PDF-Seite 12: Interoperabilität (iii)

9 Technischer Anhang und Referenz zu den Foliendiagnosemakros

9.1 Standardheader und Hilfsbibliothek

Die Vorlesungsbeispiele verwenden vielfach Makros wie `SHOW`, `SHOW_TYPE`, `SHOW_XY` und `SHOW_ARRAY_N`. Diese Bezeichner sind **keine C-Standardfunktionen**; sie werden durch die vom Kurs bereitgestellte Hilfsdatei eingebunden (in den Folien erscheinen die Schreibweisen `swc3.h` und `swc.h`, die nicht vollständig konsistent sind). Deshalb funktionieren einzelne kopierte Codebeispiele ohne diesen Header nicht. Das ist eine wichtige Besonderheit beim selbständigen Nachprogrammieren. Die vom Anhang gezeigten Standardheader sind unter anderem `<stdio.h>` für Ein-/Ausgabe, `<limits.h>` und `<float.h>` für Zahlenbereiche, `<stdint.h>` für Integer mit festgelegten Breiten, `<stddef.h>` für Typen wie `size_t`, `<stdarg.h>` für variadische Funktionen, `<ctype.h>` für Zeichenklassifikation, `<complex.h>` für komplexe Zahlen, `<stdlib.h>` für Speicherverwaltung, `<time.h>` für Zeitwerte und `<tgmath.h>` für generische mathematische Funktionen.

Anhang A – C Header & Hilfsfunktionen (i)

- Hilfsfunktionen der Folien wurden in `swc3.h` implementiert und in allen Beispielen (wo nicht anders angegeben) inkludiert

```c
#include <stdio.h>   // printf
#include <limits.h>  // integer value limits
#include <float.h>   // float value limits
#include <stdint.h>  // fixed-size integers
#include <stddef.h>
#include <stdarg.h>  // variadic functions
#include <ctype.h>   // character types
#include <complex.h> // complex numbers
#include <stdlib.h>  // malloc, free, ...
#include <time.h>
#include <tgmath.h> // type-generic math
```

C Anhang WS 2026/27 SWC3 Erik Pitzer 2

Originalfolie 07, PDF-Seite 3: Anhang A – C Header & Hilfsfunktionen (i)

9.2 `_Generic`, Typanzeigen und Ausgabeformatwahl

Die Makros im Anhang nutzen die seit C11 vorhandene **generische Selektion** `_Generic((x), typ1: wert1, typ2: wert2, default: ...)`. Sie erlaubt, abhängig vom Typ eines Ausdrucks zur Übersetzungszeit eine passende Alternative auszuwählen. `TYPENAME(x)` bildet einen Typ auf einen beschreibenden Text ab, zum Beispiel `"double"` oder `"pointer to int"`. `PRINTF_FORMAT_CHAR(X)` wählt einen dazu passenden Formatstring. Die zusammengefügten Makros `SHOW_VALUE(X)` und `SHOW(X)` geben den Ausdruck und seinen Wert aus, während `SHOW_TYPE(X)` zusätzlich den durch `_Generic` ausgewählten Typnamen zeigt.

Ergänzende Warnung: Die Anhangmakros dienen ausdrücklich der Demonstration und besitzen nicht für alle Typen vollständig portable `printf`-Signaturen. Beispielsweise erfordert `%p` einen als `void *` übergebenen Zeiger, während der Anhang teils `%x` für Zeiger verwendet; eine solche Argument-/Formatspezifikation kann undefiniertes Verhalten auslösen. Auch die Formatierung von Dezimalfließkommatypen und komplexen Zahlen ist in den Folien teilweise GCC-

spezifisch. Die `_Generic`-Auswahl kann zudem nur die aufgeführten Typen unterscheiden; für andere Typen wird meist ein `default`-Zweig verwendet. Für eigene Programme empfiehlt sich bei diagnostischen Makros die Auswertung und Übergabe jedes Nebenwirkungsausdrucks **genau einmal** und eine präzise Formattypzuordnung.

Vertiefung · Schritt für Schritt

`_Generic` erlaubt, anhand des Typs eines Ausdrucks eine passende Ersetzung zu wählen. Das Makro `TYPENAME(x)` der Folie ordnet viele Typen lesbaren Namen zu; das Formatmakro `PRINTF_FORMAT_CHAR(X)` versucht, daraus die geeignete `printf`-Konvertierung abzuleiten. Beides ist didaktischer Diagnosecode und keine generelle sichere Serialisierungsbibliothek. Bei `printf` müssen Formatstring und tatsächliche Argumenttypen einschließlich aller Standardargumentpromotions übereinstimmen. Besonders Formate für Objekt- oder Funktionszeiger und komplexe Zahlen sind in den Folien teilweise GCC- beziehungsweise plattformspezifische Tricks.

Anhang A – C Header & Hilfsfunktionen (ii)

```c
// adapted from https://stackoverflow.com/a/17290414
#define TYPENAME(x) _Generic((x),                                                 \
         bool: "bool",                   unsigned char: "unsigned char",          \
         char: "char",                     signed char: "signed char",            \
    short int: "short int",         unsigned short int: "unsigned short int",     \
          int: "int",                     unsigned int: "unsigned int",           \
     long int: "long int",           unsigned long int: "unsigned long int",      \
long long int: "long long int", unsigned long long int: "unsigned long long int", \
        float: "float",                         double: "double",                 \
  long double: "long double",                   char *: "pointer to char",        \
       void *: "pointer to void",                int *: "pointer to int",         \
   _Decimal32: "_Decimal32",            float _Complex: "float _Complex",         \
   _Decimal64: "_Decimal64",           double _Complex: "double _Complex",        \
  _Decimal128: "_Decimal128",     long double _Complex: "long double _Complex",   \
 int(*)(void): "function",                                                        \
 int(*)(const char *restrict, ...): "library function",                           \
      default: "other")
```

C Anhang WS 2026/27 SWC3 Erik Pitzer 3

Originalfolie 07, PDF-Seite 4: Anhang A – C Header & Hilfsfunktionen (ii)

Anhang A – C Header & Hilfsfunktionen (iii)

```c
// NOTE: formatting of _Complex is a hack specific to GCC
#define PRINTF_FORMAT_CHAR(X) _Generic((X),                                 \
           bool: "%d",                    unsigned char: "%hhu",     \
           char: "'%c'",                    signed char: "%hhd",     \
      short int: "%hd",              unsigned short int: "%hu",      \
            int: "%d",                     unsigned int: "%u",       \
       long int: "%ld",               unsigned long int: "%lu",      \
  long long int: "%lld",         unsigned long long int: "%llu",     \
          float: "%g",                           double: "%g",       \
    long double: "%Lg",                          char *: "\"%s\"",   \
         void *: "0x%x",                          int *: "0x%x",     \
     _Decimal32: "%Hf",                  float _Complex: "%g%+gi",   \
     _Decimal64: "%Df",                 double _Complex: "%g%+gi",   \
    _Decimal128: "%DDf",           long double _Complex: "%Lg%+Lgi", \
       int(*)(): "%p",                                               \
  int(*)(const char *restrict, ...): "%p",                           \
        default: "0x%x")
```

```c
#define SHOW_VALUE(X) printf(PRINTF_FORMAT_CHAR(X), (X))
#define SHOW(X)         \
  printf("%s = ", #X); SHOW_VALUE(X); printf("\n")
```

C Anhang WS 2026/27 SWC3 Erik Pitzer 4

Originalfolie 07, PDF-Seite 5: Anhang A – C Header & Hilfsfunktionen (iii)

9.3 Weitere Ausgabemakros, Arrays, Zahlenbereiche und Compilerparameter

`SHOW2(X, Y)` gibt zwei Ausdruckswerte auf einer Zeile aus. `SHOW_XY(P)` ist auf Strukturen mit Mitgliedern `x` und `y` zugeschnitten. `SHOW_COMPLEX(X)` benutzt Funktionen wie `creal` und `cimag`, um Real- und Imaginärteil darzustellen. `SHOW_ARRAY_N(A, N)` iteriert über `N` Elemente eines Arrays und gibt sie aus, `SHOW_POINTS_N` formatiert eine Folge von Punktstrukturen. `SHOW_DBL` und `SHOW_D32` demonstrieren die Formatierung binärer beziehungsweise dezimaler Fließkommatypen mit hoher Genauigkeit. Weitere Hilfen `SHOW_NUMBER` und `SHOW_FLOAT` machen Bitdarstellungen und Felder eines konstruierten Float-Modells sichtbar; `SHOW_HEX_CHARS` zeigt den Inhalt eines Bytearrays als Hexwerte und, wenn druckbar, als Zeichen. `SHOW_RANGE_S` und `SHOW_RANGE_U` drucken Speicherbreiten sowie Integer-Grenzwerte.

Der Anhang zeigt für die Ausführung der Beispiele eine ausführliche GCC-Kommandozeile mit `-std=c23`, `-pedantic`, den Warnungen `-Wall -Wextra -Wconversion -Wshadow`, Sanitizer-Optionen `-fsanitize=address,leak,undefined`, `-ftrivial-auto-var-init=pattern` sowie zusätzlichen Bibliotheksoptionen `-lasan -lubsan -lm -ldfp`. Diese Optionen hängen vom Compiler und der installierten Toolchain ab. Insbesondere ist `-ftrivial-auto-var-init=pattern` ein Diagnose- beziehungsweise Härtungsmechanismus und **kein Ersatz** für die vorgeschriebene Initialisierung lokaler Objekte. Die in einzelnen Folien sichtbaren Musterwerte von uninitialisierten Variablen sind keine Spracheigenschaft. Die Originalfolien zu allen Diagnosemakros sind bewusst vollständig im Anhang abgedruckt, damit ihre Quellcodekonventionen nachvollziehbar bleiben.

Vertiefung · Schritt für Schritt

Die Hilfsmakros `SHOW`, `SHOW2`, `SHOW_ARRAY_N` und `SHOW_POINTS_N` sind aus mehreren `printf`-Aufrufen zusammengesetzt. Insbesondere ein Argument mit Seiteneffekten, etwa i++, sollte nicht unbedacht an Makros übergeben werden, die es mehrfach auswerten. Das Makro `SHOW_HEX_CHARS` ist zur Sichtbarmachung von Bytes geeignet, sagt aber nichts über die semantische Gültigkeit eines selbst konstruierten Binärformats aus. Die Compilerparameter der Anhangsfolie aktivieren Warnungen sowie mehrere Instrumentierungen. Abhängig von GCC-Version, Plattform und installierten Bibliotheken kann die genaue Befehlszeile Anpassungen benötigen.

Anhang A – C Header & Hilfsfunktionen (iv)

```c
#define SHOW2(X, Y)                       \
  printf("%s = ", #X); SHOW_VALUE(X);     \
  printf(";  %s = ", #Y);  SHOW_VALUE(Y); \
  printf("\n")
```

```c
#define SHOW_XY(P)      \
  printf("%s = (", #P); \
  SHOW_VALUE((P).x);    \
  printf(", ");         \
  SHOW_VALUE((P).y);    \
  printf(")\n")
```

```c
#define SHOW_TYPE(X)             \
  printf("%s = ", #X);           \
  SHOW_VALUE(X);                 \
  printf(" (%s)\n", TYPENAME(X))
```

```c
#define SHOW_COMPLEX(X) printf("%s = %g+%gi\n", #X, creal(X), cimag(X))
```

C Anhang WS 2026/27 SWC3 Erik Pitzer 5

Originalfolie 07, PDF-Seite 6: Anhang A – C Header & Hilfsfunktionen (iv)

Anhang A – C Header & Hilfsfunktionen (v)

```c
#define SHOW_ARRAY_N(A, N)                        \
  printf("%s = {", #A);                           \
  for (size_t i = 0; i<(N); i++) {                \
    if (i > 0) printf(", ");                      \
    printf(PRINTF_FORMAT_CHAR((A)[i]), ((A)[i])); \
  }                                               \
  printf("}\n")
```

```c
#define SHOW_POINTS_N(A, N)        \
  printf("%s = {", #A);            \
  for (size_t i = 0; i<(N); i++) { \
    if (i > 0) printf(", ");       \
    printf("(");                   \
    SHOW_VALUE((A)[i].x);          \
    printf(",");                   \
    SHOW_VALUE((A)[i].y);          \
    printf(")");                   \
  }                                \
  printf("}\n")
```

```c
#define SHOW_DBL(X) printf("    double: %.25f\n", (X))
#define SHOW_D32(X) printf("_Decimal32: %.25Hf\n", (X))
```

C Anhang WS 2026/27 SWC3 Erik Pitzer 6

Originalfolie 07, PDF-Seite 7: Anhang A – C Header & Hilfsfunktionen (v)

Anhang A – C Header & Hilfsfunktionen (vi)

```c
#define SHOW_NUMBER(N) printf("%s = {.f=% 4.2f, .i=0b%032b}\n", #N, (N).f, (N).i);

#define SHOW_FLOAT(F) printf("%s = {.value=% 4.2f .sign=%c .exp=0x%02x .mant=0x%06x}\n", \
                           #F, (F).value, (F).sign == 0 ? '+' : '-', (F).exp, (F).mant);
```

```c
#include <locale.h>

#define SHOW_RANGE(name, min, max, format)                      \
  printf("=%s= \t %2zu \t %'25" #format " \t %'25" #format "\n", \
         #name, sizeof(name)*8, min, max)

#define SHOW_RANGE_S(name, abbr, format) SHOW_RANGE(name, abbr ## _MIN, abbr ## _MAX, format)
#define SHOW_RANGE_U(name, abbr, format) SHOW_RANGE(name,            0, abbr ## _MAX, format)
```

```c
#define SHOW_HEX_CHARS(C)                                       \
  for (size_t i = 0; i<sizeof((C)); ++i) {                      \
    if (i > 0 && i % 4 == 0) printf("\n");                      \
    uint8_t c = (C)[i];                                         \
    printf("%2zu: 0x%02x", i, c);                               \
    if (isprint(c)) printf(" '%c'", c); else printf("  ? ");   \
    printf(" | ");                                              \
  }
```


Originalfolie 07, PDF-Seite 8: Anhang A – C Header & Hilfsfunktionen (vi)

Anhang B – C Compiler-Parameter

- C Beispiele in den Folien wurden (falls nicht anders angegeben) mit folgendem Kommando übersetzt

```
gcc -std=c23 -pedantic \
 -Wall -Wextra -Wconversion -Wshadow \
 -fsanitize=address,leak,undefined -ftrivial-auto-var-init=pattern  \
 -lasan -lubsan -lm -ldfp \
 -o example.exe example.c
```

- Zusätzlich wurde unsichtbar bei jedem Beispiel der Header `swc.h` inkludiert

```c
#include "swc.h"
```


Originalfolie 07, PDF-Seite 9: Anhang B – C Compiler-Parameter

10 Querverbindungen und praktische Lernbeispiele

10.1 Ein Zeiger- und Speicherbeispiel vollständig begründet

Das folgende C23-Programm verbindet die Kernbegriffe Array, `size_t`, dynamischen Speicher, Funktionsparameter, Header und Lebensdauer. Es benutzt keine besonderen Folienmakros.

```c
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

int sum_array(size_t n, const int a[static n]) {
    int result = 0;
    for (size_t i = 0; i < n; ++i) {
        result += a[i];
    }
    return result;
}

int main(void) {
    const size_t n = 3;
    int *a = calloc(n, sizeof *a);
    if (a == nullptr) return EXIT_FAILURE;
    a[0] = 5;
    a[1] = 7;
    a[2] = 9;
    printf("Summe: %d\n", sum_array(n, a));
    free(a);
    return EXIT_SUCCESS;
}
```

`calloc(n, sizeof *a)` berechnet die Speichermenge für drei `int`-Elemente und initialisiert den Block byteweise mit null. Der Nullzeigertest verhindert den anschließenden Zugriff bei fehlgeschlagener Allokation. `a` ist eine automatische Zeigervariable, während die allokierten Integerobjekte unabhängig davon dynamische Speicherdauer besitzen, bis `free` aufgerufen wird. Der Parameter `const int a[static n]` passt zu einem `const int *`-Parameter mit einer Mindestlängenbedingung. Innerhalb der Funktion sind die gültigen Indizes 0 bis 2, und `result` erhält die Summe 21. Ein möglicher zusätzlicher Schutz bestünde in der Prüfung, ob die Summe in `int` darstellbar bleibt. Das Programm trifft hier aufgrund der kleinen konkreten Werte eine zulässige Annahme. `free(a)` beendet die dynamische Allokation; der automatische Parameterzeiger in `sum_array` wurde bereits bei Funktionsrückkehr entfernt.

10.2 `struct` und externe Binärdaten: Warum der Unterschied wichtig ist

In einem Datenaustauschformat wie dem verkürzten Ogg-Beispiel müssen Nutzbytes in einer **fest definierten Reihenfolge** erscheinen. Ein C-`struct` kann dagegen Padding aufweisen und kann mehrbyteige Zahlen nach der Endianness des Prozessors speichern. Ein Wert wie `uint16_t x=0xABCD` wird auf einem Little-Endian-System als Bytes `CD AB` abgelegt und auf Big-Endian-Systemen üblicherweise als `AB CD`. Die korrekte portable Kodierung für Little Endian kann gezielt als `(uint8_t)(x & 0xFF)` und `(uint8_t)((x >> 8) & 0xFF)` erzeugt werden. Die Masken wählen jeweils acht Bits aus; das Rechtsschieben um 8 Stellen legt das höhere Byte frei. Damit wird die Externrepräsentation nicht vom zufälligen `struct`-Layout abhängig gemacht.

10.3 Wann eine Warnung wirklich einen Fehler verrät

Die Warnung bei `sizeof` eines Arrayparameters weist darauf hin, dass ein Zeiger gemessen wird. Die Warnung `-Wconversion` kann auf Zahlen hinweisen, die in einem schmaleren Zieltyp Informationen verlieren. `-Wshadow` meldet, wenn eine lokale Variable einen äußeren Namen verdeckt. Linkerfehler „undefined reference“ deuten meist darauf hin, dass eine Implementierung oder ein erforderliches Bibliotheksargument beim Linken fehlt. Ein Sanitizer-Befund „heap-use-after-free“ zeigt typischerweise, dass ein zuvor mit `free` freigegebener Speicherbereich noch einmal benutzt wurde. Die Diagnose ist jeweils anhand von **Compilerphase, Typregeln, Lebensdauer und Eigentumsregeln** zu verstehen; diese vier Konzepte verknüpfen die großen Kapitel des Lehrstoffs.

11 Quellenhinweise und dokumentierte Korrekturen

Die fachliche Grundlage sind die vom Kurs bereitgestellten Folien **SWC3 – Software-Entwicklung mit C und C++**, Erik Pitzer, WS 2026/27, FH OÖ Campus Hagenberg. Die C-Foliensätze sind hier mit ihren Original-PDF-Seiten im Text dargestellt. Die C++-Lehre wurde aus dem Kursüberblick nicht übernommen. Der Abschnitt „Technischer Anhang“ dokumentiert die kursinternen `SHOW`-Makros, weil zahlreiche Versuchsausgaben andernfalls nicht unabhängig nachvollziehbar wären.

Wesentliche **gekennzeichnete fachliche Präzisierungen** betreffen: Unterschiede von Deklaration und Definition; nullterminierte Stringliterale und deren Qualifikation; fehlende automatische Arraygrenzenprüfung; Array/Zeiger-Unterschied; Portabilität von Binärrepräsentationen; `const` und Casts; signed Integerüberlauf; `volatile` ohne Thread-Synchronisation; nicht spezifizierte Auswertungsreihenfolge im Gegensatz zu undefiniertem Verhalten; Rückgabe eines lokalen Zeigers; Stackframe- und Registervereinfachungen; korrekte Zeichenketten- und Formatargumente in `printf`; Ressourcenbereinigung in GMP; die Bedeutung von `calloc` und die Größenberechnung großer `int`-Arrays. Diese Erläuterungen korrigieren offensichtliche Stellen der Folien oder ergänzen explizit die notwendigen Gültigkeitsbedingungen. Weitere geprüfte Präzisierungen betreffen die vollständige IEEE-754-Normalzahlenformel, die Trennung von Präzedenz und Operandenauswertung, die Datenbreitenangaben von Stack- und Heapbeispielen, die Rückgabewerte und Bereinigung von GMP-Objekten sowie die Kontextabhängigkeit von ASLR und Calling Conventions.

12 Folienabgleich und Bearbeitungsumfang

Die nachstehende Übersicht wird automatisch aus der Zuordnung von Originalfolien zu den Kapiteln erzeugt. Die Seitenzahlen bezeichnen **PDF-Seiten der Originaldateien** und nicht die im Fußbereich der Lehrfolien teilweise um eins versetzte interne Folienzählung. Jede hier als fachlich relevante Bildfolie angegebene Seite wurde im zugehörigen Abschnitt vollständig als Originalabbildung übernommen. Für reine Titelfolien, Abschnittstrenner und organisatorische Angaben ist dies im Abgleich ausdrücklich vermerkt. Der Abgleich dokumentiert die Zuordnung, ersetzt aber nicht die Unterscheidung zwischen den Aussagen der Vorlesung und den ergänzenden Präzisierungen in diesem Text.

12.1 Gesamtübersicht der Originalfolien

In den Dokumenten liegen **148 PDF-Seiten** vor. Davon sind **128 fachliche Originalfolien** mit Originalbild und thematischer Erläuterung in diesem Skript erfasst. Die übrigen Seiten sind Deckblätter, reine Kapiteltrenner oder organisatorische Folien. Im Überblick wird der eigenständige C++-Lernteil nicht behandelt.

12.2 01-Überblick.pdf – 7 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	1. SWC3 Überblick	Kursmotto; organisatorischer Kontext
3	Klassische Programmiersprachen — Timeline	Entwurfsziele und typische Einsatzfelder
4	Interessenskonflikte & Spannungsfelder	Entwurfsziele und typische Einsatzfelder
5	Motivation für C und C++	Entwurfsziele und typische Einsatzfelder
6	Ablauf	Ablauf: C-Teil in diesem Skript; C++-Teil ausgenommen
7	Beurteilung	Beurteilungsinformationen; nicht fachlich

12.3 02-C-Einführung(1).pdf – 39 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	2. C Einführung	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
3	Geschichte von C	Eigenschaften von C und Unterschied zu Java
4	Eigenschaften von C (i)	Eigenschaften von C und Unterschied zu Java
5	Eigenschaften von C (ii)	Eigenschaften von C und Unterschied zu Java
6	Hauptunterschiede zu Java	Eigenschaften von C und Unterschied zu Java

PDF-Seite	Originalthema / Einordnung	Behandelt in
7	Popularität von C	Eigenschaften von C und Unterschied zu Java
8	Literatur	Eigenschaften von C und Unterschied zu Java
9	Hello World	Das erste vollständige C-Programm
10	Hello World v2 (i)	Das erste vollständige C-Programm
11	Hello World v2 (ii)	Das erste vollständige C-Programm
12	Hello World v2 (iii)	Das erste vollständige C-Programm
13	Hello World v2 (iv)	Das erste vollständige C-Programm
14	Schlüsselwörter (C23)	Schlüsselwörter, Arten von Datentypen und Zeichenketten
15	Datentypen Überblick	Schlüsselwörter, Arten von Datentypen und Zeichenketten
16	Ganze Zahlen	Ganzzahlen, Größen und Wertebereiche
17	Fließkommazahlen & Complexe Zahlen	Gleitkommazahlen, Dezimalzahlen und komplexe Zahlen
18	Keine fixen Wertebereiche für Zahlen	Ganzzahlen, Größen und Wertebereiche
19	Wertebereiche Beispiel — für GCC und x86_64 unter Linux	Ganzzahlen, Größen und Wertebereiche
20	Datentypen mit fixer Größe	Ganzzahlen, Größen und Wertebereiche
21	Literale	Literale, Typumwandlungen und Bezeichner
22	Typumwandlung (type cast) (i)	Literale, Typumwandlungen und Bezeichner
23	Bezeichner (identifier)	Literale, Typumwandlungen und Bezeichner
24	Deklaration und Definition	Deklarationen, Definitionen, Initialisierung und Konstanten
25	Deklaration und Definition – Beispiel: Variable	Deklarationen, Definitionen, Initialisierung und Konstanten
26	Deklaration und Definition – Beispiel: Funktion	Deklarationen, Definitionen, Initialisierung und Konstanten
27	Deklaration und Definition – Programmierstil	Deklarationen, Definitionen, Initialisierung und Konstanten
28	Konstanten (i)	Deklarationen, Definitionen, Initialisierung und Konstanten
29	Konstanten (ii)	Deklarationen, Definitionen, Initialisierung und Konstanten
30	Weitere Zusätze	volatile, restrict, Scope, Lebensdauer und Speicherklassen
31	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (i)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
32	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (ii)	volatile, restrict, Scope, Lebensdauer und Speicherklassen

PDF-Seite	Originalthema / Einordnung	Behandelt in
33	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (iii)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
34	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (iv)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
35	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (v)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
36	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (vi)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
37	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (vii)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
38	Gültigkeitsbereiche (scope) & Lebensdauer (lifetime) (viii)	volatile, restrict, Scope, Lebensdauer und Speicherklassen
39	Speicherklassen (storage classes)	volatile, restrict, Scope, Lebensdauer und Speicherklassen

12.4 03-C-Grundlagen.pdf – 50 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	3. C Grundlagen	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
3	3.1 C Grundlagen: Benutzerdefinierte Datentypen	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
4	Aufzählungen (enum)	Aufzählungen (enum)
5	Verbunddatentypen (struct) (i)	Strukturen (struct), Initialisierung und Speicherlayout
6	Verbunddatentypen (struct) (ii)	Strukturen (struct), Initialisierung und Speicherlayout
7	Vereinigungstypen (union)	Unions, gemeinsame Speicherbereiche und Bitfelder
8	Zeiger (pointer)	Zeiger und const-Qualifikation
9	Konstante Zeiger	Zeiger und const-Qualifikation
10	Felder (arrays)	Arrays: Länge, Initialisierung und Indexzugriffe
11	Felder sind “roh”	Arrays: Länge, Initialisierung und Indexzugriffe
12	Felder: Initialisierung	Arrays: Länge, Initialisierung und Indexzugriffe
13	Felder und C-Zeichenketten	Arrays: Länge, Initialisierung und Indexzugriffe
14	Felder mit variabler Länge	Variable-length arrays und Array-to-pointer decay
15	Felder zerfallen bei Parameterübergabe (i)	Variable-length arrays und Array-to-pointer decay
16	Felder zerfallen bei	Variable-length arrays und Array-to-

PDF-Seite	Originalthema / Einordnung	Behandelt in
	Parameterübergabe (ii)	pointer decay
17	Felder sind Zeiger	Variable-length arrays und Array-to-pointer decay
18	Benutzerdefinierte Typ-Namen (typedef)	typedef und Binärformat-Beispiel Ogg
19	Beispiel Ogg-Header (gekürzt)	typedef und Binärformat-Beispiel Ogg
20	3.2 C Grundlagen: Ausdrücke	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
21	Ausdrücke (expressions) & Operatoren – Überblick	Ausdrücke und die wichtigsten Operatoren
22	Ausdrücke – Arithmetik, Logik, Zuweisungen, Vergleiche	Ausdrücke und die wichtigsten Operatoren
23	Ausdrücke – Weitere Operatoren	Ausdrücke und die wichtigsten Operatoren
24	Reihenfolge der Operator-Auswertung (precedence)	Operatorpräzedenz, Assoziativität und Auswertungsreihenfolge
25	Reihenfolge der Auswertung von Operanden	Operatorpräzedenz, Assoziativität und Auswertungsreihenfolge
26	Arithmetische Ausdrücke mit schmalen Datentypen	Integer Promotions und übliche arithmetische Umwandlungen
27	Arithmetische Ausdrücke mit unterschiedlichen Datentypen (i)	Integer Promotions und übliche arithmetische Umwandlungen
28	Arithmetische Ausdrücke mit unterschiedlichen Datentypen (ii)	Integer Promotions und übliche arithmetische Umwandlungen
29	3.3 C Grundlagen: Anweisungen & Kontrollfluss	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
30	Einfache Anweisungen	Anweisungen, Blöcke und Verzweigungen
31	Verzweigungen	Anweisungen, Blöcke und Verzweigungen
32	Schleifen	Schleifen, break, continue, goto und longjmp
33	Schleifenabbruch	Schleifen, break, continue, goto und longjmp
34	Sprünge mit goto	Schleifen, break, continue, goto und longjmp
35	Sprünge mit longjmp()	Schleifen, break, continue, goto und longjmp
36	3.4 C Grundlagen: Funktionen	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
37	Deklaration vs. Definition	Deklarationen, Definitionen und Funktionsprototypen
38	Parameterübergabe (i)	Call-by-value und Änderungen über Zeiger

PDF-Seite	Originalthema / Einordnung	Behandelt in
39	Parameterübergabe (ii)	Call-by-value und Änderungen über Zeiger
40	Parameterübergabe (iii)	Call-by-value und Änderungen über Zeiger
41	Parameterübergabe (iv)	Call-by-value und Änderungen über Zeiger
42	Variable Parameteranzahl	Variadische Funktionen und Rückgabewerte
43	Rückgabe von Werten	Variadische Funktionen und Rückgabewerte
44	Hauptfunktion (main)	Die Hauptfunktion main und Programmargumente
45	Ausgabefunktion printf (i)	printf: Formatzeichen, Breite und Genauigkeit
46	Ausgabefunktion printf (ii)	printf: Formatzeichen, Breite und Genauigkeit
47	Ausgabefunktion printf (iii)	printf: Formatzeichen, Breite und Genauigkeit
48	Ausgabefunktion printf (iv)	printf: Formatzeichen, Breite und Genauigkeit
49	Funktionszeiger	Funktionszeiger und Callback-Funktionen
50	Undefiniertes Verhalten (undefined behavior)	Undefiniertes, nicht spezifiziertes und implementationsdefiniertes Verhalten

12.5 04-C-Übersetzung.pdf – 16 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	4. C Übersetzung	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
3	Aufteilung des Quelltexts	Header und Implementierungseinheiten
4	Übersetzungsvorgang – Überblick (i)	Acht logische Übersetzungsphasen
5	Übersetzungsvorgang – Überblick (ii)	Acht logische Übersetzungsphasen
6	Übersetzungsvorgang Schematisch	Acht logische Übersetzungsphasen
7	Präprozessor (i)	Präprozessor: Textersetzung, Makros und Konditionen
8	Präprozessor: Konstanten und Makros (i)	Präprozessor: Textersetzung, Makros und Konditionen
9	Präprozessor: Konstanten und Makros (ii)	Präprozessor: Textersetzung, Makros und Konditionen
10	Präprozessor: Konstanten und Makros (iii)	Präprozessor: Textersetzung, Makros und Konditionen

PDF-Seite	Originalthema / Einordnung	Behandelt in
11	Präprozessor: PASCAL →C	Präprozessor: Textersetzung, Makros und Konditionen
12	Präprozessor: Vordefinierte Symbole	Bedingte Übersetzung, eingebaute Makros und Include-Guards
13	Compiler (i)	Objektdateien, Symboltabellen und Linken
14	Compiler (ii)	Objektdateien, Symboltabellen und Linken
15	Binden (linking)	Objektdateien, Symboltabellen und Linken
16	Beispiel Gnu Compiler Collection (GCC)	Objektdateien, Symboltabellen und Linken

12.6 05-C-Speicherverwaltung.pdf – 15 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	5. C Speicherverwaltung	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
3	Überblick Speichersegmente	Überblick über Code, Data, BSS, Heap und Stack
4	Code Segment	Ausführbarer Code und gemeinsam genutzte Bibliotheken
5	Code Segmente von gemeinsamen Bibliotheken	Ausführbarer Code und gemeinsam genutzte Bibliotheken
6	Daten-Segment	Globale Objekte und lokale statische Variablen
7	Stack-Segment (i)	Stackframes, Funktionsaufrufe und ABI
8	Stack-Segment (ii)	Stackframes, Funktionsaufrufe und ABI
9	Stack-Segment (iii)	Lebensdauer lokaler Variablen, Dangling Pointer und Rekursion
10	Parameterübergabe	Stackframes, Funktionsaufrufe und ABI
11	Rekursion	Lebensdauer lokaler Variablen, Dangling Pointer und Rekursion
12	Heap-Segment und manuelle Speicherverwaltung (i)	Dynamischer Speicher mit malloc, calloc und free
13	Heap-Segment und manuelle Speicherverwaltung (ii)	Dynamischer Speicher mit malloc, calloc und free
14	Heap-Segment und manuelle Speicherverwaltung (iii)	Dynamischer Speicher mit malloc, calloc und free
15	Sanitizer & Memory Profiler	Sanitizer und Memory-Profiling

12.7 06-C-Bibliotheken.pdf – 12 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	6. C Bibliotheken & Interoperabilität	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
3	Bibliotheken	Aufbau einer Bibliothek
4	Einbinden von Bibliotheken: Beispiel GNU MP (i)	GMP-Praxisbeispiel: Initialisieren, Multiplizieren, Ausgabe und Aufräumen
5	Einbinden von Bibliotheken: Beispiel GNU MP (ii)	Header-Suchpfad, Bibliothekssuchpfad und Linkerfehler
6	Einbinden von Bibliotheken: Beispiel GNU MP (iii)	Header-Suchpfad, Bibliothekssuchpfad und Linkerfehler
7	Einbinden von Bibliotheken: Beispiel GNU MP (iv)	Header-Suchpfad, Bibliothekssuchpfad und Linkerfehler
8	Statische und dynamische Bibliotheken (i)	Statische und dynamische Bindung
9	Statische und dynamische Bibliotheken (ii)	Statische und dynamische Bindung
10	Interoperabilität (i)	C als Sprachbrücke: Java Native Interface (JNI)
11	Interoperabilität (ii)	C als Sprachbrücke: Java Native Interface (JNI)
12	Interoperabilität (iii)	C als Sprachbrücke: Java Native Interface (JNI)

12.8 07-C-Anhang.pdf – 9 PDF-Seiten

PDF-Seite	Originalthema / Einordnung	Behandelt in
1	Deckblatt (organisatorisch)	Deckblatt; keine eigenständigen fachlichen Inhalte
2	7. C Anhang	Kapitel- oder Abschnittstrenner; keine eigenständigen fachlichen Inhalte
3	Anhang A – C Header & Hilfsfunktionen (i)	Standardheader und Hilfsbibliothek
4	Anhang A – C Header & Hilfsfunktionen (ii)	_Generic, Typanzeigen und Ausgabeformatwahl
5	Anhang A – C Header & Hilfsfunktionen (iii)	_Generic, Typanzeigen und Ausgabeformatwahl
6	Anhang A – C Header & Hilfsfunktionen (iv)	Weitere Ausgabemakros, Arrays, Zahlenbereiche und Compilerparameter
7	Anhang A – C Header &	Weitere Ausgabemakros, Arrays,

	PDF-Seite	Originalthema / Einordnung	Behandelt in
		Hilfsfunktionen (v)	Zahlenbereiche und Compilerparameter
	8	Anhang A – C Header & Hilfsfunktionen (vi)	Weitere Ausgabemakros, Arrays, Zahlenbereiche und Compilerparameter
	9	Anhang B – C Compiler-Parameter	Weitere Ausgabemakros, Arrays, Zahlenbereiche und Compilerparameter